ANTI-GRAVITY AGENT



Author: Mahdieh Mohammadi
Supervisor: Mehrdad Hassanzadeh

درباره این کتابچه

این کتابچه با هدف آشنایی با Google Antigravity و آموزش اصول استفاده‌ی عملی و ایمن از قابلیت‌های آن در پروژه‌های نرم‌افزاری تهیه شده است. محتوای آن بر پایه‌ی مستندات رسمی گوگل و با تمرکز بر محیط کاری، نحوه‌ی تعامل با ایجنت، مدیریت وظایف، بررسی و ویرایش کد، ابزارهای کمکی، تنظیمات، امنیت و قابلیت‌های پیشرفته‌ی این پلتفرم تنظیم شده است.

با توجه به توسعه‌ی مداوم Google Antigravity ، ممکن است برخی قابلیت‌ها، نام گزینه‌ها، تنظیمات، مدل‌های پشتیبانی‌شده و جزئیات محیط کار در نسخه‌های جدید تغییر کنند. بنابراین، مطالب وابسته به نسخه باید با مستندات رسمی و نسخه‌ی مورد استفاده‌ی کاربر تطبیق داده شوند.

این کتابچه جایگزین مستندات رسمی، بررسی انسانی کد یا رعایت اصول امنیتی در پروژه‌های نرم‌افزاری نیست. مسئولیت بررسی خروجی‌های ایجنت، ارزیابی تغییرات، اجرای تست‌ها، حفاظت از اطلاعات حساس و تأیید نهایی عملیات بر عهده‌ی کاربر است. هدف این کتابچه آن است که خواننده بتواند با شناخت بهتر قابلیت‌ها و محدودیت‌های Antigravity، از آن به‌عنوان ابزاری برای افزایش بهره‌وری استفاده کند و در عین حال، کنترل و نظارت خود را بر فرایند توسعه‌ی نرم‌افزار حفظ کند.

bitwiti

فهرست مطالب

پیشگفتار

فصل اول: آشنایی با Google Antigravity

۱.۱ Antigravity .چیست و چه کاربردی دارد؟

۱.۲ قابلیت‌های اصلی ایجنت

۱.۳ .حوزه‌های کاربردی و موارد استفاده از Antigravity

فصل دوم: محیط کار و راه‌اندازی پروژه

۲.۱ .آشنایی با محیطAntigravity

۲.۲ .نصب و پیش‌نیازها

۲.۳ .ساخت و باز کردن پروژه

۲.۴ .آشنایی با پنل ایجنت، ویرایشگر و ترمینال

۲.۵ .انتخاب محیط کار و مدیریت پروژه‌ها

فصل سوم: کار با ایجنت و چرخه‌ی انجام وظایف

۳.۱ .نوشتن درخواست دقیق و تعیین هدف

۳.۲ .برنامه‌ریزی و اجرای وظایف

۳.۳ .تعامل با ایجنت و هدایت مراحل کار

۳.۴ .بررسی کد و فایل‌های تغییرکرده

۳.۵ .اجرا، تست و رفع خطا

۳.۶ .تأیید نهایی و ثبت تغییرات باGit

فصل چهارم: قابلیت‌ها و ابزارهای اصلی

۴.۱ .خواندن، ایجاد و ویرایش فایل‌ها

۴.۲ .اجرای دستورات ترمینال

bitwitit

۴.۳ .کار با مرورگر و بررسی نتایج

۴.۴ .استفاده از Artifacts و گزارش‌های ایجنت

۴.۵ .مدیریت وظایف و اجرای موازی ایجنت‌ها

فصل پنجم: تنظیمات و شخصی‌سازی ایجنت

۵.۱ .تنظیمات اصلی ایجنت

۵.۲ .انتخاب مدل و تنظیمات استدلال

۵.۳ .قوانین پروژه (Rules) و فایلAGENTS.md

۵.۴ .مهارت‌ها (Skills) و گردش‌کارهای قابل استفاده‌ی مجدد

۵.۵ .مدیریت زمینه و اطلاعات موردنیاز ایجنت

فصل ششم: ابزارهای اتصال و قابلیت‌های پیشرفته

۶.۱ .پروتکل MCP و اتصال ابزارهای خارجی

۶.۲ .اتصال سرویس‌ها و منابع اطلاعاتی

۶.۳ .افزونه‌ها و ابزارهای تکمیلی

۶.۴ .کاربردهای پیشرفته و خودکارسازی وظایف

فصل هفتم: امنیت و کنترل دسترسی

۷.۱ .مجوزهای دسترسی ایجنت

۷.۲ Sandbox .و محیط اجرای ایزوله

۷.۳ .بررسی دستورات پیش از اجرا

۷.۴ .حفاظت از فایل‌های حساس و اطلاعات محرمانه

۷.۵ .خطرات استفاده‌ی خودکار از ایجنت

فصل هشتم: خطاها، محدودیت‌ها و اصول استفاده‌ی صحیح

۸.۱ .خطا در درک درخواست و برنامه‌ریزی

۸.۲ .خطاهای کدنویسی، اجرا و وابستگی‌ها

bitwiti.ir

۸.۳ .تغییرات ناخواسته و مشکلات محیط پروژه

۸.۴ .مدیریت زمینه و رفع خطاهای تکرارشونده

۸.۵ .بررسی خروجی و بازگردانی تغییرات

۸.۶ .اصول طلایی کار باAntigravity

فصل نهم: جمع‌بندی و واژه‌نامه

۹.۱ .مرور نکات کلیدی کتابچه

۹.۲ .اصول استفاده‌ی مؤثر و ایمن ازAntigravity

۹.۳ .مسیر پیشنهادی برای یادگیری بیشتر

۹.۴ . واژه‌نامه‌ی اصطلاحات تخصصی

سخن پایانی

bitwithit

پیشگفتار

با گسترش ابزارهای هوش مصنوعی، شیوه‌ی توسعه‌ی نرم‌افزار در حال تغییر است. ایجنت‌های برنامه‌نویسی می‌توانند فراتر از پاسخ‌گویی به پرسش‌ها، در برنامه‌ریزی وظایف، بررسی پروژه‌ها، نوشتن و ویرایش کد، اجرای دستورات و بررسی نتایج مشارکت کنند. با این حال، استفاده‌ی مؤثر از این ابزارها به شناخت قابلیت‌ها، محدودیت‌ها و شیوه‌ی صحیح تعامل با آن‌ها نیاز دارد.

این کتابچه با هدف معرفی و آموزش کاربردی "Google Antigravity" تهیه شده است؛ پلتفرمی مبتنی بر ایجنت که با هدف تسهیل فرایند توسعه‌ی نرم‌افزار طراحی شده است. در این کتابچه تلاش شده است مهم‌ترین قابلیت‌ها و بخش‌های این پلتفرم، از راه‌اندازی پروژه و تعامل با ایجنت تا مدیریت وظایف، استفاده از ابزارها، شخصی‌سازی، امنیت و رفع خطا، به زبانی ساده و ساختاری منظم بررسی شوند.

این کتابچه برای افرادی مناسب است که می‌خواهند با Antigravity آشنا شوند و استفاده از آن را در پروژه‌های نرم‌افزاری آغاز کنند؛ به‌ویژه دانشجویان، برنامه‌نویسان مبتدی و افرادی که تجربه‌ی محدودی در کار با ایجنت‌های برنامه‌نویسی دارند. مطالب به‌گونه‌ای تنظیم شده‌اند که خواننده بتواند ابتدا مفاهیم و محیط کاری ابزار را بشناسد و سپس به‌تدریج با قابلیت‌های عملی و پیشرفته‌تر آن آشنا شود.

پیشنهاد می‌شود مطالعه از فصل اول آغاز شود و به‌ترتیب ادامه پیدا کند. هم‌زمان با مطالعه، بهتر است مفاهیم و قابلیت‌های معرفی‌شده در یک پروژه‌ی آزمایشی اجرا شوند تا درک مطالب به تجربه‌ی عملی تبدیل شود. همچنین، به دلیل تغییر و توسعه‌ی مداوم ابزارهای هوش مصنوعی، مراجعه به مستندات رسمی برای بررسی قابلیت‌ها و تنظیمات نسخه‌ی مورد استفاده ضروری است.

نکته‌ی اساسی در تمام مراحل این است که ایجنت ابزاری برای کمک به انسان است، نه جایگزینی کامل برای دانش، قضاوت و مسئولیت او. خروجی‌های تولیدشده، تغییرات کد و دستورات اجرایی باید متناسب با اهمیت و حساسیت پروژه بررسی و آزمایش شوند. هدف نهایی این کتابچه، صرفاً آشنایی با یک ابزار نیست؛ بلکه کمک به شکل‌گیری رویکردی آگاهانه، مسئولانه و مؤثر در استفاده از ایجنت‌ها برای توسعه‌ی نرم‌افزار است.

فصل اول: آشنایی باGoogle Antigravity

Antigravity .چیست و چه کاربردی دارد؟

"Google Antigravity" یک پلتفرم توسعه‌ی مبتنی بر هوش مصنوعی است که به کاربران کمک می‌کند وظایف نرم‌افزاری را در سطحی فراتر از تولید ساده‌ی کد انجام دهند. در ابزارهای معمولی برنامه‌نویسی، توسعه‌دهنده بیشتر مراحل کار را مستقیماً انجام می‌دهد و هوش مصنوعی در صورت نیاز پیشنهاد یا پاسخ ارائه می‌کند؛ اما در محیط‌های مبتنی بر ایجنت، کاربر می‌تواند یک هدف مشخص را تعریف کند و از ایجنت بخواهد برای رسیدن به آن برنامه‌ریزی کند، مراحل لازم را اجرا کند و نتیجه را بررسی کند.

Antigravityبا ترکیب قابلیت‌های هوش مصنوعی و ابزارهای توسعه‌ی نرم‌افزار، امکان انجام وظایفی مانند بررسی کدهای موجود، ایجاد و ویرایش فایل‌ها، پیاده‌سازی قابلیت‌های جدید، رفع خطاها و آزمایش رابط کاربری را فراهم می‌کند. این پلتفرم همچنین می‌تواند برای اجرای برخی وظایف از ابزارهایی مانند ویرایشگر کد، ترمینال و مرورگر استفاده کند.

برای مثال، فرض کنید می‌خواهید یک برنامه‌ی ساده برای مدیریت کارهای روزانه طراحی کنید. در روش سنتی، باید ساختار پروژه را ایجاد کنید، فایل‌های لازم را بنویسید، رابط کاربری را طراحی کنید و در صورت بروز خطا، آن را برطرف کنید. در Antigravity می‌توانید ابتدا هدف و نیازمندی‌های برنامه را توضیح دهید تا ایجنت در طراحی مراحل اجرا، ایجاد فایل‌ها و پیاده‌سازی بخش‌های مختلف به شما کمک کند. سپس می‌توانید نتیجه‌ی کار را بررسی کنید و در صورت نیاز، اصلاحات بیشتری درخواست دهید.

نکته‌ی مهم این است که Antigravity صرفاً یک چت‌بات برای پاسخ‌گویی به پرسش‌های برنامه‌نویسی نیست؛ بلکه محیطی برای سپردن و پیگیری وظایف به ایجنت‌های هوش مصنوعی است. البته میزان استقلال ایجنت و نوع عملیاتی که می‌تواند انجام دهد، به ابزارهای در دسترس، تنظیمات محیط و مجوزهای تعریف‌شده بستگی دارد.

کاربردهای اصلیAntigravity

-توسعه‌ی نرم‌افزار: کمک به ایجاد پروژه‌ها، نوشتن کد و افزودن قابلیت‌های جدید.

-بررسی و درک پروژه: مطالعه‌ی فایل‌ها و ساختار کد برای شناخت بهتر نحوه‌ی عملکرد یک نرم‌افزار.

-رفع خطا: بررسی مشکلات احتمالی، پیشنهاد راه‌حل و کمک به اصلاح کد.

-آزمایش و بررسی نتیجه: اجرای تست‌ها و استفاده از مرورگر برای مشاهده و بررسی عملکرد برنامه، در صورت فراهم بودن ابزارهای لازم.

-مدیریت وظایف پیچیده: تقسیم یک کار بزرگ به مراحل کوچک‌تر و پیگیری پیشرفت آن‌ها.

-ارائه‌ی گزارش از کار انجام‌شده: تولید خروجی‌هایی مانند برنامه‌ی اجرایی، تغییرات کد و تصاویر مربوط به نتیجه‌ی کار که به کاربر در بررسی عملکرد ایجنت کمک می‌کنند.

این قابلیت‌ها باعث می‌شوند Antigravity برای دانشجویان، برنامه‌نویسان و توسعه‌دهندگانی مفید باشد که می‌خواهند بخشی از فرایند توسعه‌ی نرم‌افزار را با کمک هوش مصنوعی انجام دهند؛ با این حال، استفاده‌ی مؤثر از آن همچنان به تعریف درست مسئله و بررسی نتایج نیاز دارد.

نکته‌ای درباره‌ی محصولاتAntigravity

Antigravityبه یک محیط واحد محدود نیست و در حال حاضر چند شیوه‌ی استفاده از این فناوری ارائه شده است؛ از جمله"Antigravity IDE " برای کار در محیط توسعه، "Antigravity 2.0" برای مدیریت و هماهنگی ایجنت‌ها، و "Antigravity CLI" برای تعامل از طریق ترمینال. همچنین Antigravity SDK امکاناتی برای ساخت و مدیریت برنامه‌محور ایجنت‌ها فراهم می‌کند.

از آنجا که تمرکز این کتابچه بر کار با محیط توسعه، فایل‌های پروژه، ترمینال، مرورگر و تعامل مستقیم با ایجنت است، مطالب عملی آن عمدتاً بر مبنای "Antigravity IDE" تنظیم شده‌اند. ممکن است نام گزینه‌ها یا بعضی قابلیت‌ها در سایر محصولات یا نسخه‌های جدید متفاوت باشد.

قابلیت‌های اصلی ایجنت

ایجنت در Antigravity هسته‌ی اصلی انجام وظایف مبتنی بر هوش مصنوعی است. این ایجنت می‌تواند درخواست کاربر را تحلیل کند، با توجه به زمینه‌ی پروژه مراحل کار را مشخص کند، از ابزارهای موجود کمک بگیرد و نتیجه‌ی فعالیت‌های خود را در قالب خروجی‌های قابل بررسی ارائه دهد.

مهم‌ترین قابلیت‌های آن عبارت‌اند از:

درک درخواست و برنامه‌ریزی

ایجنت می‌تواند درخواست کاربر را تحلیل کند و برای انجام وظایف چندمرحله‌ای، برنامه‌ای شامل مراحل موردنیاز ارائه دهد. برای نمونه، اگر از آن بخواهید به یک پروژه قابلیت ورود کاربران اضافه کند، ممکن است ابتدا ساختار پروژه و شیوه‌ی مدیریت کاربران را بررسی کند و سپس مراحل پیاده‌سازی را مشخص کند.

البته کیفیت این برنامه به شفافیت درخواست، اطلاعات موجود در پروژه و توانایی مدل بستگی دارد. بنابراین، بهتر است پیش از اجرای تغییرات مهم، برنامه‌ی پیشنهادی بررسی شود.

خواندن، ایجاد و ویرایش فایل‌ها: ایجنت می‌تواند با توجه به مجوزها و ابزارهای در دسترس، فایل‌های پروژه را بخواند، فایل‌های جدید ایجاد کند و بخش‌هایی از کد موجود را تغییر دهد. این قابلیت به او کمک می‌کند تا تغییرات را در ارتباط با ساختار واقعی پروژه انجام دهد، نه اینکه صرفاً قطعه‌کدی جدا از پروژه تولید کند.

با این حال، هر تغییر پیشنهادی الزاماً صحیح یا سازگار با کل پروژه نیست و باید بررسی شود.

استفاده از ترمینال

ترمینال محیطی برای اجرای دستورات سیستم‌عامل و ابزارهای توسعه است. ایجنت، در صورت دسترسی مجاز، می‌تواند از آن برای انجام کارهایی مانند نصب وابستگی‌ها، اجرای برنامه یا اجرای تست‌ها استفاده کند.

از آنجا که بعضی دستورات ممکن است فایل‌ها را حذف کنند، تنظیمات را تغییر دهند یا اطلاعات حساسی را در معرض خطر قرار دهند، اجرای دستورات باید متناسب با سطح دسترسی و حساسیت عملیات کنترل شود.

تعامل با مرورگر

Antigravityمی‌تواند از قابلیت‌های مرورگر برای بررسی صفحات وب، مشاهده‌ی نتیجه‌ی اجرای یک برنامه و کمک به آزمایش رابط کاربری استفاده کند. برای مثال، پس از ایجاد یک صفحه‌ی وب، ایجنت می‌تواند با ابزارهای مرورگر آن را بازبینی کند و مشکلات قابل مشاهده را شناسایی کند.

این قابلیت به معنای بی‌نیازی از آزمایش انسانی نیست؛ ممکن است بعضی خطاها، مشکلات دسترس‌پذیری یا رفتارهای خاص کاربران در بررسی خودکار شناسایی نشوند.

ارائه‌ی گزارش و خروجی‌های قابل بررسی

ایجنت می‌تواند اطلاعات مربوط به روند کار را در قالب خروجی‌هایی مانند فهرست وظایف، برنامه‌ی پیاده‌سازی، نمایش تغییرات کد و تصاویر یا ضبط‌های مرورگر ارائه کند. در Antigravity به این نوع خروجی‌های ساختاریافته "Artifacts" گفته می‌شود.

این خروجی‌ها به کاربر کمک می‌کنند نتیجه‌ی کار را بهتر بررسی کند و در صورت نیاز، بازخورد بدهد یا درخواست اصلاح کند.

مدیریت چند وظیفه یا ایجنت

بسته به محیط و نسخه‌ی مورد استفاده، Antigravityامکان مدیریت چند گفت‌وگو یا ایجنت و اجرای برخی وظایف به‌صورت موازی را فراهم می‌کند. برای نمونه، می‌توان بخش‌هایی از بررسی یک پروژه را به وظایف جداگانه تقسیم کرد؛ البته هماهنگی میان نتایج و جلوگیری از تغییرات متناقض همچنان اهمیت دارد.

در مجموع، این قابلیت‌ها امکان می‌دهند بخشی از کارهای تکراری یا چندمرحله‌ای با کمک ایجنت انجام شوند. با این حال، انتخاب راه‌حل مناسب، تعیین اولویت‌ها و ارزیابی نهایی نتیجه همچنان به مشارکت کاربر نیاز دارد.

حوزه‌های کاربردی و موارد استفاده از Antigravity

- توسعه‌ی وب‌سایت و برنامه‌های وب: طراحی و پیاده‌سازی صفحات، ایجاد قابلیت‌های جدید، اصلاح رابط کاربری و کمک به توسعه‌ی بخش‌های مختلف وب‌سایت.
- توسعه‌ی نرم‌افزار و برنامه‌نویسی: نوشتن کد، تکمیل توابع، ایجاد فایل‌ها و پیاده‌سازی قابلیت‌های جدید در پروژه‌های نرم‌افزاری.
- یادگیری برنامه‌نویسی: توضیح کدها، آموزش مفاهیم، بررسی مثال‌ها و کمک به درک خطاها برای دانشجویان و افراد مبتدی.
- اشکال‌زدایی و بهبود کد: شناسایی مشکلات احتمالی، تحلیل پیام‌های خطا، پیشنهاد راه‌حل و کمک به خواناتر و منظم‌تر شدن کد.
- آزمون و بررسی نرم‌افزار: کمک به نوشتن تست‌ها، اجرای آزمون‌ها و بررسی رفتار برنامه برای شناسایی مشکلات.
- تحلیل و درک پروژه‌های موجود: بررسی ساختار فایل‌ها، شناسایی بخش‌های مهم پروژه و توضیح ارتباط میان اجزای مختلف آن.
- توسعه‌ی پروژه‌های داده و هوش مصنوعی: کمک به آماده‌سازی کدهای پردازش داده، پیاده‌سازی مدل‌ها و ساخت ابزارهای مرتبط؛ با این شرط که اجرای واقعی و صحت نتایج جداگانه بررسی شوند.
- خودکارسازی وظایف تکراری: کمک به انجام کارهای چندمرحله‌ای مانند تولید فایل‌های مشخص، بررسی مجموعه‌ای از فایل‌ها یا اجرای گردش‌کارهای تعریف‌شده.
- مستندسازی پروژه: تهیه‌ی توضیحات فنی، راهنمای نصب و اجرا، توضیح توابع و تنظیم یا به‌روزرسانی مستندات پروژه.

فصل دوم: محیط کار و راه‌اندازی پروژه

برای استفاده‌ی مؤثر ازGoogle Antigravity ، ابتدا باید محیط کاری آن را آماده کنیم و با بخش‌های اصلی آن آشنا شویم. شناخت محیط توسعه، نحوه‌ی ایجاد پروژه و محل قرارگیری ابزارهای مختلف باعث می‌شود هنگام کار با ایجنت، کنترل بیشتری بر فایل‌ها، دستورات و تغییرات پروژه داشته باشیم.

در این فصل، مراحل نصب و راه‌اندازیAntigravity IDE ، ساخت پروژه، آشنایی با محیط کار و مدیریت پروژه‌ها را بررسی می‌کنیم.

آشنایی با محیطAntigravity

"Antigravity IDE"محیط توسعه‌ای است که ابزارهای برنامه‌نویسی و قابلیت‌های ایجنت‌های هوش مصنوعی را در کنار یکدیگر قرار می‌دهد. در این محیط، کاربر می‌تواند فایل‌های پروژه را مشاهده و ویرایش کند، با ایجنت گفت‌وگو کند، دستورات موردنیاز را اجرا کند و نتیجه‌ی کار را بررسی کند.

برخلاف محیط‌هایی که در آن‌ها هوش مصنوعی صرفاً به پرسش‌ها پاسخ می‌دهد، Antigravityبه ایجنت امکان می‌دهد با توجه به دسترسی‌های تعریف‌شده، در بخش‌های مختلف محیط توسعه فعالیت کند. به همین دلیل، آشنایی با اجزای محیط کار برای نظارت بر فعالیت‌های ایجنت اهمیت دارد.

اجزای اصلی محیط کار

محیط ویرایش کد(Editor)

ویرایشگر بخشی از محیط توسعه است که کدهای برنامه در آن نمایش داده و ویرایش می‌شوند. کاربر می‌تواند فایل‌های مختلف را باز کند، کدها را بخواند و تغییرات پیشنهادی ایجنت را بررسی کند.

برای مثال، اگر ایجنت یک تابع جدید به برنامه اضافه کند، می‌توان فایل مربوط به آن را در ویرایشگر باز کرد و منطق کد را پیش از استفاده بررسی کرد.

پنل ایجنت(Agent Panel) : پنل ایجنت محل اصلی تعامل با هوش مصنوعی است. کاربر درخواست خود را در این بخش مطرح می‌کند و ایجنت بر اساس اطلاعات موجود و ابزارهای در دسترس، برای انجام آن اقدام می‌کند.

در این بخش می‌توان درخواست‌های جدید مطرح کرد، درباره‌ی مراحل کار سؤال پرسید، توضیحات بیشتری ارائه داد و در صورت نیاز، از ایجنت خواست تغییرات را اصلاح کند.

ترمینال(Terminal)

ترمینال محیطی برای وارد کردن و اجرای دستورات است. بسیاری از فعالیت‌های توسعه‌ی نرم‌افزار، مانند اجرای برنامه، نصب کتابخانه‌ها، مدیریت محیط پایتون و اجرای تست‌ها، از طریق ترمینال انجام می‌شوند.

ایجنت نیز در صورت برخورداری از دسترسی لازم می‌تواند از ترمینال برای اجرای برخی عملیات استفاده کند. با این حال، کاربر باید دستورات حساس یا ناشناخته را پیش از اجرا بررسی کند.

مرورگر(Browser)

مرورگر برای مشاهده‌ی صفحات وب و بررسی عملکرد رابط کاربری کاربرد دارد. برای نمونه، پس از ایجاد یک صفحه‌ی وب، می‌توان نتیجه را در مرورگر مشاهده کرد و مشکلات ظاهری یا عملکردی آن را بررسی کرد.

Antigravity قابلیت‌هایی برای تعامل ایجنت با مرورگر فراهم می‌کند؛ اما نحوه‌ی دسترسی و امکانات موجود ممکن است به نسخه و تنظیمات محیط وابسته باشد.

بخش بررسی تغییرات و خروجی‌ها

در طول کار، ایجنت ممکن است فایل‌هایی ایجاد کند یا تغییراتی در کدهای موجود اعمال کند. بررسی این تغییرات به کاربر کمک می‌کند بفهمد چه چیزی تغییر کرده و آیا نتیجه با درخواست اولیه مطابقت دارد یا خیر.

همچنین، ایجنت می‌تواند خروجی‌هایی مانند برنامه‌ی انجام کار، گزارش پیشرفت، نمایش تفاوت کدها و نتایج بررسی مرورگر ارائه دهد. این خروجی‌ها در Antigravity با عنوان کلی Artifacts شناخته می‌شوند.

ارتباط میان اجزای محیط

این بخش‌ها در کنار یکدیگر چرخه‌ی معمول کار با Antigravity را تشکیل می‌دهند. برای مثال، کاربر در پنل ایجنت درخواست ایجاد یک برنامه را مطرح می‌کند؛ ایجنت فایل‌های لازم را ایجاد می‌کند؛ کدها در ویرایشگر قابل مشاهده‌اند؛ برنامه از طریق ترمینال اجرا می‌شود و نتیجه در مرورگر بررسی می‌شود.

در پایان، کاربر تغییرات و نتیجه‌ی کار را ارزیابی می‌کند و در صورت وجود مشکل، از ایجنت می‌خواهد اصلاحات لازم را انجام دهد.

بنابراین، شناخت محیط کار تنها برای پیدا کردن گزینه‌ها نیست؛ بلکه به کاربر کمک می‌کند مسیر اجرای وظایف و نقاطی را که نیاز به بررسی انسانی دارند، بهتر بشناسد.

نصب و پیش‌نیازها

برای استفاده ازAntigravity IDE ، ابتدا باید نسخه‌ی مناسب سیستم‌عامل خود را دریافت و نصب کنید. بهتر است نصب را از وب‌سایت رسمی انجام دهید تا فایل برنامه از منبع معتبر دریافت شود.

پیش‌نیازهای نصب

بر اساس مستندات رسمی، حداقل سیستم‌عامل‌های پشتیبانی‌شده برای Antigravity IDE به شرح زیر هستند:

Windows: ویندوز ۱۰ نسخه‌ی ۶۴ بیتی یا جدیدتر.

macOS: نسخه‌ی ۱۲ (Monterey) یا جدیدتر، برای پردازنده‌های Apple Silicon یا.Intel

Linux: توزیع‌هایی که شرایط کتابخانه‌های سیستمی موردنیاز، از جمله `glibc` نسخه‌ی ۲/۲۸ یا جدیدتر و `glibcxx` نسخه‌ی ۳/۴/۲۵ یا جدیدتر را داشته باشند.

این موارد حداقل‌های اعلام‌شده برای سیستم‌عامل هستند و به‌تنهایی تضمین نمی‌کنند که برنامه روی هر دستگاهی با عملکرد مناسب اجرا شود. منابع سخت‌افزاری، اتصال اینترنت و محدودیت‌های حساب کاربری نیز می‌توانند بر تجربه‌ی استفاده تأثیر بگذارند.

مراحل نصب

مرحله‌ی اول: ورود به وب‌سایت رسمی

به نشانی زیر مراجعه کنید:

https://antigravity.google/download

در صفحه‌ی دانلود، نسخه‌ی متناسب با سیستم‌عامل خود را انتخاب کنید.

مرحله‌ی دوم: دریافت فایل نصب

فایل مربوط به سیستم‌عامل خود را دریافت کنید. پیش از اجرا، مطمئن شوید فایل را از وب‌سایت رسمی دریافت کرده‌اید و نسخه‌ی انتخاب‌شده با سیستم‌عامل شما سازگار است.

مرحله‌ی سوم: نصب برنامه

فایل دریافت‌شده را اجرا کنید و مراحل نصب را دنبال کنید. در صورت نمایش پیام‌های مربوط به جایگزینی نسخه‌ی قبلی، دستورالعمل رسمی همان نسخه را بررسی کنید.

مرحله‌ی چهارم: اجرای برنامه و ورود به حساب

پس از نصب، Antigravityرا اجرا کنید. برای استفاده از قابلیت‌های حساب کاربری، طبق راهنمای رسمی با حساب Google شخصی خود وارد شوید یا در صورت استفاده‌ی سازمانی، روش ورود مورد تأیید سازمان را انتخاب کنید.

مرحله‌ی پنجم: بررسی آماده‌بودن محیط

پس از ورود، محیط کاری را باز کنید و بررسی کنید که برنامه بدون خطا اجرا می‌شود. در صورت مشاهده‌ی درخواست به‌روزرسانی، راهنمای رسمی نسخه‌ی خود را دنبال کنید.

توجه داشته باشید که نام گزینه‌ها و جزئیات مراحل نصب ممکن است با انتشار نسخه‌های جدید تغییر کنند. بنابراین، اگر محیط برنامه با توضیحات این کتابچه تفاوت داشت، مستندات رسمی نسخه‌ی مورد استفاده را ملاک قرار دهید.

ساخت و باز کردن پروژه

پس از نصبAntigravity ، گام بعدی ایجاد یا باز کردن یک پروژه است. پروژه محدوده‌ای از فایل‌ها و منابع را مشخص می‌کند که ایجنت برای انجام وظایف خود در اختیار دارد.

انتخاب درست پروژه اهمیت زیادی دارد؛ زیرا ایجنت برای درک ساختار برنامه، بررسی کدهای موجود و اعمال تغییرات به اطلاعات همان محیط متکی است.

پروژه چیست؟

در توسعه‌ی نرم‌افزار، پروژه مجموعه‌ای از فایل‌ها و منابع مرتبط با یک برنامه یا سامانه است. این مجموعه ممکن است شامل کدهای منبع، فایل‌های تنظیمات، مستندات، تست‌ها و وابستگی‌های نرم‌افزاری باشد.

برای مثال، یک پروژه‌ی پایتون می‌تواند فایل اصلی برنامه، ماژول‌های کمکی، فایل وابستگی‌ها و پوشه‌ی تست‌ها را در خود داشته باشد.

وقتی یک پوشه یا مخزن کد را به‌عنوان پروژه معرفی می‌کنید، ایجنت می‌تواند با توجه به مجوزهای خود از فایل‌های آن برای درک وظیفه استفاده کند. به همین دلیل، بهتر است پوشه‌ی انتخاب‌شده همان محدوده‌ای باشد که قصد دارید روی آن کار کنید.

ایجاد یک پروژه‌ی جدید

برای ساخت پروژه در محیطAntigravity ، طبق راهنمای رسمی این مراحل را دنبال کنید:

مرحله‌ی اول: باز کردن بخش پروژه‌ها

در نوار کناری محیطAntigravity ، گزینه‌ی مربوط به پروژه‌ها را پیدا کنید. در راهنمای رسمی، این بخش با نماد پوشه و علامت `+` مشخص شده است.

مرحله‌ی دوم: انتخابNew Project

روی گزینه‌ی `New Project` کلیک کنید تا فرایند ایجاد پروژه آغاز شود.

مرحله‌ی سوم: افزودن پوشه

با استفاده از گزینه‌ی`Add Folder` ، پوشه‌ی موردنظر را انتخاب کنید. اگر از قبل پروژه‌ای ساخته‌اید، می‌توانید پوشه‌ی آن را معرفی کنید. در صورت نیاز، امکان افزودن چند پوشه یا مخزن Git نیز وجود دارد تا ایجنت بتواند ارتباط میان بخش‌های مختلف پروژه را بهتر درک کند.

مرحله‌ی چهارم: ایجاد پروژه

پس از انتخاب پوشه‌ها، گزینه‌ی `Create` را انتخاب کنید تا پروژه ایجاد شود.

مرحله پنجم: بررسی تنظیمات پروژه

در صورت نیاز، تنظیمات اختصاصی پروژه را بررسی کنید. هر پروژه می‌تواند تنظیمات و سیاست‌های امنیتی جداگانه‌ای داشته باشد؛ بنابراین بهتر است پیش از شروع کار، محدوده‌ی دسترسی و تنظیمات مرتبط را بشناسید.

باز کردن یک پروژه‌ی موجود

اگر پروژه‌ای از قبل روی سیستم خود دارید، لازم نیست آن را از ابتدا بسازید. می‌توانید از بخش پروژه‌ها، پوشه یا مخزن مربوط به آن را انتخاب کنید و در محیط Antigravity باز کنید.

پیش از شروع کار، چند مورد را بررسی کنید:

-پوشه‌ی انتخاب‌شده متعلق به پروژه‌ی صحیح باشد.

-فایل‌های اصلی و پوشه‌های مهم پروژه در دسترس باشند.

-فایل‌های محرمانه یا اطلاعاتی که نباید در اختیار ابزار قرار گیرند، به‌درستی محافظت شده باشند.

-اگر پروژه از Git استفاده می‌کند، وضعیت تغییرات موجود را بررسی کرده باشید.

-اگر پروژه به محیط مجازی، کتابخانه‌ها یا تنظیمات خاص نیاز دارد، آن‌ها را مطابق مستندات پروژه آماده کنید.

نکته: برای یادگیری اولیه، بهتر است یک پروژه‌ی آزمایشی و ساده بسازید. این کار کمک می‌کند بدون به خطر انداختن فایل‌های یک پروژه‌ی مهم، با نحوه‌ی عملکرد ایجنت آشنا شوید.

آشنایی با پنل ایجنت، ویرایشگر و ترمینال

پس از ایجاد پروژه، باید بتوانید سه بخش مهم محیط را در کنار یکدیگر به کار بگیرید: پنل ایجنت، ویرایشگر و ترمینال.

پنل ایجنت؛ محل تعریف و پیگیری وظایف

در پنل ایجنت، وظیفه‌ای را که می‌خواهید انجام شود به زبان طبیعی توضیح می‌دهید. برای مثال:

"ساختار این پروژه‌ی پایتون را بررسی کن، فایل‌های اصلی را معرفی کن و بدون ایجاد تغییر، مشکلات احتمالی را گزارش بده"

این درخواست از ایجنت می‌خواهد ابتدا پروژه را بررسی کند و گزارشی ارائه دهد؛ نه اینکه بلافاصله کدها را تغییر دهد.

برای دریافت نتیجه‌ی بهتر، درخواست را تا حد امکان مشخص کنید. اگر هدف شما فقط بررسی است، این موضوع را صریح بیان کنید. اگر اجازه‌ی تغییر فایل‌ها را می‌دهید، محدوده‌ی تغییرات مورد انتظار را نیز مشخص کنید.

هنگام کار با ایجنت، به برنامه‌ی پیشنهادی، گزارش مراحل انجام کار و پیام‌های مربوط به خطا یا نیاز به تأیید توجه کنید. اگر نتیجه با هدف شما مطابقت ندارد، می‌توانید درخواست را دقیق‌تر کنید یا از ایجنت بخواهید پیش از ادامه، برنامه‌ی خود را اصلاح کند.

ویرایشگر؛ محل بررسی کدها

ویرایشگر به شما اجازه می‌دهد کدهایی را که ایجنت تولید کرده یا تغییر داده است مستقیماً مشاهده کنید.

برای نمونه، اگر ایجنت تابعی برای محاسبه‌ی میانگین اعداد بنویسد، می‌توانید کد را در ویرایشگر بررسی کنید و مطمئن شوید که حالت‌های مورد انتظار، مانند ورودی خالی یا داده‌های نامعتبر، در طراحی آن در نظر گرفته شده‌اند.

بهتر است هنگام بررسی تغییرات به این موارد توجه کنید:

-آیا کد با هدف اولیه مطابقت دارد؟

-آیا نام متغیرها و توابع معنادار است؟

-آیا تغییرات غیرضروری در بخش‌های دیگر ایجاد شده‌اند؟

-آیا کد با ساختار و سبک موجود در پروژه سازگار است؟

-آیا اطلاعات حساس یا تنظیمات مهم ناخواسته تغییر کرده‌اند؟

ترمینال؛ محل اجرای دستورات

ترمینال برای اجرای دستورات موردنیاز پروژه استفاده می‌شود. برای مثال، در یک پروژه‌ی پایتون می‌توان از دستور زیر برای بررسی نسخه‌ی نصب‌شده‌ی پایتون استفاده کرد:

```
python --version
```

در برخی سیستم‌ها، به‌ویژه اگر چند نسخه‌ی پایتون نصب باشد، ممکن است لازم باشد از دستور `python3 --version` استفاده کنید.

برای اجرای یک فایل ساده‌ی پایتون، می‌توانید از دستوری مانند زیر استفاده کنید:

```
python main.py
```

این دستور زمانی کاربرد دارد که فایل `main.py` در مسیر کاری فعلی وجود داشته باشد و محیط پایتون به‌درستی تنظیم شده باشد.

اگر ایجنت دستوری برای نصب کتابخانه، حذف فایل یا تغییر تنظیمات پیشنهاد کرد، پیش از اجرا هدف و پیامدهای آن را بررسی کنید. هر دستوری که ایجنت تولید می‌کند الزاماً بی‌خطر نیست.

ارتباط میان این سه بخش

برای استفاده‌ی مؤثر ازAntigravity ، بهتر است پنل ایجنت، ویرایشگر و ترمینال را ابزارهایی مکمل در نظر بگیرید:

۱ .در پنل ایجنت، وظیفه را تعریف می‌کنید.

۲ .در صورت نیاز، برنامه‌ی پیشنهادی ایجنت را بررسی می‌کنید.

۳ .ایجنت تغییرات را اعمال می‌کند و شما کدهای مربوطه را در ویرایشگر مشاهده می‌کنید.

۴ .برنامه یا تست‌ها را اجرا می‌کنید و نتیجه را بررسی می‌کنید.

۵ .اگر خطایی وجود داشت، اطلاعات آن را در اختیار ایجنت قرار می‌دهید تا برای رفع آن اقدام کند.

۶ .پس از تأیید نتیجه، تغییرات را ثبت می‌کنید یا در صورت لزوم آن‌ها را بازمی‌گردانید.

این چرخه در فصل‌های بعد با جزئیات بیشتری بررسی خواهد شد.

انتخاب محیط کار و مدیریت پروژه‌ها

Antigravityمی‌تواند در شیوه‌های مختلفی برای کار با کد و ایجنت مورد استفاده قرار گیرد. انتخاب محیط مناسب به نوع پروژه، ابزارهای موردنیاز و سطح تجربه‌ی کاربر بستگی دارد.

انتخاب محیط مناسب

در مستندات فعلیGoogle Antigravity ، چند روش اصلی برای استفاده از این فناوری معرفی شده‌اند:

Antigravity IDE: محیط توسعه‌ای مستقل که ایجنت را در کنار ویرایشگر، ترمینال و مرورگر در اختیار کاربر قرار می‌دهد.

Antigravity 2.0: محیطی برای مدیریت و هماهنگی ایجنت‌ها و پروژه‌ها، با امکاناتی برای تعریف وظایف و انتخاب شیوه‌ی اجرای آن‌ها.

Antigravity CLI:نسخه‌ی خط فرمان که امکان تعامل با ایجنت را مستقیماً از طریق ترمینال فراهم می‌کند.

این روش‌ها از نظر شیوه‌ی تعامل و محیط کار با یکدیگر تفاوت دارند. در این کتابچه، تمرکز اصلی بر Antigravity IDEاست؛ بنابراین، مراحل عملی را باید در چارچوب همین محیط در نظر گرفت.

مدیریت پروژه‌ها

با افزایش تعداد پروژه‌ها، مدیریت درست آن‌ها اهمیت بیشتری پیدا می‌کند. بهتر است برای هر پروژه، ساختار مشخصی داشته باشید و از انتخاب پوشه‌های نامرتبط یا مبهم خودداری کنید.

چند اصل کاربردی برای مدیریت پروژه‌ها عبارت‌اند از:

نام‌گذاری روشن

برای پوشه‌ها و پروژه‌ها نام‌هایی انتخاب کنید که هدف آن‌ها را مشخص کنند. برای مثال، نام `daily_task_manager`از نام‌هایی مانند `project1` یا `test2` اطلاعات بیشتری درباره‌ی کاربرد پروژه ارائه می‌دهد.

جداسازی پروژه‌ها

اگر روی چند برنامه‌ی مستقل کار می‌کنید، فایل‌های آن‌ها را در پوشه‌های جداگانه نگه دارید. این کار احتمال تغییر اشتباه فایل‌ها و سردرگمی هنگام انتخاب پروژه را کاهش می‌دهد.

حفظ ساختار پروژه

پوشه‌های کد، تست‌ها، مستندات و فایل‌های تنظیمات را تا حد امکان با ساختاری منظم نگه دارید. ساختار مناسب به خودتان و ایجنت کمک می‌کند فایل‌های مرتبط را سریع‌تر پیدا کنید.

استفاده ازGit

Gitابزاری برای ثبت تاریخچه‌ی تغییرات فایل‌هاست. با استفاده از آن می‌توانید تغییرات را پیگیری کنید و در شرایط مناسب به نسخه‌های قبلی بازگردید.

پیش از سپردن وظایف مهم به ایجنت، بهتر است وضعیت مخزن Git و تغییرات ثبت‌نشده‌ی خود را بررسی کنید. اگر تغییرات مهمی دارید، آن‌ها را به شکلی مناسب ثبت یا پشتیبان‌گیری کنید تا در صورت بروز مشکل امکان بازیابی داشته باشید.

بررسی تنظیمات و محدوده‌ی دسترسی

پروژه‌ها ممکن است تنظیمات و سیاست‌های امنیتی متفاوتی داشته باشند. پیش از اجرای وظایف، مطمئن شوید ایجنت به فایل‌ها و منابع موردنیاز دسترسی دارد، اما دسترسی غیرضروری به اطلاعات حساس در اختیارش قرار نگرفته است.

انتخاب روش اجرای وظایف

در محیط‌هایی که از حالت‌های اجرایی متفاوت پشتیبانی می‌کنند، باید با توجه به شرایط پروژه تصمیم بگیرید که ایجنت مستقیماً روی پوشه‌ی فعال کار کند یا در محیطی جداشده، مانندGit worktree ، فعالیت کند.

کار مستقیم روی پوشه‌ی فعال می‌تواند برای وظایف ساده و کم‌خطر مناسب باشد. محیط جداشده نیز می‌تواند به کاهش تداخل تغییرات کمک کند، به‌ویژه زمانی که چند وظیفه به‌طور موازی انجام می‌شوند. البته محیط جداشده به‌تنهایی تضمین‌کننده‌ی امنیت نیست و همچنان باید تغییرات و دستورات اجراشده را بررسی کرد.

در این فصل با محیط کاریAntigravity ، پیش‌نیازها و مراحل نصب، نحوه‌ی ایجاد یا باز کردن پروژه و نقش پنل ایجنت، ویرایشگر و ترمینال آشنا شدیم. همچنین اصول اولیه‌ی مدیریت پروژه‌ها، استفاده از Gitو انتخاب محیط مناسب برای اجرای وظایف را مرور کردیم.

پیش از رفتن به فصل بعد، بهتر است Antigravity را اجرا کنید، یک پروژه‌ی آزمایشی بسازید و از ایجنت بخواهید ساختار آن را بدون ایجاد تغییر بررسی کند. سپس گزارش ارائه‌شده را با فایل‌های واقعی پروژه مقایسه کنید.

فصل سوم: کار با ایجنت و چرخه‌ی انجام وظایف

شناخت محیط Antigravity به‌تنهایی برای استفاده‌ی مؤثر از آن کافی نیست. بخش مهمی از کار با این پلتفرم به نحوه‌ی تعریف وظایف، ارتباط با ایجنت و نظارت بر مراحل اجرا مربوط می‌شود. هرچه کاربر بتواند هدف خود را دقیق‌تر توضیح دهد و نتیجه‌ی کار را بهتر ارزیابی کند، احتمال رسیدن به خروجی مناسب بیشتر خواهد شد.

در این فصل، چرخه‌ی انجام وظایف در Antigravity را بررسی می‌کنیم؛ چرخه‌ای که از تعریف مسئله آغاز می‌شود، با برنامه‌ریزی و اجرای مراحل ادامه پیدا می‌کند و پس از بررسی و آزمایش نتیجه به پایان می‌رسد.

نوشتن درخواست دقیق و تعیین هدف

نقطه‌ی شروع کار با ایجنت، ارائه‌ی یک درخواست است. ایجنت باید بداند کاربر چه هدفی دارد، چه محدودیت‌هایی وجود دارند و چه نتیجه‌ای مورد انتظار است. اگر درخواست مبهم باشد، ممکن است ایجنت برداشت متفاوتی از مسئله داشته باشد یا راه‌حلی انتخاب کند که با نیاز واقعی کاربر مطابقت ندارد.

برای مثال، درخواست «برنامه‌ام را بهتر کن» مشخص نمی‌کند که منظور از بهتر شدن چیست. ممکن است کاربر به دنبال اصلاح ظاهر برنامه، افزایش سرعت، رفع خطا یا افزودن قابلیت جدید باشد. در مقابل، درخواست «صفحه‌ی ورود این برنامه را برای نمایش در تلفن همراه بهینه کن و منطق ورود کاربران را تغییر نده» هدف و محدوده‌ی کار را روشن‌تر می‌کند.

ویژگی‌های یک درخواست مناسب

تعیین هدف

درخواست باید مشخص کند که ایجنت قرار است چه کاری انجام دهد. هدف بهتر است با یک نتیجه‌ی قابل تشخیص بیان شود.

نمونه:

"یک برنامه‌ی ساده با پایتون بساز که بتواند نام کارها را دریافت کند و آن‌ها را در فهرستی نمایش دهد"

مشخص کردن محدوده‌ی کار

گاهی لازم است مشخص کنید ایجنت کدام فایل‌ها یا بخش‌های پروژه را تغییر دهد و از چه تغییراتی خودداری کند.

نمونه:

"فقط ظاهر صفحه‌ی اصلی را اصلاح کن. منطق پردازش اطلاعات و فایل‌های مربوط به پایگاه داده را تغییر نده"

ارائه‌ی اطلاعات لازم

اگر پروژه از زبان برنامه‌نویسی، کتابخانه یا چارچوب خاصی استفاده می‌کند، بهتر است آن را در درخواست مشخص کنید. همچنین اگر محدودیت یا نیازمندی مهمی وجود دارد، آن را توضیح دهید.

نمونه:

"این پروژه با Django نوشته شده است. قابلیت جدید را با ساختار فعلی پروژه هماهنگ کن و پیش از ایجاد فایل جدید، فایل‌های موجود را بررسی کن"

مشخص کردن نتیجه‌ی مورد انتظار

اگر معیار مشخصی برای موفقیت وجود دارد، آن را بیان کنید. این کار به ارزیابی خروجی کمک می‌کند.

نمونه:

"کاربر باید بتواند یک کار جدید اضافه کند، کارهای ثبت‌شده را ببیند و یک کار را حذف کند. پس از پیاده‌سازی، عملکرد هر سه قابلیت را آزمایش کن"

تعیین محدودیت‌های اجرایی

در صورت نیاز، مشخص کنید چه اقداماتی مجاز نیستند یا باید پیش از انجام آن‌ها از شما تأیید گرفته شود.

نمونه:

"فایل‌های موجود را حذف نکن، کتابخانه‌ی جدید نصب نکن و پیش از اجرای دستوراتی که فایل‌ها یا تنظیمات را تغییر می‌دهند، دلیل اجرای آن‌ها را توضیح بده"

الگوی پیشنهادی برای نوشتن درخواست

برای بسیاری از وظایف برنامه‌نویسی می‌توانید از الگوی زیر استفاده کنید:

-هدف: چه کاری باید انجام شود؟

-وضعیت فعلی: پروژه اکنون در چه شرایطی است؟

-محدوده: کدام فایل‌ها یا بخش‌ها باید بررسی یا تغییر کنند؟

-محدودیت‌ها: چه کارهایی نباید انجام شوند؟

-معیار موفقیت: از کجا بفهمیم کار درست انجام شده است؟

-روش بررسی: چه تست یا بررسی‌ای باید انجام شود؟

لازم نیست تمام درخواست‌ها طولانی باشند. برای کارهای ساده، یک یا دو جمله‌ی روشن کافی است؛ اما برای وظایف پیچیده، ارائه‌ی اطلاعات بیشتر می‌تواند از برداشت‌های اشتباه جلوگیری کند.

همچنین بهتر است هر بار یک هدف اصلی را دنبال کنید. اگر چند کار نامرتبط را در یک درخواست قرار دهید، بررسی نتیجه و تشخیص خطا دشوارتر می‌شود.

برنامه‌ریزی و اجرای وظایف

پس از دریافت درخواست، ایجنت باید مسئله را تحلیل کند و بر اساس پیچیدگی کار، مراحل موردنیاز را مشخص کند. وظایف ساده ممکن است مستقیماً اجرا شوند؛ اما کارهای بزرگ‌تر معمولاً به بررسی پروژه، برنامه‌ریزی، اجرای چند مرحله و آزمایش نتیجه نیاز دارند.

برای مثال، ساخت یک سامانه‌ی ثبت‌نام کاربران ممکن است شامل بررسی ساختار پروژه، طراحی فرم، اعتبارسنجی اطلاعات، اتصال به پایگاه داده، مدیریت خطاها و نوشتن تست‌ها باشد. انجام همه‌ی این مراحل بدون برنامه‌ریزی می‌تواند احتمال ایجاد ناسازگاری یا فراموش شدن بخشی از نیازمندی‌ها را افزایش دهد.

مراحل معمول اجرای یک وظیفه

مرحله‌ی اول: تحلیل درخواست

ایجنت درخواست را بررسی می‌کند تا هدف، نیازمندی‌ها و محدودیت‌های آن را تشخیص دهد. در این مرحله ممکن است برای درک بهتر پروژه، فایل‌های مرتبط را بررسی کند یا درباره‌ی بخش‌های نامشخص سؤال بپرسد.

مرحله‌ی دوم: بررسی وضعیت پروژه

پیش از اعمال تغییرات، لازم است ساختار پروژه و کدهای مرتبط شناخته شوند. این بررسی کمک می‌کند راه‌حل انتخاب‌شده با معماری و شیوه‌ی فعلی پروژه هماهنگ باشد.

مرحله‌ی سوم: تعیین مراحل اجرا

برای وظایف پیچیده، کار به مراحل کوچک‌تر تقسیم می‌شود. هر مرحله باید هدف مشخصی داشته باشد و در صورت امکان بتوان نتیجه‌ی آن را بررسی کرد.

مرحله‌ی چهارم: اجرای تغییرات

ایجنت با استفاده از ابزارهای مجاز، تغییرات لازم را انجام می‌دهد. این تغییرات ممکن است شامل ایجاد فایل، ویرایش کد، اجرای دستورات یا آماده‌سازی تست‌ها باشند.

مرحله‌ی پنجم: بررسی نتیجه

پس از اجرای تغییرات، باید بررسی شود که آیا نتیجه با درخواست اولیه مطابقت دارد یا خیر. اجرای تست‌ها و مشاهده‌ی عملکرد برنامه می‌تواند به شناسایی مشکلات کمک کند.

مرحله‌ی ششم: گزارش نهایی

در پایان، ایجنت باید نتیجه‌ی کار، تغییرات مهم و مشکلات باقی‌مانده را تا حد امکان روشن کند. کاربر نیز باید خروجی را بررسی کند و در صورت نیاز، مراحل اصلاحی بیشتری درخواست دهد.

چرا باید وظایف بزرگ را تقسیم کنیم؟

فرض کنید می‌خواهید یک سامانه‌ی مدیریت دانشجو بسازید. درخواست کلی «یک سامانه‌ی کامل مدیریت دانشجو بساز» بخش‌های مختلف کار را مشخص نمی‌کند و ارزیابی نتیجه‌ی آن نیز دشوار است.

می‌توان این وظیفه را به مراحل زیر تقسیم کرد:

۱ .بررسی نیازمندی‌ها و پیشنهاد ساختار پروژه.

۲ .ایجاد مدل اطلاعاتی دانشجو و تعریف فیلدهای لازم.

۳ .پیاده‌سازی قابلیت ثبت و مشاهده‌ی اطلاعات دانشجویان.

۴ .افزودن قابلیت ویرایش و حذف اطلاعات.

۵ .طراحی رابط کاربری.

۶ .نوشتن تست‌ها و بررسی عملکرد.

۷ .اصلاح مشکلات و مستندسازی نتیجه.

با این روش، می‌توانید خروجی هر مرحله را بررسی کنید و پیش از ادامه، مشکلات احتمالی را برطرف کنید. تقسیم کار به معنای نیاز به تأیید تک‌تک خطوط کد نیست؛ هدف آن است که نقاط مهم تصمیم‌گیری و بررسی مشخص باشند.

تعامل با ایجنت و هدایت مراحل کار

کار با ایجنت معمولاً با ارسال یک درخواست آغاز می‌شود، اما نباید آن را به ارسال یک پیام و انتظار برای نتیجه محدود کرد. در بسیاری از پروژه‌ها، تعامل مستمر با ایجنت باعث می‌شود درخواست‌ها دقیق‌تر شوند و خروجی نهایی با نیاز کاربر تطابق بیشتری داشته باشد.

ارائه‌ی بازخورد دقیق

اگر نتیجه‌ی اولیه مناسب نبود، بهتر است به‌جای درخواست کلی «درستش کن»، مشکل را مشخص کنید.

برای مثال:

درخواست نامناسب:

"این صفحه خوب نیست. درستش کن"

درخواست بهتر:

"فاصله‌ی میان فرم و عنوان صفحه زیاد است. فاصله را کاهش بده، اما اندازه‌ی متن و رنگ‌های فعلی را حفظ کن"

در درخواست دوم، مشکل و محدوده‌ی اصلاح مشخص شده‌اند؛ بنابراین ایجنت می‌تواند با احتمال بیشتری تغییر مورد انتظار را اعمال کند.

اصلاح برنامه‌ی ایجنت

اگر در حین کار متوجه شدید ایجنت مسیر نامناسبی انتخاب کرده است، می‌توانید پیش از ادامه، درخواست را اصلاح کنید یا از آن بخواهید برنامه‌ی خود را بازنگری کند.

برای مثال:

"پیش از ادامه، برنامه‌ی اجرا را دوباره بررسی کن. هدف من استفاده از ساختار فعلی پروژه است و نمی‌خواهم چارچوب برنامه‌نویسی تغییر کند"

این روش به‌ویژه زمانی مفید است که تغییرات پیشنهادی گسترده باشند یا تصمیم ایجنت بر معماری پروژه اثر بگذارد.

استفاده از اطلاعات خطا

هنگامی که برنامه با خطا مواجه می‌شود، متن کامل خطا و شرایطی که در آن رخ داده است می‌توانند اطلاعات ارزشمندی در اختیار ایجنت قرار دهند.

بهتر است به‌جای نوشتن «برنامه اجرا نمی‌شود»، توضیح دهید که چه دستوری اجرا شده، چه نتیجه‌ای انتظار داشتید و چه خطایی دریافت کردید. در صورت امکان، خروجی ترمینال یا مراحل بازتولید خطا را نیز ارائه کنید.

البته اطلاعات محرمانه، رمزهای عبور و کلیدهای دسترسی نباید در گزارش خطا یا درخواست‌های ارسالی قرار گیرند.

حفظ زمینه‌ی گفت‌وگو

ایجنت برای درک درخواست‌های بعدی از زمینه‌ی موجود در تعامل و اطلاعات قابل دسترسی پروژه استفاده می‌کند. اگر درخواست جدید به تصمیمی در مراحل قبل وابسته است، بهتر است آن تصمیم را روشن بیان کنید.

برای مثال:

"بر اساس همان ساختاری که در مرحله‌ی قبل انتخاب کردیم، قابلیت جست‌وجوی دانشجو را اضافه کن. ساختار مدل اطلاعاتی را تغییر نده"

اگر گفت‌وگو طولانی شده یا تصمیم‌های مهم به‌درستی در زمینه‌ی فعلی منعکس نشده‌اند، خلاصه‌ای از هدف، تصمیم‌های قطعی و محدودیت‌ها ارائه کنید. این کار احتمال سوءبرداشت را کاهش می‌دهد.

چه زمانی باید اجرای ایجنت را متوقف کنیم؟

در صورت مشاهده‌ی رفتار غیرمنتظره، افزایش بی‌دلیل محدوده‌ی تغییرات، اجرای دستورات مشکوک یا تلاش برای دسترسی به اطلاعات نامرتبط، بهتر است ادامه‌ی کار را متوقف کنید و وضعیت را بررسی کنید.

همچنین اگر ایجنت چند بار راه‌حل ناموفق مشابهی را تکرار می‌کند، ادامه‌ی همان روند لزوماً مفید نیست. در چنین شرایطی، باید مسئله را دوباره تعریف کرد، اطلاعات بیشتری فراهم کرد یا روش حل مسئله را تغییر داد.

بررسی کد و فایل‌های تغییرکرده

پس از اجرای وظیفه، یکی از مهم‌ترین مراحل بررسی خروجی است. پایان یافتن عملیات ایجنت به معنای تأیید صحت همه‌ی تغییرات نیست. ممکن است ایجنت فایل‌های غیرضروری را تغییر داده باشد، بخشی از نیازمندی‌ها را نادیده گرفته باشد یا کدی تولید کرده باشد که فقط در شرایط خاص درست کار می‌کند.

بنابراین، پیش از پذیرش نتیجه باید تغییرات را متناسب با اهمیت وظیفه بررسی کنید.

بررسی فهرست فایل‌های تغییرکرده

ابتدا مشخص کنید کدام فایل‌ها ایجاد، حذف یا ویرایش شده‌اند. اگر هدف شما اصلاح یک تابع بوده است، تغییر گسترده در فایل‌های نامرتبط می‌تواند نشانه‌ای باشد که باید دلیل آن را بررسی کنید.

در پروژه‌هایی که از Git استفاده می‌کنند، می‌توانید با دستور زیر وضعیت فایل‌ها را مشاهده کنید:

```
git status
```

این دستور وضعیت فایل‌های مخزن را نمایش می‌دهد و کمک می‌کند تغییرات ثبت‌نشده را شناسایی کنید.

برای مشاهده‌ی تفاوت محتوای فایل‌های تغییرکرده با نسخه‌ی ثبت‌شده در Git نیز می‌توانید از دستور زیر استفاده کنید:

```
git diff
```

توجه داشته باشید که این دستورها برای بررسی تغییرات ثبت‌نشده در مخزن Git کاربرد دارند. فایل‌های جدیدی که هنوز به مخزن اضافه نشده‌اند ممکن است محتوای خود را در خروجی معمول `git diff` نشان ندهند؛ بنابراین، فهرست وضعیت فایل‌ها را نیز بررسی کنید.

بررسی منطق کد

هنگام خواندن کدهای تغییرکرده، تنها به این نکته توجه نکنید که کد ظاهراً مرتب یا قابل اجراست. منطق آن نیز باید با هدف پروژه مطابقت داشته باشد.

برای بررسی کد می‌توانید این پرسش‌ها را مطرح کنید:

-آیا کد دقیقاً همان قابلیت موردنظر را پیاده‌سازی می‌کند؟

-آیا ورودی‌های نامعتبر یا حالت‌های خاص در نظر گرفته شده‌اند؟

-آیا تغییرات با توابع و ماژول‌های موجود سازگارند؟

-آیا اعتبارسنجی و مدیریت خطا به‌اندازه‌ی کافی انجام شده است؟

-آیا کد به اطلاعات محرمانه یا تنظیمات ناامن وابسته شده است؟

-آیا ایجنت تغییرات غیرضروری ایجاد کرده است؟

اگر بخشی از کد را درک نمی‌کنید، از ایجنت بخواهید منطق آن را توضیح دهد؛ اما توضیح ایجنت را نیز در صورت اهمیت موضوع با مستندات و آزمایش عملی بررسی کنید

بررسی وابستگی‌ها و تنظیمات

بعضی تغییرات ممکن است به نصب کتابخانه‌ی جدید یا تغییر فایل‌های تنظیمات نیاز داشته باشند. این موارد را نادیده نگیرید؛ زیرا ممکن است بر اجرای برنامه در دستگاه‌های دیگر یا محیط استقرار اثر بگذارند.

اگر وابستگی جدیدی اضافه شده است، دلیل نیاز به آن و سازگاری نسخه‌ی انتخاب‌شده را بررسی کنید. همچنین مطمئن شوید فایل‌های حاوی اطلاعات محرمانه به‌طور ناخواسته وارد مخزن یا خروجی پروژه نشده‌اند.

استفاده از بررسی انسانی و ابزارهای خودکار

بازبینی کد، تحلیل ایستا و اجرای تست‌ها روش‌های مکمل یکدیگرند. تست موفق می‌تواند برخی خطاها را آشکار کند، اما به‌تنهایی ثابت نمی‌کند که برنامه از نظر امنیتی یا منطقی کاملاً صحیح است.

به همین دلیل، بهتر است تغییرات مهم با ترکیبی از بررسی انسانی، تست‌های مناسب و ابزارهای کمکی ارزیابی شوند.

اجرا، تست و رفع خطا

پس از بررسی اولیه‌ی کد، باید برنامه یا بخش تغییرکرده را اجرا کنید و ببینید آیا مطابق انتظار عمل می‌کند یا خیر. این مرحله اهمیت زیادی دارد؛ زیرا بعضی خطاها تنها هنگام اجرا یا در تعامل با داده‌های واقعی ظاهر می‌شوند.

تفاوت اجرا و تست

اجرا به معنای راه‌اندازی برنامه یا اجرای یک عملیات برای مشاهده‌ی عملکرد آن است.

تست فرایندی هدفمند برای بررسی رفتار برنامه در شرایط مشخص و مقایسه‌ی نتیجه با انتظار است.

برای مثال، اجرای یک برنامه‌ی مدیریت کارها نشان می‌دهد که آیا برنامه راه‌اندازی می‌شود؛ اما تست افزودن، نمایش و حذف کارها بررسی می‌کند که آیا قابلیت‌های اصلی آن درست کار می‌کنند.

مراحل پیشنهادی برای بررسی عملکرد

مرحله‌ی اول: اجرای برنامه

از دستور مناسب پروژه برای اجرا استفاده کنید. دستور اجرا به زبان برنامه‌نویسی، چارچوب و ساختار پروژه بستگی دارد. اگر روش اجرا مشخص نیست، فایل راهنمای پروژه و تنظیمات آن را بررسی کنید یا از ایجنت بخواهید دستور مناسب را توضیح دهد.

مرحله‌ی دوم: مشاهده‌ی خروجی

بررسی کنید که آیا برنامه بدون خطا اجرا می‌شود و نتیجه‌ی مورد انتظار را ارائه می‌دهد. در برنامه‌های دارای رابط کاربری، صفحات و رفتارهای مهم را نیز در مرورگر یا محیط مربوطه مشاهده کنید.

مرحله‌ی سوم: اجرای تست‌های موجود

اگر پروژه تست‌های خودکار دارد، آن‌ها را اجرا کنید. این تست‌ها می‌توانند به بررسی عملکرد بخش‌های مختلف و شناسایی مشکلاتی که در اثر تغییرات ایجاد شده‌اند کمک کنند.

مرحله‌ی چهارم: بررسی حالت‌های مختلف

تنها حالت عادی را بررسی نکنید. برای مثال، در یک فرم ثبت اطلاعات، ورودی خالی، داده‌ی نامعتبر و ورودی تکراری نیز ممکن است نیاز به بررسی داشته باشند.

مرحله‌ی پنجم: ثبت و تحلیل خطا

اگر مشکلی رخ داد، متن خطا، دستور اجراشده و شرایط بروز آن را ثبت کنید. سپس از ایجنت بخواهید علت احتمالی را بررسی کند.

روش صحیح رفع خطا با ایجنت

رفع خطا نباید صرفاً به تغییر کد تا زمان ناپدید شدن پیام خطا محدود شود. هدف، پیدا کردن علت اصلی مشکل و اصلاح آن بدون ایجاد خطاهای جدید است.

برای این منظور می‌توانید از ایجنت بخواهید:

۱ .خطا را تحلیل کند و علت‌های احتمالی را توضیح دهد.

۲ .فایل‌ها یا بخش‌های مرتبط را بررسی کند.

۳ .پیش از اعمال تغییرات گسترده، راه‌حل پیشنهادی را ارائه دهد.

۴ .تغییرات ضروری را با حداقل دست‌کاری در بخش‌های نامرتبط اعمال کند.

۵ .تست مرتبط را اجرا کند و نتیجه را گزارش دهد.

برای مثال، اگر هنگام اجرای برنامه با خطای نبودن یک کتابخانه مواجه شدید، نباید بدون بررسی تمام وابستگی‌ها را حذف و دوباره نصب کنید. ابتدا باید مشخص شود کدام کتابخانه پیدا نشده، محیط اجرای فعلی کدام است و آیا وابستگی واقعاً نصب نشده یا برنامه در محیط اشتباهی اجرا می‌شود.

اگر خطا دوباره تکرار شد:

اگر اصلاح اولیه نتیجه نداد، همان درخواست را بدون اطلاعات جدید بارها تکرار نکنید. بهتر است خروجی جدید را با خطای قبلی مقایسه کنید، تغییرات اخیر را بررسی کنید و اطلاعات بیشتری درباره‌ی محیط اجرا فراهم کنید.

گاهی نیز مشکل از کد نیست؛ ممکن است نسخه‌ی کتابخانه، تنظیمات محیط، دسترسی فایل یا متغیرهای محیطی عامل اصلی باشد. تشخیص درست علت، از تغییرات غیرضروری جلوگیری می‌کند.

تأیید نهایی و ثبت تغییرات باGit

پس از اجرای برنامه و بررسی نتیجه، باید تصمیم بگیرید که آیا تغییرات آماده‌ی پذیرش هستند یا خیر. در پروژه‌های کوچک، این تصمیم ممکن است ساده باشد؛ اما در پروژه‌های مهم، بهتر است معیارهای مشخصی برای تأیید نهایی داشته باشید.

معیارهای تأیید نهایی

پیش از پذیرش تغییرات، این موارد را بررسی کنید:

-نیازمندی‌های اصلی درخواست برآورده شده‌اند.

-فایل‌های تغییرکرده با محدوده‌ی موردنظر مطابقت دارند.

-برنامه یا قابلیت جدید به‌درستی اجرا می‌شود.

-تست‌های مرتبط اجرا شده‌اند و نتیجه‌ی آن‌ها بررسی شده است.

-خطاهای مهم و مشکلات شناخته‌شده مشخص شده‌اند.

-اطلاعات محرمانه یا تنظیمات ناامن به پروژه اضافه نشده‌اند.

-امکان بازگردانی تغییرات در صورت بروز مشکل وجود دارد.

اگر بخشی از کار کامل نشده است، بهتر است این موضوع مشخص شود و از اعلام موفقیت کامل خودداری شود.

Gitچیست و چرا اهمیت دارد؟

Gitیک سیستم کنترل نسخه است که تاریخچه‌ی تغییرات فایل‌های پروژه را ثبت می‌کند. با استفاده از آن می‌توان بررسی کرد چه تغییراتی انجام شده‌اند، تغییرات را در قالب نسخه‌های مشخص ثبت کرد و در شرایط مناسب به وضعیت قبلی بازگشت.

Gitبه‌ویژه هنگام کار با ایجنت اهمیت دارد؛ زیرا ایجنت ممکن است چندین فایل را تغییر دهد و کاربر به راهی برای مقایسه، ثبت و بازیابی تغییرات نیاز داشته باشد.

مراحل پایه‌ی ثبت تغییرات

مرحله‌ی اول: بررسی وضعیت مخزن

```
git status
```

این دستور فایل‌های تغییرکرده، فایل‌های جدید و وضعیت مخزن را نمایش می‌دهد.

مرحله‌ی دوم: بررسی تفاوت‌ها

```
git diff
```

این دستور به شما کمک می‌کند تغییرات ثبت‌نشده‌ی فایل‌های تحت کنترل نسخه را مشاهده کنید.

مرحله‌ی سوم: انتخاب فایل‌های موردنظر

برای آماده‌سازی یک فایل مشخص جهت ثبت، می‌توانید از دستور زیر استفاده کنید:

```
git add path/to/file
```

به‌جای `path/to/file` مسیر واقعی فایل را قرار دهید. پیش از افزودن فایل‌ها، مطمئن شوید اطلاعات محرمانه یا تغییرات نامرتبط در آن‌ها وجود ندارد.

مرحله‌ی چهارم: ثبت تغییرات

پس از بررسی تغییرات آماده‌شده، می‌توانید آن‌ها را با پیامی روشن ثبت کنید:

```
"git commit -m "Add task management feature
```

پیام ثبت باید خلاصه‌ای از تغییرات انجام‌شده باشد. دستور بالا تنها زمانی موفق می‌شود که فایل‌های موردنظر قبلاً آماده‌ی ثبت شده باشند و تنظیمات لازم Git انجام شده باشد.

این مراحل نمونه‌ای از گردش‌کار پایه‌ی Git هستند. اگر پروژه از قبل تغییرات ثبت‌نشده دارد، پیش از افزودن فایل‌ها و ثبت نسخه‌ی جدید باید تغییرات قبلی را نیز بررسی کنید تا کار دیگران یا تغییرات خودتان ناخواسته در یک ثبت نامرتبط ادغام نشوند.

بازگرداندن تغییرات

اگر ایجنت تغییر نامناسبی ایجاد کرد، Gitمی‌تواند به بازیابی وضعیت قبلی کمک کند؛ اما بازگردانی باید با دقت انجام شود. بعضی دستورهای Git تغییرات را به‌طور دائمی حذف می‌کنند یا ممکن است تغییرات شخصی ثبت‌نشده را از بین ببرند.

پیش از اجرای هر دستور بازگردانی، وضعیت مخزن و فایل‌های موردنظر را بررسی کنید و از تغییرات مهم نسخه‌ی پشتیبان داشته باشید. اگر با معنای یک دستور آشنا نیستید، ابتدا درباره‌ی اثر آن سؤال کنید و بدون بررسی از دستورات مخرب استفاده نکنید.

همچنین ثبت تغییرات در Git به‌تنهایی به معنای ارسال آن‌ها به مخزن راه دور نیست. عملیات‌هایی مانند `push` تغییرات را به مخزن راه دور می‌فرستند و باید با توجه به مجوزها و سیاست‌های پروژه انجام شوند.

آیا باید همه‌ی تغییرات ایجنت را ثبت کنیم؟

خیر. تنها تغییراتی را ثبت کنید که بررسی شده‌اند و قصد دارید در تاریخچه‌ی پروژه باقی بمانند. اگر بخشی از تغییرات نامرتبط، آزمایشی یا ناقص است، ابتدا آن را جدا کنید یا اصلاح کنید.

در پروژه‌های گروهی، پیش از ثبت یا ارسال تغییرات، هماهنگی با شیوه‌ی کاری تیم و بررسی تغییرات سایر اعضا نیز ضروری است.

چرخه‌ی کار با ایجنت در Antigravity از تعریف هدف آغاز می‌شود و با برنامه‌ریزی، اجرا، بررسی، تست و تأیید نتیجه ادامه می‌یابد. کیفیت این چرخه فقط به توانایی ایجنت وابسته نیست؛ بلکه به شفافیت درخواست، کیفیت اطلاعات ارائه‌شده، نحوه‌ی هدایت کار و دقت کاربر در ارزیابی خروجی نیز بستگی دارد.

برای استفاده‌ی مؤثر از ایجنت، وظایف را متناسب با پیچیدگی به مراحل قابل بررسی تقسیم کنید، تغییرات فایل‌ها را زیر نظر بگیرید، خطاها را بر اساس شواهد تحلیل کنید و پیش از پذیرش نهایی، نتیجه را آزمایش کنید. استفاده از Git نیز امکان پیگیری تاریخچه و مدیریت بهتر تغییرات را فراهم می‌کند.

فصل چهارم: قابلیت‌ها و ابزارهای اصلی

یکی از ویژگی‌های مهمGoogle Antigravity ، امکان استفاده از ابزارهای مختلف برای انجام وظایف توسعه‌ی نرم‌افزار است. ایجنت می‌تواند با توجه به درخواست کاربر و سطح دسترسی خود، فایل‌های

پروژه را بررسی کند، کد بنویسد، دستورات را اجرا کند و نتیجه‌ی فعالیت‌هایش را در قالب خروجی‌های قابل بررسی ارائه دهد.

در این فصل، قابلیت‌های اصلی Antigravity و کاربرد آن‌ها را بررسی می‌کنیم تا مشخص شود هر ابزار در چه شرایطی مفید است و چگونه می‌توان از آن به‌صورت مؤثر استفاده کرد.

خواندن، ایجاد و ویرایش فایل‌ها

فایل‌ها بخش اصلی هر پروژه‌ی نرم‌افزاری هستند. کدهای برنامه، تنظیمات، مستندات، داده‌های موردنیاز و تست‌ها معمولاً در فایل‌های مختلف ذخیره می‌شوند. یکی از قابلیت‌های مهم ایجنت درAntigravity ، کار با این فایل‌ها برای درک ساختار پروژه و اعمال تغییرات موردنیاز است.

خواندن و بررسی فایل‌ها

پیش از ایجاد تغییر، ایجنت می‌تواند فایل‌های مرتبط را بررسی کند تا با ساختار و منطق پروژه آشنا شود. این کار به‌ویژه در پروژه‌های قدیمی یا پروژه‌هایی که چندین ماژول دارند اهمیت دارد.

برای مثال، اگر بخواهید قابلیت جست‌وجوی اطلاعات دانشجویان را به یک پروژه اضافه کنید، بهتر است ابتدا از ایجنت بخواهید فایل‌های مربوط به مدل داده، رابط کاربری و منطق پردازش را شناسایی کند.

نمونه‌ی درخواست:

"ساختار این پروژه را بررسی کن، فایل‌های اصلی را معرفی کن و توضیح بده اطلاعات دانشجویان در کدام بخش مدیریت می‌شود. فعلاً هیچ فایلی را تغییر نده"

این درخواست به ایجنت کمک می‌کند ابتدا وضعیت موجود را بررسی کند. مشخص کردن اینکه در این مرحله تغییری انجام نشود، برای جلوگیری از اصلاحات زودهنگام مفید است.

ایجاد فایل‌های جدید

ایجنت می‌تواند برای افزودن قابلیت‌های جدید، فایل‌هایی ایجاد کند. این فایل‌ها ممکن است شامل ماژول‌های برنامه، فایل‌های تنظیمات، تست‌ها یا مستندات باشند.

برای مثال، در یک پروژه‌ی پایتون می‌توان از ایجنت خواست یک ماژول جداگانه برای اعتبارسنجی اطلاعات ایجاد کند تا منطق اعتبارسنجی از بخش اصلی برنامه مستقل باشد.

نمونه‌ی درخواست:

"یک فایل به نام `validators.py` ایجاد کن و توابع لازم برای اعتبارسنجی نام و شماره‌ی دانشجویی را در آن قرار بده. ساختار فعلی پروژه را حفظ کن و پیش از ایجاد فایل، بررسی کن که فایل مشابهی از قبل وجود نداشته باشد"

درخواست دقیق‌تر، احتمال ایجاد فایل‌های تکراری یا ساختاری ناسازگار را کاهش می‌دهد.

ویرایش فایل‌های موجود

گاهی نیازی به ایجاد فایل جدید نیست و باید کدهای فعلی اصلاح شوند. ایجنت می‌تواند بخش‌هایی از فایل را تغییر دهد، خطاهای موجود را اصلاح کند یا قابلیت جدیدی به کد اضافه کند.

برای نمونه:

"تابع ثبت دانشجو را بررسی کن. اگر شماره‌ی دانشجویی تکراری است، از ثبت اطلاعات جلوگیری کن و پیام خطای مناسبی نمایش بده. سایر بخش‌های برنامه را تغییر نده"

در چنین شرایطی باید بررسی شود که تغییر جدید با منطق قبلی برنامه سازگار است و رفتارهای موجود را ناخواسته مختل نمی‌کند.

حذف یا جابه‌جایی فایل‌ها

در برخی وظایف، ممکن است لازم باشد فایل‌ها حذف شوند، تغییر نام دهند یا به پوشه‌ی دیگری منتقل شوند. این عملیات نسبت به خواندن یا ایجاد فایل حساس‌تر هستند؛ زیرا ممکن است وابستگی‌های پروژه را مختل کنند یا اطلاعات مهمی را از بین ببرند.

بنابراین، پیش از درخواست چنین تغییراتی باید هدف و محدوده‌ی آن‌ها مشخص باشد. همچنین بهتر است از ایجنت بخواهید وابستگی‌های فایل‌های موردنظر را بررسی کند و پیش از حذف فایل‌های مهم، دلیل این اقدام را توضیح دهد.

اصول کار با فایل‌ها

برای استفاده‌ی مؤثر از این قابلیت، چند اصل را رعایت کنید:

-پیش از تغییر، ساختار پروژه و فایل‌های مرتبط را بررسی کنید.

-محدوده‌ی تغییرات مورد انتظار را مشخص کنید.

-از ایجاد فایل‌های تکراری یا تغییرات غیرضروری جلوگیری کنید.

-فایل‌های حساس و تنظیمات مهم را با دقت بیشتری بررسی کنید.

-پس از اعمال تغییرات، تفاوت نسخه‌ی قبلی و جدید فایل‌ها را مشاهده کنید.

-در پروژه‌های مهم، از Git یا روش مناسب دیگری برای ثبت و بازیابی تغییرات استفاده کنید.

اجرای دستورات ترمینال

ترمینال یکی از ابزارهای مهم توسعه‌ی نرم‌افزار است. بسیاری از فعالیت‌ها، مانند اجرای برنامه، نصب وابستگی‌ها، اجرای تست‌ها و بررسی وضعیت پروژه، از طریق دستورات ترمینال انجام می‌شوند.

درAntigravity ، ایجنت می‌تواند با توجه به ابزارها و مجوزهای در دسترس، برخی از این دستورات را اجرا کند. این قابلیت باعث می‌شود انجام وظایف چندمرحله‌ای ساده‌تر شود؛ اما به دلیل اثر احتمالی دستورات بر سیستم و فایل‌ها، بررسی آن‌ها اهمیت زیادی دارد.

کاربردهای ترمینال

اجرای برنامه

پس از نوشتن کد، می‌توان برنامه را از طریق ترمینال اجرا کرد تا عملکرد آن بررسی شود. برای مثال، در یک پروژه‌ی پایتون که فایل اصلی آن `main.py` نام دارد، دستور زیر ممکن است برای اجرا مناسب باشد:

```
python main.py
```

این دستور به ساختار پروژه و تنظیمات محیط اجرا وابسته است. اگر فایل در مسیر دیگری قرار داشته باشد یا پروژه به تنظیمات خاصی نیاز داشته باشد، باید دستور مناسب همان پروژه استفاده شود.

نصب وابستگی‌ها

بعضی پروژه‌ها برای اجرا به کتابخانه‌های خارجی نیاز دارند. ایجنت ممکن است برای نصب آن‌ها دستوری پیشنهاد کند یا اجرا کند.

برای مثال، در یک پروژه‌ی پایتون می‌توان از دستور زیر برای نصب وابستگی‌های فهرست‌شده در فایل `requirements.txt` استفاده کرد:

```
python -m pip install -r requirements.txt
```

پیش از اجرا، باید مطمئن شوید فایل مربوطه متعلق به پروژه‌ی موردنظر است و نصب وابستگی‌ها در محیط مناسب انجام می‌شود. بهتر است برای هر پروژه‌ی پایتون از محیط مجازی مناسب استفاده کنید تا کتابخانه‌های پروژه با سایر برنامه‌ها تداخل نداشته باشند.

اجرای تست‌ها

اگر پروژه تست‌های خودکار داشته باشد، ایجنت می‌تواند برای اجرای آن‌ها از ترمینال استفاده کند. دستور دقیق به ابزار تست پروژه بستگی دارد؛ برای مثال، در یک پروژه‌ی پایتون ممکن است از `pytest` استفاده شود.

بررسی وضعیت محیط

ترمینال امکان بررسی نسخه‌ی ابزارها، وضعیت مخزن Git و برخی تنظیمات پروژه را فراهم می‌کند. این اطلاعات هنگام عیب‌یابی خطاها مفید هستند.

خطرهای اجرای دستورات

همه‌ی دستورات ترمینال بی‌خطر نیستند. برخی از آن‌ها ممکن است فایل‌ها را حذف کنند، تنظیمات سیستم را تغییر دهند، بسته‌های نرم‌افزاری را نصب کنند یا به منابع خارجی متصل شوند.

به همین دلیل:

-هدف هر دستور را پیش از اجرا بررسی کنید.

-دستورات حذف فایل یا تغییر تنظیمات را با دقت بیشتری ارزیابی کنید.

-از اجرای دستورهای ناشناخته بدون درک پیامدهای آن‌ها خودداری کنید.

-در پروژه‌های حساس، از ایجنت بخواهید پیش از اجرای عملیات مهم، دستور و دلیل آن را توضیح دهد.

-به پیام‌های تأیید و تنظیمات مجوزها توجه کنید.

اجرای موفق یک دستور نیز به معنای صحیح بودن نتیجه‌ی نهایی نیست. پس از اجرا باید خروجی آن بررسی شود و در صورت نیاز، تست‌های تکمیلی انجام شوند.

کار با مرورگر و بررسی نتایج

در توسعه‌ی نرم‌افزار، به‌ویژه توسعه‌ی وب، اجرای کد به‌تنهایی برای اطمینان از صحت عملکرد کافی نیست. باید بررسی شود که برنامه از دید کاربر چگونه نمایش داده می‌شود و آیا تعاملات مورد انتظار به‌درستی انجام می‌شوند.

Antigravityامکان استفاده از ابزارهای مرورگر را برای مشاهده و بررسی صفحات وب فراهم می‌کند. ایجنت می‌تواند در صورت دسترسی مناسب، به بررسی رابط کاربری و شناسایی برخی مشکلات قابل مشاهده کمک کند.

بررسی رابط کاربری

فرض کنید ایجنت یک صفحه‌ی ثبت‌نام طراحی کرده است. پس از اجرای برنامه، می‌توان از مرورگر برای بررسی موارد زیر استفاده کرد:

-آیا صفحه به‌درستی بارگذاری می‌شود؟

-آیا عنوان‌ها، فرم‌ها و دکمه‌ها نمایش داده می‌شوند؟

-آیا چیدمان صفحه با اندازه‌ی پنجره‌ی مرورگر سازگار است؟

-آیا دکمه‌ها و فرم‌ها رفتار مورد انتظار را دارند؟

-آیا پیام‌های خطا و موفقیت به شکل مناسبی نمایش داده می‌شوند؟

این بررسی‌ها به شناسایی مشکلاتی کمک می‌کنند که ممکن است صرفاً با خواندن کد مشخص نشوند.

بررسی عملکرد صفحات

گاهی صفحه از نظر ظاهری درست است، اما عملکرد آن با نیازمندی‌های پروژه مطابقت ندارد. برای مثال، ممکن است فرم ثبت اطلاعات نمایش داده شود، ولی با فشردن دکمه‌ی ثبت، اطلاعات ذخیره نشوند.

در چنین شرایطی باید عملکرد واقعی برنامه بررسی شود. می‌توان از ایجنت خواست یک سناریوی مشخص را آزمایش کند و نتیجه را گزارش دهد.

نمونه‌ی درخواست:

"صفحه‌ی ثبت کار جدید را در مرورگر بررسی کن. یک کار با عنوان معتبر ثبت کن، سپس رفتار برنامه را هنگام ارسال فرم خالی بررسی کن. نتیجه‌ی هر دو حالت را گزارش بده و بدون تأیید من، تغییرات گسترده اعمال نکن"

این درخواست دو حالت متفاوت را بررسی می‌کند: ورودی معتبر و ورودی نامعتبر.

نقش تصاویر و گزارش‌های مرورگر

درAntigravity ، خروجی‌هایی مانند تصاویر صفحه و گزارش‌های مرتبط با بررسی مرورگر می‌توانند به کاربر کمک کنند نتیجه‌ی کار ایجنت را بهتر ارزیابی کند. این خروجی‌ها به‌ویژه برای بررسی مشکلات ظاهری یا مشاهده‌ی وضعیت برنامه در یک مرحله‌ی مشخص مفید هستند.

با این حال، یک تصویر ثابت نمی‌تواند همه‌ی رفتارهای برنامه را نشان دهد. ممکن است صفحه در یک حالت درست به نظر برسد، اما هنگام ارسال فرم، بارگذاری داده‌ها یا تغییر اندازه‌ی پنجره دچار مشکل شود. بنابراین، بررسی بصری باید با آزمایش عملکردی تکمیل شود.

محدودیت‌های بررسی خودکار

بررسی مرورگر نمی‌تواند جایگزین کامل آزمایش انسانی باشد. ممکن است ایجنت برخی خطاها را تشخیص ندهد یا تنها سناریوهای محدودی را آزمایش کند. همچنین نتیجه‌ی بررسی به وضعیت محیط اجرا، داده‌های آزمایش و ابزارهای در دسترس وابسته است.

برای برنامه‌های حساس، باید تست‌های مناسب، بررسی امنیتی و ارزیابی انسانی نیز انجام شوند.

استفاده از Artifacts و گزارش‌های ایجنت

درAntigravity ، ایجنت می‌تواند علاوه بر تغییر فایل‌ها و اجرای دستورات، خروجی‌هایی برای توضیح و نمایش روند کار ارائه دهد. به این خروجی‌های ساختاریافته"Artifacts" گفته می‌شود.

Artifactsکمک می‌کنند فعالیت‌های ایجنت شفاف‌تر شوند و کاربر بتواند برنامه‌ی اجرا، تغییرات یا نتیجه‌ی بررسی‌ها را بهتر ارزیابی کند.

انواع خروجی‌های قابل بررسی

بسته به نوع وظیفه و قابلیت‌های نسخه‌ی مورد استفاده، ممکن است با خروجی‌هایی مانند موارد زیر روبه‌رو شوید:

برنامه‌ی اجرا(Implementation Plan)

برای وظایف پیچیده، ایجنت می‌تواند مراحلی را که برای انجام کار در نظر گرفته است ارائه کند. این برنامه به کاربر اجازه می‌دهد پیش از اجرای تغییرات گسترده، منطق کلی راه‌حل را بررسی کند.

برای مثال، در یک پروژه‌ی مدیریت دانشجو، برنامه‌ی اجرا ممکن است شامل بررسی مدل اطلاعاتی، پیاده‌سازی فرم ثبت دانشجو، افزودن اعتبارسنجی و نوشتن تست‌ها باشد.

گزارش پیشرفت یا نتیجه‌ی کار

گزارش ایجنت می‌تواند توضیح دهد چه اقداماتی انجام شده‌اند، کدام فایل‌ها تغییر کرده‌اند و چه مشکلاتی باقی مانده‌اند. چنین گزارشی به کاربر کمک می‌کند نتیجه را با درخواست اولیه مقایسه کند.

نمایش تغییرات کد

بررسی تفاوت نسخه‌ی قبلی و جدید فایل‌ها کمک می‌کند تغییرات ایجنت قابل مشاهده و ارزیابی باشند. این کار به‌ویژه در پروژه‌هایی که چند فایل تغییر کرده‌اند اهمیت دارد.

تصاویر و خروجی‌های مرورگر

در وظایفی که به رابط کاربری مربوطاند، تصاویر صفحه یا نتایج بررسی مرورگر می‌توانند وضعیت ظاهری برنامه را نشان دهند و به شناسایی مشکلات کمک کنند.

چگونه از Artifacts استفاده کنیم؟

Artifactsرا باید ابزارهایی برای بررسی و تصمیم‌گیری دانست، نه مدرکی قطعی بر صحیح بودن نتیجه. هنگام مشاهده‌ی آن‌ها، این پرسش‌ها را مطرح کنید:

-آیا برنامه‌ی اجرا با هدف اولیه مطابقت دارد؟

-آیا گزارش، همه‌ی تغییرات مهم را توضیح می‌دهد؟

-آیا نتیجه‌ی نمایش‌داده‌شده با رفتار واقعی برنامه سازگار است؟

-آیا تست‌ها واقعاً اجرا شده‌اند و نتیجه‌ی آن‌ها مشخص است؟

-آیا مشکلات باقی‌مانده یا محدودیت‌های اجرا بیان شده‌اند؟

اگر گزارشی ادعا می‌کند تست‌ها موفق بوده‌اند، در صورت اهمیت موضوع باید خروجی واقعی تست‌ها را نیز بررسی کنید. صرف ارائه‌ی گزارش، جایگزین تأیید مستقل نیست.

تفاوت گزارش ایجنت با مستندات پروژه

گزارش ایجنت معمولاً درباره‌ی فعالیت‌هایی است که در یک وظیفه انجام شده‌اند؛ اما مستندات پروژه اطلاعات پایدارتری درباره‌ی معماری، نحوه‌ی نصب، روش اجرا و شیوه‌ی استفاده ارائه می‌کنند.

اگر تغییری مهم و پایدار در پروژه ایجاد شده است، بهتر است در صورت نیاز مستندات خود پروژه نیز به‌روزرسانی شوند. نباید فرض کرد که گزارش هر وظیفه به‌طور خودکار جایگزین مستندات رسمی پروژه می‌شود.

مدیریت وظایف و اجرای موازی ایجنت‌ها

در پروژه‌های کوچک، ممکن است یک ایجنت بتواند بیشتر مراحل را به‌صورت متوالی انجام دهد. اما در پروژه‌های بزرگ‌تر، تقسیم کار به چند وظیفه‌ی مستقل می‌تواند روند بررسی و توسعه را منظم‌تر کند.

Antigravityقابلیت‌هایی برای مدیریت وظایف و کار با چند ایجنت فراهم می‌کند. نحوه‌ی استفاده و امکانات دقیق این قابلیت‌ها به نسخه و محیط انتخاب‌شده بستگی دارد.

تقسیم یک کار بزرگ به وظایف کوچک‌تر

فرض کنید قصد دارید یک برنامه‌ی مدیریت کتابخانه بسازید. به‌جای سپردن تمام جزئیات پروژه در قالب یک وظیفه‌ی بسیار گسترده، می‌توانید کار را به بخش‌هایی مانند موارد زیر تقسیم کنید:

-بررسی نیازمندی‌ها و طراحی ساختار پروژه.

-پیاده‌سازی مدل کتاب و اطلاعات کاربران.

-طراحی رابط کاربری.

-ایجاد قابلیت جست‌وجوی کتاب.

-نوشتن تست‌ها و بررسی عملکرد.

این تقسیم‌بندی به شما کمک می‌کند هر بخش را جداگانه بررسی کنید و پیش از ادامه، مشکلات احتمالی را شناسایی کنید.

چه زمانی اجرای موازی مفید است؟

اجرای موازی زمانی مفید است که چند وظیفه به‌اندازه‌ی کافی مستقل باشند و انجام هم‌زمان آن‌ها باعث تداخل در فایل‌ها یا تصمیم‌های معماری نشود.

برای مثال، بررسی مستندات یک کتابخانه و بررسی ساختار تست‌های یک پروژه ممکن است بتوانند به‌صورت مستقل انجام شوند. اما اگر دو ایجنت هم‌زمان یک فایل مشترک را تغییر دهند، احتمال ایجاد تعارض یا ناسازگاری افزایش می‌یابد.

بنابراین، موازی‌سازی همیشه به معنای سرعت بیشتر یا نتیجه‌ی بهتر نیست. میزان استقلال وظایف، منابع در دسترس و هزینه‌ی هماهنگی باید در نظر گرفته شوند.

اصول تقسیم کار میان ایجنت‌ها

- تعیین مسئولیت هر وظیفه

هر وظیفه باید هدف مشخصی داشته باشد. برای مثال، یک ایجنت می‌تواند ساختار پروژه را بررسی کند و ایجنت دیگر مستندات یک کتابخانه را تحلیل کند.

- تعیین محدوده‌ی تغییرات

اگر چند ایجنت در یک پروژه فعالیت می‌کنند، باید مشخص باشد هرکدام مجاز به تغییر کدام بخش‌ها هستند. این کار احتمال تغییرات متناقض را کاهش می‌دهد.

- مشخص کردن وابستگی‌ها

اگر یک وظیفه به نتیجه‌ی وظیفه‌ی دیگری وابسته است، بهتر است تا مشخص شدن نتیجه‌ی مرحله‌ی قبلی صبر کنید. برای مثال، پیاده‌سازی رابط کاربری ممکن است به نهایی شدن ساختار داده‌ها وابسته باشد.

- یکپارچه‌سازی نتایج

پس از پایان وظایف مستقل، باید نتایج آن‌ها با یکدیگر هماهنگ شوند. کدهای جداگانه ممکن است به‌تنهایی درست کار کنند، اما هنگام ترکیب با هم ناسازگار باشند.

- بررسی نهایی توسط کاربر

حتی اگر چند ایجنت وظایف خود را با موفقیت تمام کرده باشند، نتیجه‌ی کلی پروژه باید بررسی شود. تست یکپارچه، بازبینی تغییرات و بررسی عملکرد نهایی همچنان ضروری هستند.

مدیریت تعارض‌ها

گاهی دو وظیفه به یک فایل یا بخش مشترک وابسته‌اند. در چنین شرایطی، بهتر است یکی از وظایف ابتدا انجام شود و سپس نتیجه‌ی آن در اختیار وظیفه‌ی بعدی قرار گیرد؛ یا برای هر وظیفه محیط کاری جداگانه‌ای در نظر گرفته شود.

اگر از Git یا Git worktree استفاده می‌کنید، می‌توانید تغییرات بخش‌های مختلف را در محیط‌های مجزا مدیریت کنید. با این حال، ادغام تغییرات باید با بررسی تعارض‌ها و اجرای تست‌های لازم همراه باشد.

آیا همیشه باید از چند ایجنت استفاده کرد؟ خیر. برای وظایف کوچک و روشن، استفاده از یک ایجنت معمولاً ساده‌تر است. مدیریت چند ایجنت می‌تواند پیچیدگی بیشتری ایجاد کند؛ زیرا کاربر باید نتایج، وابستگی‌ها و تغییرات هر بخش را هماهنگ کند.

بهتر است ابتدا وظیفه را به‌درستی تعریف کنید و سپس بر اساس پیچیدگی و میزان استقلال مراحل تصمیم بگیرید که اجرای متوالی کافی است یا اجرای موازی ارزش افزوده دارد.

قابلیت‌های اصلی Google Antigravity به ایجنت امکان می‌دهند با فایل‌های پروژه کار کند، دستورات ترمینال را اجرا کند، از ابزارهای مرورگر برای بررسی نتیجه کمک بگیرد و خروجی‌هایی برای توضیح و ارزیابی مراحل کار ارائه دهد. همچنین، مدیریت وظایف و اجرای موازی می‌تواند در پروژه‌های پیچیده به تقسیم کار و هماهنگی بهتر کمک کند.

با این حال، هیچ‌یک از این قابلیت‌ها به معنای بی‌نیازی از نظارت کاربر نیست. پیش از پذیرش تغییرات باید فایل‌ها و دستورات بررسی شوند، نتیجه‌ی اجرای برنامه آزمایش شود و گزارش‌های ایجنت با شواهد واقعی مقایسه شوند.

فصل پنجم: تنظیمات و شخصی‌سازی ایجنت

یکی از مهم‌ترین ویژگی‌های ابزارهای مبتنی بر ایجنت، امکان تنظیم نحوه‌ی عملکرد آن‌ها متناسب با نیاز کاربر و شرایط پروژه است. اگر ایجنت بدون شناخت ساختار پروژه، قواعد کدنویسی و هدف نهایی فعالیت کند، ممکن است راه‌حل‌هایی ارائه دهد که از نظر فنی درست باشند، اما با نیازهای واقعی پروژه سازگار نباشند.

درGoogle Antigravity ، شناخت تنظیمات، انتخاب مدل مناسب، تعریف دستورالعمل‌های پروژه و مدیریت اطلاعات در اختیار ایجنت می‌تواند به بهبود کیفیت خروجی‌ها کمک کند. هدف این فصل، آشنایی با روش‌هایی است که به کمک آن‌ها می‌توان رفتار ایجنت را بهتر هدایت کرد و از تکرار توضیحات و بروز اشتباهات قابل پیشگیری کاست.

تنظیمات اصلی ایجنت

تنظیمات ایجنت شامل گزینه‌هایی است که نحوه‌ی تعامل با ابزار، اجرای وظایف و استفاده از قابلیت‌های مختلف را تحت تأثیر قرار می‌دهند. محل قرارگیری و نام دقیق این گزینه‌ها ممکن است با توجه به نسخه‌ی Antigravityو تنظیمات حساب کاربر تغییر کند؛ بنابراین، بهتر است هنگام کار با ابزار، گزینه‌های موجود در نسخه‌ی نصب‌شده بررسی شوند.

برخی از تنظیمات و مواردی که هنگام کار با ایجنت اهمیت دارند عبارت‌اند از:

تنظیمات مربوط به نحوه‌ی تعامل: این تنظیمات می‌توانند بر شیوه‌ی ارسال درخواست‌ها، مشاهده‌ی فعالیت‌های ایجنت و تعامل کاربر با آن تأثیر بگذارند. برای مثال، در یک پروژه‌ی آموزشی ممکن است بخواهید ایجنت علاوه بر تولید کد، مراحل انجام کار را نیز توضیح دهد؛ اما در یک وظیفه‌ی ساده، پاسخ کوتاه‌تر و مستقیم‌تر کافی باشد.

مجوزهای اجرای عملیات

برخی فعالیت‌های ایجنت، مانند خواندن فایل‌ها، ویرایش کد یا اجرای دستورات ترمینال، ممکن است به مجوزهای مشخصی نیاز داشته باشند. تنظیمات مربوط به تأیید عملیات تعیین می‌کنند که کدام فعالیت‌ها به تأیید کاربر نیاز دارند و کدام فعالیت‌ها می‌توانند با محدودیت‌های تعیین‌شده انجام شوند.

برای نمونه، می‌توان برای اجرای یک دستور ساده‌ی آزمایشی رویکردی متفاوت از اجرای دستوری در نظر گرفت که فایل‌های زیادی را حذف می‌کند یا تنظیمات مهم سیستم را تغییر می‌دهد. هدف، ایجاد تعادل میان سرعت کار و کنترل انسانی است.

تنظیمات محیط اجرا و ابزارها

ایجنت برای انجام بعضی وظایف از ابزارهایی مانند ترمینال، ویرایشگر یا مرورگر استفاده می‌کند. دسترسی به این ابزارها و نحوه‌ی استفاده از آن‌ها به محیط کاری و تنظیمات موجود بستگی دارد. پیش از سپردن یک وظیفه‌ی پیچیده، باید مطمئن شوید ابزار موردنیاز در دسترس است و مجوز لازم را دارد.

تنظیمات مربوط به مدل

در صورتی که نسخه‌ی مورد استفاده امکان انتخاب مدل را فراهم کند، می‌توان بر اساس نوع وظیفه، مدل مناسب را انتخاب کرد. مدل‌های مختلف ممکن است از نظر سرعت، کیفیت استدلال، هزینه‌ی استفاده یا محدودیت‌های دسترسی با یکدیگر تفاوت داشته باشند.

نکته: فعال بودن یک قابلیت در یک نسخه یا حساب کاربری به معنای در دسترس بودن آن برای همه‌ی کاربران نیست. بهتر است به‌جای اتکا به آموزش‌های قدیمی، تنظیمات واقعی محیط خود را بررسی کنید.

انتخاب مدل و تنظیمات استدلال

مدل هوش مصنوعی بخشی است که درخواست را پردازش می‌کند، اطلاعات موجود را تحلیل می‌کند و در تولید پاسخ یا انتخاب مراحل انجام وظیفه نقش دارد. در یک محیط ایجنت‌محور، مدل فقط متن تولید نمی‌کند؛ بلکه ممکن است در برنامه‌ریزی مراحل، انتخاب ابزار، تحلیل نتایج و تصمیم‌گیری درباره‌ی گام بعدی نیز نقش داشته باشد.

با این حال، همه‌ی وظایف به یک سطح از توانایی و تحلیل نیاز ندارند. انتخاب مدل یا سطح استدلال مناسب می‌تواند بر سرعت و کیفیت انجام کار تأثیر بگذارد.

انتخاب مدل بر اساس نوع وظیفه

هنگام انتخاب مدل، بهتر است چند عامل را در نظر بگیرید:

-پیچیدگی وظیفه: تغییر یک عنوان در صفحه‌ی وب به تحلیل کمتری نسبت به طراحی ساختار یک برنامه‌ی بزرگ نیاز دارد.

-دقت موردنیاز: در وظایفی که خطا می‌تواند باعث خرابی بخش‌های مهم پروژه شود، بررسی دقیق‌تر اهمیت بیشتری دارد.

-سرعت انجام کار: برای پرسش‌های ساده و تغییرات کوچک، استفاده از گزینه‌ای سریع‌تر ممکن است کافی باشد.

-محدودیت‌های دسترسی و هزینه: دسترسی به مدل‌ها و میزان مصرف مجاز آن‌ها ممکن است به طرح کاربری و تنظیمات سرویس وابسته باشد.

برای مثال، اگر بخواهید نام یک متغیر را تغییر دهید، معمولاً نیازی به تحلیل گسترده‌ی کل پروژه نیست. اما اگر هدف شما بررسی علت یک خطای پیچیده در چند فایل مرتبط باشد، تحلیل عمیق‌تر می‌تواند مفید باشد.

تنظیمات استدلال چیست؟

استدلال به فرایندی گفته می‌شود که مدل برای تحلیل مسئله، بررسی گزینه‌ها و رسیدن به یک راه‌حل به کار می‌گیرد. در بعضی ابزارها، کاربر می‌تواند سطح یا میزان استدلال را انتخاب کند؛ البته نام گزینه‌ها و تعداد سطوح در محصولات و نسخه‌های مختلف یکسان نیست.

به‌طور کلی، تفاوت میان سطح‌های استدلال را می‌توان این‌گونه درک کرد:

-سطح پایین‌تر: برای وظایف ساده، محدود و نسبتاً مشخص مناسب است؛ مانند توضیح یک تابع یا تغییر جزئی در کد.

-سطح متوسط: برای وظایفی که به بررسی چند بخش مرتبط نیاز دارند، مانند اصلاح یک قابلیت کوچک و اجرای آزمون‌های مربوط به آن، می‌تواند انتخاب مناسبی باشد.

-سطح بالاتر: برای مسائل پیچیده‌تر، تحلیل خطاهای چندمرحله‌ای یا بررسی راه‌حل‌های جایگزین مفید است؛ اما ممکن است زمان یا منابع بیشتری مصرف کند.

این دسته‌بندی، توضیحی کلی درباره‌ی مفهوم سطح استدلال است و لزوماً نام گزینه‌های موجود در Antigravityنیست. برای انتخاب دقیق باید گزینه‌های واقعی نسخه‌ی مورد استفاده را بررسی کنید.

آیا استدلال بیشتر همیشه بهتر است؟

خیر. افزایش میزان استدلال لزوماً به معنای تولید خروجی درست‌تر نیست. اگر درخواست مبهم باشد، فایل‌های مهم در اختیار ایجنت قرار نگرفته باشند یا مسئله به اطلاعاتی خارج از دسترس آن نیاز داشته باشد، تحلیل بیشتر هم ممکن است به نتیجه‌ی اشتباه منجر شود.

برای استفاده‌ی مؤثر، ابتدا وظیفه را روشن کنید، اطلاعات لازم را در اختیار ایجنت قرار دهید و سپس متناسب با پیچیدگی کار، مدل یا تنظیمات مناسب را انتخاب کنید.

مثال عملی:

فرض کنید می‌خواهید خطایی را در یک پروژه‌ی پایتون رفع کنید. به‌جای اینکه صرفاً بنویسید «کدم را درست کن»، می‌توانید از ایجنت بخواهید ابتدا پیام خطا و فایل‌های مرتبط را بررسی کند، علت احتمالی را توضیح دهد، کمترین تغییر لازم را پیشنهاد کند و پس از تأیید شما، آزمون مناسب را اجرا کند.

در این مثال، روشن بودن مراحل کار به‌اندازه‌ی انتخاب مدل اهمیت دارد.

قوانین پروژه (Rules) و فایلAGENTS.md

یکی از مشکلات رایج هنگام کار با ایجنت‌ها، تکرار دستورالعمل‌های یکسان در درخواست‌های مختلف است. برای مثال، ممکن است لازم باشد هر بار یادآوری کنید که کد باید با نسخه‌ی مشخصی از پایتون سازگار باشد، نام متغیرها به زبان انگلیسی نوشته شود یا پیش از تغییر فایل‌های اصلی، ساختار پروژه بررسی شود.

قوانین پروژه و فایل‌های دستورالعملی می‌توانند به ثبت این اطلاعات کمک کنند تا در وظایف بعدی، ایجنت بتواند آن‌ها را به‌عنوان راهنمای کار در نظر بگیرد.

قوانین پروژه چه هستند؟

قوانین پروژه مجموعه‌ای از دستورالعمل‌ها هستند که مشخص می‌کنند ایجنت هنگام کار در یک پروژه باید چه اصولی را رعایت کند. این قوانین می‌توانند درباره‌ی ساختار فایل‌ها، سبک کدنویسی، ابزارهای مجاز، شیوه‌ی آزمایش و محدودیت‌های پروژه باشند.

قوانین خوب باید روشن، مشخص و قابل بررسی باشند. برای مثال، جمله‌ی «کد تمیز بنویس» به‌تنهایی معیار دقیقی ارائه نمی‌دهد؛ اما جمله‌ی «توابع طولانی را در صورت امکان به توابع کوچک‌تر تقسیم کن و پیش از افزودن وابستگی جدید، ضرورت آن را توضیح بده» راهنمای عملی‌تری است.

نمونه‌هایی از قوانین مفید برای یک پروژه عبارت‌اند از:

-پیش از تغییر کد، فایل‌های مرتبط و ساختار پروژه را بررسی کن.

-از چارچوب‌ها و کتابخانه‌های موجود پروژه استفاده کن و بدون ضرورت، وابستگی جدید اضافه نکن.

-تغییرات را تا حد امکان به وظیفه‌ی درخواستی محدود نگه دار.

-پس از اصلاح کد، آزمون‌های مرتبط را اجرا کن؛ اگر امکان اجرای آزمون وجود ندارد، این موضوع را گزارش بده.

-هیچ‌گاه اطلاعات محرمانه، رمز عبور یا کلیدهای دسترسی را در کد یا گزارش‌ها قرار نده.

-اگر درخواست با ساختار فعلی پروژه ناسازگار است، پیش از اعمال تغییر گسترده، مشکل را توضیح بده.

فایل AGENTS.md چیست؟

فایل `AGENTS.md` نامی رایج برای نگهداری دستورالعمل‌های مخصوص ایجنت‌های برنامه‌نویسی است. این فایل معمولاً به‌صورت متن ساده نوشته می‌شود و می‌تواند شامل توضیح پروژه، قواعد کدنویسی، روش اجرای برنامه و محدودیت‌های مهم باشد.

برای مثال، در یک پروژه‌ی پایتون می‌توان چنین محتوایی را در فایل `AGENTS.md` قرار داد:

نمونه‌ی محتوای فایل:

-زبان اصلی پروژهPython :

-پیش از تغییر کد، ساختار پوشه‌ها و فایل‌های مرتبط را بررسی کن.

-برای نصب وابستگی‌ها، ابتدا فایل‌های مدیریت وابستگی موجود را بررسی کن.

-تغییرات را کوچک و مرتبط با درخواست نگه دار.

-پس از تغییر، آزمون‌های موجود را اجرا کن.

-در پایان، فایل‌های تغییرکرده و نتیجه‌ی آزمون‌ها را گزارش بده.

-فایل‌های حاوی اطلاعات محرمانه را نخوان یا تغییر نده، مگر اینکه وظیفه و مجوز آن به‌طور مشخص چنین کاری را ضروری کند.

این نمونه صرفاً یک دستورالعمل پیشنهادی است و باید با نیازهای واقعی پروژه سازگار شود.

چگونه از AGENTS.md در Antigravity استفاده کنیم؟

پیش از استفاده، باید بررسی کنید که نسخه‌ی مورد استفاده‌ی Antigravity از چه سازوکارهایی برای دریافت دستورالعمل‌های پروژه پشتیبانی می‌کند و آیا فایل `AGENTS.md` را به‌طور مستقیم و خودکار می‌خواند یا خیر. وجود این فایل در پوشه‌ی پروژه به‌تنهایی تضمین نمی‌کند که هر ایجنت یا ابزاری آن را به‌صورت خودکار تشخیص دهد.

اگر پشتیبانی مستقیم وجود نداشته باشد، می‌توانید محتوای دستورالعمل را در قالب یا محل مورد پشتیبانی ابزار تعریف کنید یا در صورت لزوم، از ایجنت بخواهید فایل را بخواند. با این حال، باید مطمئن شوید که این کار واقعاً انجام شده است و صرفاً بر اساس نام فایل فرض نکنید که قوانین رعایت می‌شوند.

همچنین، دستورالعمل‌های پروژه جایگزین مجوزهای امنیتی نیستند. برای مثال، نوشتن جمله‌ی «هیچ فایل مهمی را حذف نکن» در یک فایل راهنما، به‌تنهایی تضمین نمی‌کند که ایجنت قادر به حذف فایل نباشد. برای عملیات حساس باید از محدودیت‌های دسترسی، تأیید کاربر و نسخه‌ی پشتیبان نیز استفاده کرد.

تفاوت قوانین پروژه با درخواست معمولی

درخواست معمولی، دستور مربوط به وظیفه‌ی فعلی است؛ اما قوانین پروژه، اصولی هستند که انتظار می‌رود در چندین وظیفه رعایت شوند.

برای نمونه:

-درخواست معمولی: «یک تابع برای اعتبارسنجی ایمیل اضافه کن».

-قانون پروژه: «برای اعتبارسنجی ورودی‌ها، ابتدا راهکارهای موجود در پروژه را بررسی کن و از ایجاد منطق تکراری خودداری کن».

این دو مورد مکمل یکدیگرند. قانون پروژه چارچوب کلی کار را مشخص می‌کند و درخواست معمولی هدف فعلی را تعیین می‌کند.

مهارت‌ها (Skills) و گردش‌کارهای قابل استفاده‌ی مجدد

در برخی محیط‌های ایجنت‌محور، مفهوم مهارت یا Skill برای تعریف دستورالعمل‌ها و گردش‌کارهای قابل استفاده‌ی مجدد به کار می‌رود. مهارت به ایجنت کمک می‌کند یک نوع وظیفه را بر اساس روشی مشخص انجام دهد؛ به‌ویژه زمانی که انجام آن وظیفه به مراحل تکرارشونده، قواعد خاص یا دانش تخصصی نیاز دارد.

برای مثال، اگر در پروژه‌های مختلف به‌طور مرتب فایل‌های پایتون را بررسی کنید، می‌توانید گردش‌کاری داشته باشید که مراحل بررسی کیفیت کد، شناسایی خطاهای احتمالی، اجرای آزمون‌ها و ارائه‌ی گزارش را مشخص کند.

مهارت چه کمکی می‌کند؟

مهارت می‌تواند چند فایده داشته باشد:

-کاهش تکرار: لازم نیست در هر درخواست تمام مراحل یکسان را از ابتدا توضیح دهید.

-یکپارچگی عملکرد: وظایف مشابه با روشی نزدیک به هم انجام می‌شوند.

-ثبت روش کار: مراحل انجام یک کار مشخص می‌شوند و امکان اصلاح و بهبود آن‌ها وجود دارد.

-افزایش قابلیت استفاده‌ی مجدد: یک گردش‌کار مناسب را می‌توان در موقعیت‌های مشابه دوباره به کار گرفت.

البته مهارت به‌تنهایی تضمین‌کننده‌ی کیفیت خروجی نیست. دستورالعمل ناقص یا قدیمی ممکن است باعث شود ایجنت یک روش نامناسب را به‌صورت تکراری اجرا کند.

یک نمونه از گردش‌کار قابل استفاده‌ی مجدد

فرض کنید می‌خواهید برای بررسی کدهای یک پروژه، روشی ثابت داشته باشید. گردش‌کار شما می‌تواند این مراحل را شامل شود:

۱ .ساختار پروژه و فایل‌های مرتبط را بررسی کن.

۲ .هدف فایل یا قابلیت مورد بررسی را مشخص کن.

۳ .خطاهای احتمالی، کدهای تکراری و مشکلات خوانایی را شناسایی کن.

۴ .پیش از اعمال تغییرات گسترده، یافته‌ها و پیشنهادها را گزارش بده.

۵ .پس از تأیید کاربر، تغییرات ضروری را اعمال کن.

۶ .آزمون‌های مرتبط را اجرا کن و نتیجه را همراه با محدودیت‌های باقی‌مانده گزارش بده.

این گردش‌کار را می‌توان در قالب یک مهارت یا سازوکار دستورالعملی پشتیبانی‌شده در محیط مورد استفاده ثبت کرد. قالب فایل‌ها، روش نصب و نحوه‌ی فراخوانی مهارت‌ها به ابزار و نسخه‌ی آن بستگی دارد؛ بنابراین، باید از مستندات همان محیط پیروی کرد.

تفاوت Skills با Rules و ابزارها

برای درک بهتر این مفاهیم، باید نقش هرکدام را از یکدیگر جدا کنیم:

Rules: اصول و محدودیت‌هایی را مشخص می‌کنند که ایجنت باید هنگام کار در نظر بگیرد؛ مانند سبک کدنویسی یا محدود کردن تغییرات.

Skills: روش انجام یک نوع وظیفه را به شکل قابل استفاده‌ی مجدد تعریف می‌کنند؛ مانند بررسی کد یا آماده‌سازی گزارش آزمون.

Tools: امکاناتی هستند که ایجنت با کمک آن‌ها عملیاتی انجام می‌دهد؛ مانند خواندن فایل، اجرای دستور ترمینال یا دسترسی به یک سرویس خارجی.

برای مثال، یک قانون می‌تواند ایجنت را ملزم کند که پیش از نصب کتابخانه‌ی جدید، ضرورت آن را توضیح دهد. یک مهارت می‌تواند مراحل بررسی و اصلاح وابستگی‌ها را تعریف کند و ابزار ترمینال می‌تواند برای اجرای دستور نصب به کار برود.

این مفاهیم با یکدیگر ارتباط دارند، اما جایگزین یکدیگر نیستند. همچنین، پشتیبانی مستقیم Antigravity از هر قالب مهارت یا روش فراخوانی آن باید در نسخه‌ی مورد استفاده بررسی شود.

مدیریت زمینه و اطلاعات موردنیاز ایجنت

زمینه یا Context به اطلاعاتی گفته می‌شود که ایجنت برای درک درخواست و انجام وظیفه در اختیار دارد. این اطلاعات می‌توانند شامل پیام‌های قبلی، توضیحات کاربر، محتوای فایل‌ها، ساختار پروژه، دستورالعمل‌ها و نتایج ابزارهای استفاده‌شده باشند.

کیفیت زمینه نقش مهمی در کیفیت عملکرد ایجنت دارد. اگر اطلاعات ضروری در دسترس نباشند، ممکن است ایجنت بر اساس فرض‌های نادرست تصمیم بگیرد. از طرف دیگر، ارائه‌ی اطلاعات زیاد و نامرتبط نیز می‌تواند تمرکز بر مسئله‌ی اصلی را دشوار کند.

چرا مدیریت زمینه اهمیت دارد؟

فرض کنید از ایجنت می‌خواهید یک خطا را در پروژه‌ی خود رفع کند، اما فقط پیام «برنامه کار نمی‌کند» را می‌فرستید. در این حالت، ایجنت اطلاعات کافی درباره‌ی محل خطا، رفتار مورد انتظار و شرایط اجرای برنامه ندارد.

در مقابل، اگر نسخه‌ی خطا، مراحل بازتولید آن، فایل‌های مرتبط و نتیجه‌ی اجرای قبلی را ارائه کنید، احتمال رسیدن به راه‌حل مناسب بیشتر می‌شود.

مدیریت زمینه یعنی اطلاعات مرتبط را به‌صورت روشن و هدفمند در اختیار ایجنت قرار دهید و مطمئن شوید که محدودیت‌ها و تصمیم‌های مهم پروژه از قلم نیفتاده‌اند.

روش‌های مدیریت بهتر زمینه

۱. هدف را مشخص کنید

به‌جای درخواست کلی، نتیجه‌ی مورد انتظار را توضیح دهید. برای مثال، به‌جای «کد را بهتر کن» بنویسید: «تابع اعتبارسنجی ایمیل را بررسی کن، خطاهای احتمالی را مشخص کن و بدون تغییر بخش‌های دیگر، اصلاحات ضروری را پیشنهاد بده».

۲ .اطلاعات مرتبط را ارائه کنید

اگر وظیفه به فایل یا تابع خاصی مربوط است، نام و محل آن را مشخص کنید. در صورت لزوم، توضیح دهید کدام فایل‌ها منبع اصلی اطلاعات هستند و کدام بخش‌ها نباید تغییر کنند.

۳ .تصمیم‌های مهم را ثبت کنید

اگر در طول کار تصمیم گرفته‌اید از یک کتابخانه‌ی خاص استفاده کنید، ساختار پوشه‌ها را تغییر ندهید یا روش مشخصی برای مدیریت داده‌ها انتخاب کنید، این تصمیم‌ها را در دستورالعمل‌های پروژه یا مستندات مناسب ثبت کنید.

۴ .وظایف بزرگ را به بخش‌های کوچک تقسیم کنید

وقتی چند هدف متفاوت را در یک درخواست ترکیب می‌کنید، بررسی خروجی دشوارتر می‌شود. بهتر است پروژه‌های بزرگ به وظایف کوچک و قابل ارزیابی تقسیم شوند؛ برای مثال، ابتدا بررسی ساختار، سپس اصلاح یک بخش، بعد اجرای آزمون‌ها و در نهایت مستندسازی.

۵ .در صورت تغییر موضوع، زمینه‌ی لازم را دوباره روشن کنید

در گفت‌وگوهای طولانی، ممکن است بخشی از اطلاعات قبلی دیگر مرتبط نباشد یا ایجنت جزئیات مهم را به‌درستی در نظر نگیرد. در چنین شرایطی، خلاصه‌ای از هدف فعلی، تصمیم‌های مهم و وضعیت پروژه ارائه کنید. اگر ابزار امکان شروع یک وظیفه‌ی جدید یا ایجاد گفت‌وگوی جداگانه را فراهم می‌کند، این کار می‌تواند برای تفکیک موضوعات مفید باشد.

۶ .اطلاعات محرمانه را وارد زمینه نکنید.

برای رفع خطا معمولاً نیازی نیست رمز عبور، کلیدهای دسترسی یا اطلاعات شخصی واقعی را در اختیار ایجنت قرار دهید. در صورت نیاز به نمونه‌ی داده، از اطلاعات ساختگی استفاده کنید و فایل‌های محرمانه را با تنظیمات دسترسی مناسب محافظت کنید.

زمینه‌ی مناسب چه ویژگی‌هایی دارد؟

زمینه‌ی مناسب لزوماً طولانی نیست؛ بلکه باید برای وظیفه‌ی موردنظر کافی، مرتبط و دقیق باشد. پیش از ارسال درخواست، این پرسش‌ها را بررسی کنید:

- آیا هدف و نتیجه‌ی مورد انتظار مشخص است؟
- آیا ایجنت اطلاعات لازم برای شناخت بخش مربوط به پروژه را دارد؟

- آیا محدودیت‌ها و تصمیم‌های مهم توضیح داده شده‌اند؟
- آیا معیار مشخصی برای تشخیص موفقیت کار وجود دارد؟
- آیا اطلاعات غیرضروری یا محرمانه در درخواست وجود دارد؟

مثال عملی: ارائه‌ی زمینه برای رفع خطا

فرض کنید برنامه‌ی پایتون هنگام خواندن یک فایل CSV با خطا روبه‌رو می‌شود. درخواست ضعیف این است:

"برنامه‌ام خطا می‌دهد؛ درستش کن"

درخواست دقیق‌تر می‌تواند چنین باشد:

»در فایل data_loader.py هنگام خواندن فایل CSV خطا دریافت می‌کنم. لطفاً ابتدا کد مرتبط و پیام کامل خطا را بررسی کن. مشخص کن مشکل از مسیر فایل، ساختار داده یا روش خواندن آن است. پیش از تغییر کد، علت احتمالی را توضیح بده. سپس کمترین تغییر لازم را پیشنهاد کن و پس از تأیید من، آزمون مرتبط را اجرا کن. بخش‌های نامرتبط پروژه را تغییر نده«.

در درخواست دوم، هدف، محل بررسی، ترتیب کار، محدودیت تغییرات و معیار ارزیابی مشخص‌تر هستند. این اطلاعات به ایجنت کمک می‌کنند به‌جای حدس‌زدن، مسئله را مرحله‌به‌مرحله بررسی کند.

شخصی‌سازی ایجنت به معنای فراهم کردن چارچوبی روشن برای انجام وظایف است. تنظیمات اصلی، انتخاب مدل و سطح استدلال، قوانین پروژه، مهارت‌ها و مدیریت زمینه هرکدام نقش متفاوتی در این فرایند دارند.

برای استفاده‌ی مؤثر ازAntigravity ، لازم نیست از ابتدا همه‌ی تنظیمات را تغییر دهید. بهتر است ابتدا با گزینه‌های پیش‌فرض آشنا شوید، یک پروژه‌ی آزمایشی بسازید و سپس متناسب با نیاز واقعی خود، دستورالعمل‌ها و گردش‌کارهای مناسب را اضافه کنید.

به یاد داشته باشید که هیچ تنظیم یا فایل دستورالعملی نمی‌تواند جایگزین بررسی خروجی، آزمون کد و کنترل دسترسی‌ها شود. ایجنت زمانی بیشترین فایده را دارد که در کنار دستورالعمل‌های روشن، اطلاعات کافی و نظارت انسانی مناسب فعالیت کند.

فصل ششم: ابزارهای اتصال و قابلیت‌های پیشرفته

یکی از مزیت‌های ایجنت‌های برنامه‌نویسی، امکان استفاده از ابزارها و منابع مختلف برای انجام وظایف پیچیده‌تر است. ایجنتی که تنها به متن درخواست و کدهای موجود دسترسی دارد، ممکن است برای انجام

برخی کارها با محدودیت روبه‌رو شود؛ اما با اتصال ابزارهای مناسب می‌تواند به منابع اطلاعاتی، سرویس‌های خارجی و قابلیت‌های تکمیلی دسترسی پیدا کند.

Google Antigravityبا رویکردی ایجنت‌محور طراحی شده است و امکانات آن می‌تواند به انجام وظایفی فراتر از تولید کد کمک کند. بااین‌حال، قابلیت‌های دقیق اتصال، روش پیکربندی و ابزارهای قابل استفاده به نسخه، تنظیمات و امکانات در دسترس کاربر بستگی دارند. بنابراین، در این فصل علاوه بر معرفی مفاهیم اصلی، اصول استفاده‌ی صحیح و ایمن از این قابلیت‌ها را بررسی می‌کنیم.

پروتکل MCP و اتصال ابزارهای خارجی

MCPچیست؟

MCPمخفف عبارت Model Context Protocol و به معنای «پروتکل زمینه‌ی مدل» است. این پروتکل یک استاندارد برای ارتباط میان برنامه‌های هوش مصنوعی و ابزارها یا منابع اطلاعاتی خارجی فراهم می‌کند.

برای درک بهترMCP ، فرض کنید یک ایجنت برنامه‌نویسی می‌تواند کدهای پروژه را بخواند و ویرایش کند، اما برای انجام یک وظیفه به اطلاعات موجود در یک پایگاه داده یا محتوای یک سرویس خارجی نیاز دارد. اگر اتصال مناسب وجود داشته باشد، ایجنت می‌تواند از ابزارهای ارائه‌شده توسط آن سرویس استفاده کند؛ البته فقط در محدوده‌ی دسترسی و مجوزهای تعریف‌شده.

MCPبه‌عنوان یک روش استاندارد ارتباطی کمک می‌کند ابزارها و منابع مختلف، بدون نیاز به طراحی یک روش ارتباطی کاملاً اختصاصی برای هر ترکیب، به برنامه‌های سازگار متصل شوند.

اجزای اصلیMCP

برای شناخت این پروتکل، آشنایی با چند مفهوم اصلی ضروری است:

- Host

برنامه‌ای است که محیط اصلی تعامل با مدل را فراهم می‌کند و ارتباط کلی با اجزای MCP را مدیریت می‌کند. یک محیط ایجنت‌محور می‌تواند در نقش میزبان عمل کند.

- MCP Client

بخشی است که ارتباط با یک سرور MCP را برقرار و پیام‌ها را مطابق پروتکل ردوبدل می‌کند. یک برنامه ممکن است برای ارتباط با چند سرور، چند اتصال یا کلاینت داشته باشد.

- MCP Server

سرویسی است که قابلیت‌ها یا منابع مشخصی را در اختیار برنامه‌ی متصل قرار می‌دهد. این سرویس ممکن است امکان جست‌وجو، خواندن اطلاعات، دسترسی به منابع یا اجرای عملیات خاصی را فراهم کند.

- Tools: ابزارها عملیات قابل فراخوانی هستند؛ مانند جست‌وجو در یک منبع، دریافت اطلاعات یا اجرای یک عملیات مشخص. ایجنت می‌تواند در صورت پشتیبانی محیط و با رعایت مجوزها از این ابزارها استفاده کند.
- Resources

منابع، اطلاعاتی هستند که از طریق سرور در اختیار برنامه قرار می‌گیرند؛ برای مثال، محتوای یک سند یا اطلاعات ساختاریافته‌ای که ابزار به‌عنوان منبع ارائه می‌کند.

- Prompts

برخی سرورهای MCP می‌توانند الگوهای آماده‌ی درخواست را ارائه کنند تا انجام وظایف خاص ساده‌تر شود.

همه‌ی سرورهای MCP الزاماً تمام این قابلیت‌ها را ارائه نمی‌کنند. امکانات واقعی هر سرور به طراحی و پیاده‌سازی آن بستگی دارد.

MCPچه کاربردی در توسعه‌ی نرم‌افزار دارد؟

کاربرد MCP به نوع ابزار متصل‌شده بستگی دارد. چند نمونه‌ی متداول عبارت‌اند از:

-دسترسی به مستندات: کمک به یافتن اطلاعات فنی موردنیاز برای استفاده از یک کتابخانه یا چارچوب.

-جست‌وجو در منابع اطلاعاتی: بازیابی اطلاعات از منابعی که سرور مربوطه برای آن‌ها دسترسی تعریف کرده است.

-تعامل با سامانه‌های توسعه: دریافت اطلاعات یا انجام عملیات پشتیبانی‌شده در سرویس‌های مرتبط با پروژه.

-کار با داده‌ها: دسترسی کنترل‌شده به اطلاعات یک منبع داده، در صورت وجود ابزار مناسب.

-تسهیل گردش‌کارها: ترکیب چند ابزار برای انجام وظایفی که به چند مرحله یا منبع نیاز دارند.

برای مثال، فرض کنید در حال توسعه‌ی یک برنامه‌ی وب هستید و برای استفاده از یک کتابخانه به مستندات آن نیاز دارید. اگر ابزار مناسب و قابل اعتماد از طریق MCP در دسترس باشد، ایجنت می‌تواند مستندات مرتبط را بازیابی کند و بر اساس آن‌ها پیشنهاد دقیق‌تری ارائه دهد.

آیا MCP به‌تنهایی قابلیت جدیدی به ایجنت اضافه می‌کند؟

MCPبه‌خودی‌خود یک مدل هوش مصنوعی نیست و به‌تنهایی توانایی تحلیل یا برنامه‌نویسی ایجاد نمی‌کند. این پروتکل راه ارتباطی استانداردی میان برنامه و قابلیت‌های خارجی فراهم می‌کند.

برای استفاده‌ی واقعی، باید یک محیط سازگار، سرور MCP مناسب و دسترسی لازم وجود داشته باشد. کیفیت و امنیت عملکرد نیز به سرور، ابزارهای ارائه‌شده، مجوزها و نحوه‌ی استفاده‌ی ایجنت از آن‌ها وابسته است.

نکته‌ی مهم درباره‌یAntigravity

پیش از پیکربندی MCP درAntigravity ، مستندات رسمی نسخه‌ی مورد استفاده را بررسی کنید تا مشخص شود چه روش‌هایی برای تعریف اتصال، تنظیم سرور، مدیریت مجوزها و عیب‌یابی پشتیبانی می‌شوند. نباید فرض کرد تمام نمونه‌های پیکربندی مربوط به سایر محیط‌های ایجنت‌محور، بدون تغییر در Antigravityنیز قابل استفاده‌اند.

اتصال سرویس‌ها و منابع اطلاعاتی

اتصال سرویس‌های خارجی به ایجنت می‌تواند دامنه‌ی کارهایی را که انجام می‌دهد گسترش دهد. برای مثال، در یک پروژه‌ی نرم‌افزاری ممکن است لازم باشد اطلاعات مستندات، وضعیت وظایف تیم، داده‌های آزمایشی یا محتوای یک پایگاه داده بررسی شوند.

این اتصال‌ها می‌توانند از روش‌های مختلفی انجام شوند؛ مانندMCP ، رابط برنامه‌نویسی کاربردی(API) ، ابزارهای داخلی محیط یا یکپارچه‌سازی‌های از پیش آماده‌شده. روش مناسب به سرویس موردنظر و قابلیت‌های محیط بستگی دارد.

APIچیست؟

APIمخفف Application Programming Interface و به معنای «رابط برنامه‌نویسی کاربردی» است API .مجموعه‌ای از قواعد و روش‌هاست که به یک نرم‌افزار اجازه می‌دهد با نرم‌افزار یا سرویس دیگری ارتباط برقرار کند.

برای مثال، یک سرویس ممکن است از طریق API امکان دریافت اطلاعات، ثبت یک رکورد یا جست‌وجو در داده‌ها را فراهم کند. اگر ایجنت از طریق یک ابزار یا اتصال مجاز به این API دسترسی داشته باشد، می‌تواند در چارچوب عملیات پشتیبانی‌شده از آن استفاده کند.

MCPو API یک مفهوم واحد نیستند .API رابطی برای استفاده از قابلیت‌های یک سرویس است؛ MCP نیز پروتکلی استاندارد برای ارتباط برنامه‌های هوش مصنوعی با ابزارها و منابع فراهم می‌کند. در برخی پیاده‌سازی‌ها، یک سرور MCP می‌تواند در پشت صحنه با API یک سرویس ارتباط برقرار کند.

نمونه‌هایی از اتصال سرویس‌ها

اتصال به منابع مستندات

فرض کنید برای استفاده از یک کتابخانه‌ی برنامه‌نویسی به مستندات آن نیاز دارید. اگر اتصال مناسب وجود داشته باشد، ایجنت می‌تواند اطلاعات مرتبط را بازیابی کند و در توضیح یا اصلاح کد از آن‌ها کمک بگیرد.

اتصال به ابزارهای مدیریت پروژه

در یک تیم نرم‌افزاری، وظایف، گزارش خطاها و وضعیت پیشرفت پروژه ممکن است در یک سامانه‌ی مدیریت پروژه ثبت شوند. اتصال مجاز به چنین سامانه‌ای می‌تواند به ایجنت کمک کند اطلاعات مربوط به یک وظیفه را بررسی کند یا عملیات پشتیبانی‌شده را انجام دهد.

اتصال به پایگاه داده

در برخی پروژه‌ها، ایجنت ممکن است برای بررسی ساختار داده‌ها یا تحلیل اطلاعات آزمایشی به ابزار متصل‌شده به پایگاه داده نیاز داشته باشد. در این شرایط، باید نوع دسترسی مشخص باشد و برای وظایف عادی، دسترسی فقط‌خواندنی در اولویت قرار گیرد.

اتصال به سرویس‌های آزمایشی و توسعه

بعضی سرویس‌ها امکان بررسی وضعیت اجرای برنامه، دریافت گزارش خطا یا مشاهده‌ی نتایج آزمون را فراهم می‌کنند. اگر ابزار اتصال مناسب وجود داشته باشد، این اطلاعات می‌توانند در فرایند عیب‌یابی مفید باشند.

پیش از اتصال یک سرویس چه مواردی را بررسی کنیم؟

قبل از فعال کردن هر اتصال، بهتر است موارد زیر بررسی شوند:

اعتبار سرویس: آیا سرویس از منبعی معتبر و قابل اعتماد دریافت شده است؟

نوع دسترسی: آیا سرویس فقط اطلاعات را می‌خواند یا می‌تواند داده‌ها را تغییر دهد؟

دامنه‌ی مجوزها: آیا دسترسی به کل حساب لازم است یا می‌توان آن را به بخش مشخصی محدود کرد؟

اطلاعات محرمانه: آیا اتصال به کلیدAPI ، رمز عبور یا اطلاعات حساس نیاز دارد؟

عملیات حساس: آیا ابزار می‌تواند داده‌ها را حذف کند، پیام ارسال کند یا تغییری خارج از پروژه ایجاد کند؟

قابلیت لغو دسترسی: آیا می‌توان در صورت بروز مشکل، اتصال یا مجوزهای آن را غیرفعال کرد؟

وجود یک اتصال فعال به معنای امن بودن همه‌ی عملیات آن نیست. باید مشخص شود ابزار دقیقاً چه کاری انجام می‌دهد و آیا این کار با هدف پروژه سازگار است.

افزونه‌ها و ابزارهای تکمیلی

افزونه‌ها و ابزارهای تکمیلی می‌توانند امکانات یک محیط توسعه را گسترش دهند. برای مثال، ممکن است ابزاری برای پشتیبانی از یک زبان برنامه‌نویسی، بررسی کیفیت کد، مدیریت نسخه‌ها یا ارتباط با یک سرویس خارجی در اختیار کاربر قرار گیرد.

بااین‌حال، باید میان افزونه‌ی محیط توسعه، ابزار قابل استفاده توسط ایجنت و سرور MCP تفاوت قائل شد. این سه مفهوم ممکن است در برخی سناریوها با یکدیگر ارتباط داشته باشند، اما یکسان نیستند.

افزونه چیست؟

افزونه یا Extension نرم‌افزاری است که برای اضافه کردن یا بهبود قابلیت‌های یک برنامه طراحی می‌شود. افزونه‌ها ممکن است امکاناتی مانند تشخیص بهتر ساختار یک زبان برنامه‌نویسی، قالب‌بندی کد، بررسی خطاها یا اتصال به سرویس‌های خارجی را فراهم کنند.

برای مثال، یک افزونه‌ی مرتبط با پایتون ممکن است امکاناتی برای شناسایی فایل‌های پایتون یا کمک به بررسی خطاها ارائه کند. اما این موضوع لزوماً به معنای آن نیست که ایجنت می‌تواند تمام قابلیت‌های افزونه را به‌طور خودکار فراخوانی کند؛ این رفتار به طراحی محیط و نحوه‌ی یکپارچه‌سازی وابسته است.

تفاوت افزونه، ابزار و مهار

برای انتخاب روش مناسب، نقش هرکدام را به‌صورت جداگانه در نظر بگیرید:

افزونه: قابلیت‌های یک برنامه یا محیط توسعه را گسترش می‌دهد.

ابزار: امکان انجام یک عملیات مشخص را فراهم می‌کند؛ مانند اجرای آزمون یا جست‌وجو در فایل‌ها.

مهارت: روش یا گردش‌کار انجام یک نوع وظیفه را تعریف می‌کند.

سرور MCP: از طریق یک پروتکل استاندارد، ابزارها یا منابع را در اختیار یک برنامه‌ی سازگار قرار می‌دهد.

برای مثال، ممکن است یک افزونه امکان بررسی کیفیت کد را فراهم کند، اما ایجنت تنها زمانی می‌تواند مستقیماً از قابلیت مشخصی استفاده کند که محیط آن را در دسترس ایجنت قرار دهد.

اصول انتخاب ابزار تکمیلی

هنگام انتخاب افزونه یا ابزار، به نکات زیر توجه کنید:

- ابزار باید با نسخه و محیط کاری شما سازگار باشد.
- منبع دریافت آن باید معتبر باشد.
- مجوزهایی که درخواست می‌کند باید با کاربرد مورد انتظار تناسب داشته باشند.
- بهتر است ابتدا آن را در یک پروژه‌ی آزمایشی بررسی کنید.
- ابزارهای غیرضروری را نصب یا فعال نکنید.

پس از نصب، بررسی کنید که آیا ابزار واقعاً مشکل شما را حل می‌کند یا تنها پیچیدگی محیط را افزایش می‌دهد.

افزودن ابزارهای بیشتر همیشه به معنای عملکرد بهتر نیست. هر ابزار جدید می‌تواند نیاز به پیکربندی، نگهداری و بررسی امنیتی بیشتری ایجاد کند.

اگر ابزار موردنیاز در دسترس نباشد چه کنیم؟

اگر ابزار موردنیاز در Antigravity وجود ندارد، ابتدا بررسی کنید آیا راهکار پشتیبانی‌شده‌ای برای افزودن آن وجود دارد. ممکن است بتوان از قابلیت داخلی، افزونه‌ی سازگار، APIیا اتصال MCP استفاده کرد.

بااین‌حال، صرف امکان فنی اتصال به معنای مناسب بودن آن نیست. اگر ابزار به مجوزهای گسترده یا اطلاعات حساس نیاز دارد، باید پیش از فعال‌سازی، ضرورت آن را بررسی کنید. همچنین، اگر اتصال در نسخه‌ی مورد استفاده پشتیبانی نمی‌شود، بهتر است از روش جایگزین مستند و قابل کنترل استفاده شود.

کاربردهای پیشرفته و خودکارسازی وظایف

خودکارسازی به معنای سپردن مراحل تکراری یا قابل پیش‌بینی به ابزارها و گردش‌کارهای نرم‌افزاری است. در محیط‌های ایجنت‌محور، می‌توان برخی وظایف را با ترکیب دستورالعمل‌ها، ابزارها و سرویس‌های متصل‌شده ساده‌تر کرد.

برای مثال، بررسی اولیه‌ی یک خطای نرم‌افزاری می‌تواند شامل خواندن گزارش خطا، بررسی فایل مرتبط، پیشنهاد اصلاح و اجرای آزمون باشد. اگر ابزارهای لازم در دسترس باشند، ایجنت می‌تواند بخش‌هایی از این فرایند را انجام دهد.

البته میزان خودکارسازی ممکن است در هر محیط متفاوت باشد. نباید فرض کرد همه‌ی مراحل را می‌توان بدون دخالت کاربر اجرا کرد یا Antigravity از هر گردش‌کار خودکاری پشتیبانی می‌کند.

نمونه‌ی اول: خودکارسازی بررسی کد

فرض کنید می‌خواهید پس از هر تغییر مهم در کد، بررسی اولیه‌ای انجام شود. می‌توان گردش‌کاری با مراحل زیر تعریف کرد:

۱ .فایل‌های تغییرکرده را شناسایی کن.

۲ .تغییرات را از نظر ارتباط با وظیفه‌ی موردنظر بررسی کن.

۳ .خطاهای احتمالی، کدهای تکراری و مشکلات خوانایی را گزارش بده.

۴ .آزمون‌های مرتبط را در صورت امکان اجرا کن.

۵ .نتیجه‌ی بررسی و موارد نیازمند توجه را خلاصه کن.

این گردش‌کار می‌تواند به‌صورت دستورالعملی تکرارشونده تعریف شود. اجرای خودکار آن در زمان‌های مشخص یا پس از هر تغییر، به امکانات واقعی محیط و ابزارهای اتصال وابسته است.

نمونه‌ی دوم: کمک به مستندسازی پروژه

در یک پروژه‌ی بزرگ، ممکن است مستندات با تغییر کدها به‌روز نباشند. ایجنت می‌تواند برای شناسایی بخش‌هایی که نیاز به مستندسازی دارند، تهیه‌ی پیش‌نویس توضیحات و پیشنهاد تغییرات کمک کند.

برای مثال، می‌توان از آن خواست:

- فایل‌های اصلی پروژه را بررسی کند.
- هدف و مسئولیت هر ماژول را خلاصه کند.
- توابعی را که توضیحات کافی ندارند شناسایی کند.
- پیش‌نویس مستندات را تولید کند.
- پیش از اعمال تغییرات نهایی، نتیجه را برای بررسی ارائه دهد.

با این روش، کاربر می‌تواند از ایجنت برای تهیه‌ی مستندات کمک بگیرد، بدون آنکه مسئولیت بررسی صحت مطالب را کنار بگذارد.

نمونه‌ی سوم: ترکیب چند ابزار

گاهی یک وظیفه به چند ابزار مختلف نیاز دارد. برای نمونه، بررسی یک خطای نرم‌افزاری ممکن است شامل دسترسی به فایل‌های پروژه، اجرای یک دستور در ترمینال و دریافت اطلاعات از مستندات باشد.

در چنین شرایطی، ایجنت می‌تواند با استفاده از ابزارهای مجاز مراحل مختلف را به هم مرتبط کند. بااین‌حال، هر مرحله باید بر اساس نتیجه‌ی مرحله‌ی قبل و در محدوده‌ی مجوزهای تعریف‌شده انجام شود.

ترکیب ابزارها به‌ویژه زمانی مفید است که ترتیب انجام مراحل اهمیت دارد. اگر یک مرحله با خطا روبه‌رو شود، نباید ایجنت بدون بررسی علت، ادامه‌ی عملیات را به‌گونه‌ای انجام دهد که مشکل را پیچیده‌تر کند.

نمونه‌ی چهارم: استفاده از ایجنت در فرایند یادگیری

فرض کنید در حال یادگیری یک دوره‌ی برنامه‌نویسی هستید و می‌خواهید محتوای هر بخش را به یادداشت و تمرین تبدیل کنید. یک گردش‌کار پیشنهادی می‌تواند شامل این مراحل باشد:

۱ .دریافت متن یا محتوای مجاز درس.

۲ .استخراج مفاهیم اصلی و اصطلاحات تخصصی.

۳ .تهیه‌ی خلاصه‌ای ساختاریافته.

۴ .تولید چند پرسش برای ارزیابی یادگیری.

۵ .پیشنهاد تمرین عملی متناسب با سطح یادگیرنده.

۶ .بررسی پاسخ‌های کاربر و مشخص کردن بخش‌هایی که نیاز به مرور دارند.

این مثال یک الگوی مفهومی برای خودکارسازی است، نه تضمینی درباره‌ی وجود قابلیت آماده‌ی انجام همه‌ی این مراحل در .Antigravity برای پیاده‌سازی واقعی، باید ابزارها و امکانات موجود بررسی شوند.

اصول طراحی یک گردش‌کار خودکار مناسب

برای طراحی گردش‌کار قابل اعتماد، رعایت چند اصل ضروری است:

۱ .وظیفه را به مراحل مشخص تقسیم کنید.

هر مرحله باید هدف روشنی داشته باشد و نتیجه‌ی آن قابل بررسی باشد.

۲ .ورودی و خروجی هر مرحله را مشخص کنید.

برای مثال، مرحله‌ی بررسی کد باید فایل‌های مورد بررسی و نوع گزارش مورد انتظار را روشن کند.

۳ .نقاط تأیید انسانی تعیین کنید.

عملیات حساس، مانند حذف فایل‌ها، تغییر ساختار پایگاه داده، انتشار نسخه‌ی جدید یا ارسال اطلاعات به سرویس خارجی، بهتر است بدون بررسی و مجوز مناسب انجام نشوند.

۴ .برای خطاها برنامه داشته باشید.

اگر ابزار در دسترس نبود، آزمون شکست خورد یا داده‌ی لازم دریافت نشد، گردش‌کار باید این موضوع را گزارش کند و از ادامه‌ی نامطمئن عملیات خودداری کند.

۵ .نتیجه را ثبت و بررسی کنید.

گزارش مراحل انجام‌شده، فایل‌های تغییرکرده، خطاها و آزمون‌های اجراشده به کاربر کمک می‌کند تصمیم بگیرد آیا نتیجه قابل قبول است یا به اصلاح بیشتری نیاز دارد.

۶ .ابتدا در محیط آزمایشی اجرا کنید.

پیش از استفاده از یک گردش‌کار در پروژه‌ی واقعی، آن را با داده‌ها و فایل‌های غیرحساس آزمایش کنید. این کار به شناسایی مشکلات پیکربندی و رفتارهای پیش‌بینی‌نشده کمک می‌کند.

اتصال ابزارهای خارجی و استفاده از قابلیت‌های پیشرفته می‌تواند دامنه‌ی فعالیت ایجنت را گسترش دهد . MCPراهی استاندارد برای ارتباط برنامه‌های هوش مصنوعی با ابزارها و منابع فراهم می‌کند؛API ها امکان تعامل نرم‌افزارها با سرویس‌ها را می‌دهند؛ افزونه‌ها قابلیت‌های محیط توسعه را افزایش می‌دهند و گردش‌کارهای خودکار می‌توانند انجام وظایف تکراری را ساده‌تر کنند.

بااین‌حال، پیش از استفاده از هر اتصال یا ابزار باید سازگاری، اعتبار، مجوزها و پیامدهای احتمالی آن بررسی شود. همچنین، خودکارسازی نباید به معنای حذف کنترل انسانی باشد؛ به‌ویژه زمانی که عملیات می‌تواند فایل‌های مهم، داده‌های واقعی یا سرویس‌های خارجی را تحت تأثیر قرار دهد.

فصل هفتم: امنیت و کنترل دسترسی

ایجنت‌های برنامه‌نویسی می‌توانند فایل‌ها را بخوانند و تغییر دهند، دستورات ترمینال را اجرا کنند و در صورت وجود ابزارهای مناسب، با سرویس‌های خارجی ارتباط برقرار کنند. همین قابلیت‌ها باعث می‌شوند انجام بسیاری از وظایف سریع‌تر شود؛ اما در صورت استفاده‌ی نادرست، ممکن است به حذف اطلاعات، افشای داده‌های محرمانه، اجرای دستورات خطرناک یا ایجاد تغییرات ناخواسته در پروژه منجر شوند.

به همین دلیل، امنیت در محیط‌های ایجنت‌محور فقط به امنیت مدل هوش مصنوعی محدود نمی‌شود. نحوه‌ی اعطای مجوزها، محدود کردن دسترسی‌ها، بررسی دستورات و کنترل عملیات ایجنت نیز اهمیت زیادی دارد.

در این فصل، اصول اصلی امنیت هنگام کار با Google Antigravity را بررسی می‌کنیم تا بتوانیم از قابلیت‌های آن استفاده کنیم، بدون آنکه کنترل پروژه و اطلاعات خود را نادیده بگیریم.

مجوزهای دسترسی ایجنت

مجوز دسترسی مشخص می‌کند یک ابزار یا ایجنت اجازه دارد چه عملیاتی انجام دهد و به چه منابعی دسترسی داشته باشد. برای مثال، خواندن یک فایل متنی با حذف آن تفاوت دارد؛ همچنین، اجرای یک دستور ساده در ترمینال با اجرای دستوری که تنظیمات سیستم را تغییر می‌دهد، از نظر میزان خطر یکسان نیست.

ایجنت برای انجام برخی وظایف به دسترسی‌های مختلف نیاز دارد. اگر این دسترسی‌ها بیش از حد گسترده باشند، یک اشتباه در درک درخواست یا انتخاب ابزار می‌تواند پیامدهای جدی‌تری داشته باشد.

انواع دسترسی

برای درک بهتر مجوزها، می‌توان دسترسی‌ها را به چند دسته تقسیم کرد:

۱ .دسترسی خواندن(Read)

این دسترسی به ایجنت اجازه می‌دهد اطلاعات یک فایل یا منبع را مشاهده کند، بدون آنکه الزاماً امکان تغییر آن را داشته باشد.

برای مثال، ایجنت می‌تواند فایل تنظیمات پروژه را بخواند تا علت یک خطا را بررسی کند.

۲ .دسترسی نوشتن(Write)

این دسترسی امکان ایجاد یا تغییر فایل‌ها را فراهم می‌کند. برای مثال، ایجنت می‌تواند یک تابع جدید به فایل پایتون اضافه کند یا محتوای یک فایل مستندات را اصلاح کند.

دسترسی نوشتن باید با دقت مدیریت شود؛ زیرا حتی یک تغییر کوچک ممکن است بر عملکرد بخش‌های دیگر پروژه تأثیر بگذارد.

۳ .دسترسی اجرای دستورات(Execute)

این دسترسی امکان اجرای دستورات یا برنامه‌ها را فراهم می‌کند. برای مثال، ایجنت ممکن است برای اجرای آزمون‌ها، نصب وابستگی‌ها یا راه‌اندازی برنامه از ترمینال استفاده کند.

خطر این دسترسی به دستور اجراشده بستگی دارد. برخی دستورات فقط اطلاعات را نمایش می‌دهند، درحالی‌که برخی دیگر می‌توانند فایل‌ها را حذف کنند، برنامه‌ای ناشناس را اجرا کنند یا تغییرات گسترده‌ای در سیستم ایجاد کنند.

۴ .دسترسی به سرویس‌های خارجی

اگر ایجنت به ابزارهای اتصال یا سرویس‌های خارجی دسترسی داشته باشد، ممکن است بتواند اطلاعاتی را دریافت کند یا عملیاتی را در آن سرویس انجام دهد. برای مثال، یک ابزار ممکن است به پایگاه داده یا سامانه‌ی مدیریت پروژه متصل باشد.

در این حالت، باید مشخص شود ایجنت به کدام اطلاعات دسترسی دارد و چه عملیاتی را می‌تواند انجام دهد.

اصل حداقل دسترسی

یکی از مهم‌ترین اصول امنیتی، اصل حداقل دسترسی (Principle of Least Privilege) است. بر اساس این اصل، هر ابزار یا کاربر باید فقط به میزان دسترسی لازم برای انجام وظیفه‌ی خود دسترسی داشته باشد.

برای مثال، اگر هدف فقط بررسی اطلاعات یک پایگاه داده است، بهتر است از دسترسی خواندنی استفاده شود، نه دسترسی کامل برای تغییر یا حذف داده‌ها.

در کار با Antigravity نیز می‌توان این اصل را به شکل‌های زیر رعایت کرد:

-فقط پروژه و فایل‌های موردنیاز را در اختیار ایجنت قرار دهید.

-برای انجام وظایف ساده، دسترسی‌های گسترده و غیرضروری فعال نکنید.

-عملیات حساس را به تأیید کاربر وابسته نگه دارید.

-از حساب‌ها و کلیدهای دسترسی اختصاصی با مجوزهای محدود استفاده کنید.

-پس از پایان کار، دسترسی‌های موقت و غیرضروری را لغو کنید.

باید توجه داشت که محدود کردن دسترسی در متن درخواست با محدودیت فنی دسترسی یکسان نیست. برای مثال، نوشتن جمله‌ی «به فایل‌های دیگر دست نزن» راهنمای مفیدی است، اما به‌تنهایی مانع فنی ایجاد نمی‌کند. برای حفاظت واقعی، باید از مجوزهای محیط، تنظیمات ابزار و روش‌های مناسب کنترل دسترسی استفاده کرد.

تأیید عملیات حساس

در برخی محیط‌ها، اجرای بعضی عملیات به تأیید کاربر نیاز دارد. این سازوکار به کاربر فرصت می‌دهد پیش از اجرای دستور یا انجام تغییر، آن را بررسی کند.

هنگام مشاهده‌ی درخواست تأیید، فقط به نام دستور اکتفا نکنید. بررسی کنید:

-دستور دقیقاً چه کاری انجام می‌دهد؟

-آیا برای وظیفه‌ی فعلی ضروری است؟

-آیا فایل‌ها یا اطلاعات مهمی را تغییر می‌دهد؟

-آیا به اینترنت یا یک سرویس خارجی متصل می‌شود؟

-آیا راه کم‌خطرتر و محدودتری برای انجام کار وجود دارد؟

اگر دستور برای شما نامفهوم است، از ایجنت بخواهید آن را توضیح دهد. در صورت باقی ماندن ابهام، اجرای آن را متوقف کنید تا بتوانید درباره‌ی پیامدهایش تحقیق کنید.

Sandbox و محیط اجرای ایزوله

Sandboxچیست؟

Sandboxیا «محیط ایزوله» روشی برای اجرای برنامه‌ها و عملیات در محدوده‌ای کنترل‌شده است. هدف آن کاهش تأثیر احتمالی یک عملیات بر بخش‌های دیگر سیستم است.

برای مثال، فرض کنید می‌خواهید یک برنامه‌ی ناشناخته یا مجموعه‌ای از آزمون‌های آزمایشی را اجرا کنید. اگر اجرای آن در محیطی با دسترسی محدود امکان‌پذیر باشد، احتمال تأثیر مستقیم آن بر فایل‌های شخصی یا سایر بخش‌های سیستم کاهش پیدا می‌کند.

محیط ایزوله می‌تواند به شکل‌های مختلفی پیاده‌سازی شود؛ مانند محدود کردن دسترسی به فایل‌ها، کنترل ارتباط شبکه یا استفاده از محیط‌های جداگانه برای اجرای برنامه‌ها. میزان حفاظت به نوع پیاده‌سازی و تنظیمات آن بستگی دارد.

چرا محیط ایزوله اهمیت دارد؟

برنامه‌هایی که ایجنت اجرا می‌کند ممکن است حاوی خطا باشند یا رفتاری متفاوت از انتظار داشته باشند. برای مثال:

-یک اسکریپت ممکن است به‌اشتباه فایل‌های زیادی را حذف کند.

-یک برنامه‌ی آزمایشی ممکن است اطلاعات را در محل نامناسبی ذخیره کند.

-یک وابستگی ناشناس ممکن است رفتار ناخواسته‌ای داشته باشد.

-یک دستور ممکن است تنظیمات محیط توسعه را تغییر دهد.

محیط ایزوله می‌تواند دامنه‌ی پیامدهای چنین اتفاق‌هایی را محدود کند؛ البته فقط در محدوده‌ای که واقعاً محافظت می‌کند.

آیا Sandbox امنیت کامل ایجاد می‌کند؟

خیر. وجود Sandbox به معنای حذف همه‌ی خطرات نیست. میزان حفاظت به تنظیمات، مجوزهای اعطاشده، نحوه‌ی پیاده‌سازی و نوع عملیاتی که اجرا می‌شود بستگی دارد.

برای مثال، اگر یک محیط ایزوله اجازه‌ی دسترسی به پوشه‌ی حاوی اطلاعات محرمانه را داشته باشد، آن اطلاعات همچنان ممکن است در معرض خطر قرار بگیرند. همچنین، اگر محیط اجازه‌ی ارتباط شبکه داشته باشد، احتمال ارسال اطلاعات به خارج از سیستم باید در نظر گرفته شود.

بنابراین، Sandboxباید یکی از لایه‌های امنیتی باشد، نه تنها راهکار حفاظت.

استفاده‌ی ایمن از محیط اجرا

هنگام اجرای کد یا دستورات از طریقAntigravity ، این اصول را رعایت کنید:

-برای آزمایش کدهای ناشناس، از پروژه یا محیط آزمایشی استفاده کنید.

-دسترسی به پوشه‌های شخصی و فایل‌های غیرمرتبط را تا حد امکان محدود کنید.

-پیش از اجرای اسکریپت‌های ناشناس، محتوای آن‌ها و وابستگی‌هایشان را بررسی کنید.

-در صورت امکان، از داده‌های ساختگی به‌جای داده‌های واقعی استفاده کنید.

-پیش از عملیات گسترده، از فایل‌های مهم نسخه‌ی پشتیبان تهیه کنید.

-وضعیت دسترسی شبکه و فایل‌ها را در محیط اجرای مورد استفاده بررسی کنید.

- ✓ نکته درباره‌یAntigravity : نباید فرض کرد هر دستور یا هر ایجنتی به‌صورت پیش‌فرض در یک Sandbox با سطح حفاظت یکسان اجرا می‌شود. سازوکارهای واقعی ایزوله‌سازی، محدودیت‌ها و مجوزها را باید در مستندات و تنظیمات نسخه‌ی مورد استفاده بررسی کرد.

بررسی دستورات پیش از اجرا

ترمینال یکی از ابزارهای مهم در توسعه‌ی نرم‌افزار است. از طریق آن می‌توان برنامه را اجرا کرد، وابستگی‌ها را نصب کرد، آزمون‌ها را اجرا کرد و اطلاعات محیط را بررسی کرد. ایجنت نیز ممکن است برای انجام این کارها دستوراتی پیشنهاد دهد یا در صورت داشتن مجوز، آن‌ها را اجرا کند.

بااین‌حال، هر دستور ترمینال می‌تواند پیامدهای خاص خود را داشته باشد. بنابراین، پیش از اجرای دستورات تولیدشده توسط ایجنت باید محتوای آن‌ها را بررسی کرد.

دسته‌بندی دستورات بر اساس میزان خطر

برای تصمیم‌گیری بهتر، می‌توان دستورات را به‌صورت کلی در سه دسته قرار داد.

۱ .دستورات کم‌خطر

این دستورات معمولاً برای مشاهده‌ی اطلاعات یا بررسی وضعیت استفاده می‌شوند و تغییر مهمی در سیستم ایجاد نمی‌کنند.

برای مثال:

`pwd` -برای نمایش مسیر پوشه‌ی فعلی

`git status` -برای بررسی وضعیت تغییرات مخزنGit

`python --version` -برای نمایش نسخه‌ی پایتون

بااین‌حال، حتی درباره‌ی دستورات ظاهراً ساده نیز باید محیط اجرا و پیامدهای واقعی را در نظر گرفت.

۲ .دستورات نیازمند بررسی

برخی دستورات می‌توانند فایل‌ها را تغییر دهند، بسته‌های نرم‌افزاری نصب کنند یا برنامه‌ای را اجرا کنند. این دستورات لزوماً خطرناک نیستند، اما باید پیش از اجرا بررسی شوند.

برای مثال:

-نصب یک کتابخانه با `pip install`

-اجرای یک اسکریپت پایتون

-به‌روزرسانی وابستگی‌های پروژه

-اجرای آزمون‌هایی که ممکن است فایل یا پایگاه داده را تغییر دهند

پیش از اجرای این دستورات، منبع وابستگی‌ها، فایل‌های درگیر و اثرات احتمالی آن‌ها را بررسی کنید.

۳ .دستورات پرخطر

این دستورات ممکن است فایل‌های مهم را حذف کنند، مجوزها را تغییر دهند، اطلاعات را به بیرون ارسال کنند یا تنظیمات حساس سیستم را تحت تأثیر قرار دهند.

برای مثال:

-حذف بازگشت‌ناپذیر فایل‌ها یا پوشه‌های مهم

-اجرای اسکریپت ناشناس با دسترسی مدیریتی

-تغییر گسترده‌ی مجوزهای فایل‌ها

-اجرای دستوراتی که اطلاعات محرمانه را به سرویس خارجی ارسال می‌کنند

-اجرای دستور ترکیبی‌ای که بخش‌های مختلف آن به‌روشنی مشخص نیستند

در چنین مواردی، نباید صرفاً به دلیل پیشنهاد ایجنت دستور را اجرا کرد. ابتدا باید هدف، منبع و پیامدهای آن مشخص شوند.

روش بررسی یک دستور

فرض کنید ایجنت برای رفع خطای پروژه پیشنهاد می‌دهد یک بسته‌ی جدید نصب کنید. پیش از تأیید، این مراحل را دنبال کنید:

۱.هدف دستور را مشخص کنید. چرا نصب این بسته ضروری است؟

۲ .نام بسته را بررسی کنید. آیا همان بسته‌ای است که پروژه به آن نیاز دارد؟ آیا منبع آن معتبر است؟

۳ .تأثیر بر پروژه را بسنجید. آیا نصب بسته، وابستگی‌های دیگر را تغییر می‌دهد؟

۴. گزینه‌های دستور را بخوانید. آیا گزینه‌ای وجود دارد که باعث حذف، بازنویسی یا تغییر گسترده شود؟

۵ .نتیجه‌ی احتمالی را بررسی کنید. اگر عملیات ناموفق باشد، آیا امکان بازگشت به وضعیت قبلی وجود دارد؟

۶ .در صورت ابهام، توقف کنید. از ایجنت بخواهید دستور را بخش‌به‌بخش توضیح دهد یا راهکار محدودتری پیشنهاد کند.

مراقب اجرای مستقیم دستورات اینترنتی باشید

یکی از الگوهای پرخطر، دریافت یک اسکریپت از اینترنت و اجرای مستقیم آن بدون بررسی محتواست. اگر منبع قابل اعتماد نباشد یا محتوای اسکریپت تغییر کرده باشد، ممکن است کد ناخواسته اجرا شود.

در صورت نیاز به استفاده از چنین اسکریپت‌هایی:

-منبع و اعتبار آن را بررسی کنید.

-در صورت امکان، ابتدا فایل را دریافت و محتوای آن را مطالعه کنید.

-از اجرای اسکریپت با دسترسی مدیریتی غیرضروری خودداری کنید.

-آن را ابتدا در محیط آزمایشی اجرا کنید.

-اگر اسکریپت به اطلاعات محرمانه یا منابع سیستمی دسترسی دارد، ضرورت این دسترسی را بررسی کنید.

اصل مهم این است که هیچ دستوری نباید صرفاً به این دلیل اجرا شود که ایجنت آن را پیشنهاد داده است.

حفاظت از فایل‌های حساس و اطلاعات محرمانه

در پروژه‌های نرم‌افزاری، اطلاعاتی وجود دارند که نباید به‌صورت عمومی منتشر شوند یا بدون ضرورت در اختیار ابزارها و سرویس‌های خارجی قرار بگیرند. این اطلاعات ممکن است شامل رمزهای عبور، کلیدهایAPI ، توکن‌های دسترسی، اطلاعات شخصی کاربران یا داده‌های واقعی یک سامانه باشند.

افشای چنین اطلاعاتی می‌تواند به دسترسی غیرمجاز، سوءاستفاده از حساب‌ها یا نقض حریم خصوصی منجر شود.

چه اطلاعاتی حساس محسوب می‌شوند؟

نمونه‌هایی از اطلاعاتی که باید از آن‌ها محافظت کرد عبارت‌اند از:

-رمزهای عبور حساب‌ها و پایگاه‌های داده

-کلیدهای API و توکن‌های احراز هویت

-فایل‌های تنظیماتی که اطلاعات اتصال به سرویس‌ها را نگه می‌دارند

-کلیدهای خصوصی و گواهی‌های محرمانه

-اطلاعات شناسایی کاربران

-اطلاعات پزشکی و پرونده‌های سلامت

-نسخه‌های پشتیبان حاوی داده‌های واقعی

-اطلاعات داخلی سازمان‌ها یا پروژه‌هایی که اجازه‌ی انتشار آن‌ها وجود ندارد

حساس بودن اطلاعات فقط به نوع فایل وابسته نیست. برای مثال، یک فایل CSV معمولی ممکن است حاوی اطلاعات شخصی کاربران باشد و به همین دلیل نیاز به حفاظت داشته باشد.

فایل env. چیست؟

در بسیاری از پروژه‌ها، از فایل `.env` برای نگهداری متغیرهای محیطی استفاده می‌شود. این فایل ممکن است شامل اطلاعاتی مانند کلیدهایAPI ، تنظیمات اتصال به پایگاه داده یا مقادیر محرمانه‌ی موردنیاز برنامه باشد.

برای مثال، ممکن است یک پروژه به متغیری با نام `DATABASE_URL` نیاز داشته باشد. مقدار واقعی آن می‌تواند شامل اطلاعات اتصال به پایگاه داده باشد؛ بنابراین، نباید بدون بررسی در کد عمومی، گزارش‌ها یا مخازن قابل مشاهده منتشر شود.

برای حفاظت بهتر:

-فایل‌های حاوی اطلاعات محرمانه را به مخزن عمومی Git اضافه نکنید.

-از فایل `.gitignore` برای جلوگیری از ثبت ناخواسته‌ی فایل‌های حساس استفاده کنید.

-در صورت نیاز، یک فایل نمونه مانند `.env.example` بسازید که فقط نام متغیرها و مقادیر ساختگی را نشان دهد.

-اطلاعات واقعی را در محل مناسب و با دسترسی محدود نگهداری کنید.

-در صورت افشای یک کلید یا رمز، آن را بی‌درنگ از طریق سرویس مربوطه باطل یا تعویض کنید.

توجه کنید که افزودن فایل به `.gitignore` فقط از ثبت ناخواسته‌ی آن در مراحل بعدی جلوگیری می‌کند. اگر فایل قبلاً وارد تاریخچه‌ی Git شده باشد، صرف اضافه کردن آن به این فایل، اطلاعات را از تاریخچه‌ی قبلی حذف نمی‌کند.

چگونه از اطلاعات حساس در تعامل با ایجنت محافظت کنیم؟

هنگام کار باAntigravity ، بهتر است اطلاعات محرمانه فقط در صورتی در اختیار ابزار قرار گیرند که برای وظیفه ضروری باشند و روش استفاده از آن‌ها امن باشد.

چند اصل مهم:

۱ .اطلاعات واقعی را بی‌دلیل ارسال نکنید.

اگر هدف آزمایش یک تابع است، معمولاً می‌توان از داده‌های ساختگی استفاده کرد. نیازی نیست برای آزمایش، اطلاعات واقعی کاربران را وارد درخواست کنید.

۲ .دسترسی ایجنت به فایل‌ها را محدود کنید.

پوشه‌هایی را که حاوی اطلاعات شخصی یا محرمانه هستند، بدون ضرورت در محدوده‌ی کار قرار ندهید. اگر محیط امکان تعریف محدودیت دسترسی دارد، آن را متناسب با وظیفه تنظیم کنید.

۳ .محتوای گزارش‌ها را بررسی کنید.

گزارش خطا، خروجی ترمینال و فایل‌های تولیدشده ممکن است ناخواسته حاوی رمز یا اطلاعات حساس باشند. پیش از اشتراک‌گذاری آن‌ها، محتوایشان را بررسی کنید.

۴ .پیش از اتصال سرویس خارجی، جریان اطلاعات را بشناسید.

اگر از ابزار خارجی استفاده می‌کنید، بررسی کنید چه اطلاعاتی برای آن ارسال می‌شود، سرویس چه داده‌هایی را دریافت می‌کند و سیاست‌های نگهداری اطلاعات آن چیست.

۵ .در صورت افشای اطلاعات، فقط به حذف متن اکتفا نکنید.

اگر کلید دسترسی یا رمز واقعی افشا شده است، ممکن است لازم باشد آن را باطل یا تعویض کنید. حذف پیام یا فایل به‌تنهایی تضمین نمی‌کند که اطلاعات افشاشده دیگر قابل استفاده نباشند.

حمله‌ی تزریق دستور(Prompt Injection) : یکی از خطرات مهم در سیستم‌های ایجنت‌محور، تزریق دستور یا Prompt Injection است. در این نوع حمله، محتوایی که ایجنت می‌خواند، تلاش می‌کند او را به انجام کاری خارج از هدف اصلی هدایت کند.

برای مثال، ممکن است در یک فایل پروژه، صفحه‌ی وب یا سندی که ایجنت بررسی می‌کند، متنی وجود داشته باشد که از ایجنت بخواهد اطلاعات محرمانه را ارسال کند یا دستورالعمل‌های قبلی را نادیده بگیرد.

چنین متنی صرفاً به دلیل قرار گرفتن در یک فایل یا صفحه، نباید به‌عنوان دستور معتبر و مورد تأیید کاربر تلقی شود.

برای کاهش این خطر:

-منابع ناشناس را قابل اعتماد فرض نکنید.

-از ایجنت بخواهید محتوای خارجی را به‌عنوان داده‌ی مورد بررسی در نظر بگیرد، نه مجوزی برای انجام هر عملی.

-دسترسی به اطلاعات محرمانه و ابزارهای حساس را محدود کنید.

-پیش از ارسال اطلاعات یا انجام عملیات مهم، تأیید انسانی را حفظ کنید.

-در صورت مشاهده‌ی دستورهای نامرتبط یا مشکوک در فایل‌ها و صفحات، آن‌ها را بررسی کنید.

هیچ دستور متنی به‌تنهایی نمی‌تواند تمام حملات تزریق دستور را خنثی کند. محدود کردن دسترسی‌ها و کنترل ابزارها همچنان ضروری است.

خطرات استفاده‌ی خودکار از ایجنت

خودکارسازی می‌تواند انجام وظایف تکراری را آسان‌تر کند، اما وقتی یک ایجنت اجازه دارد بدون بررسی انسانی مراحل زیادی را اجرا کند، احتمال گسترش خطا نیز افزایش می‌یابد.

اگر ایجنت درخواست را اشتباه درک کند، ممکن است همان برداشت نادرست را در چندین مرحله دنبال کند. اگر یک دستور اشتباه اجرا شود، مراحل بعدی نیز ممکن است بر اساس نتیجه‌ی نادرست آن ادامه پیدا کنند.

بنابراین، میزان نظارت باید متناسب با اهمیت وظیفه و پیامدهای احتمالی آن باشد.

مهم‌ترین خطرات

۱ .تغییر یا حذف ناخواسته‌ی فایل‌ها

ایجنت ممکن است برای انجام یک تغییر، فایل‌هایی را اصلاح کند که خارج از محدوده‌ی درخواست هستند. در صورت داشتن مجوز حذف، امکان از دست رفتن اطلاعات نیز وجود دارد.

راهکار: پیش از تغییرات گسترده، وضعیت Git را بررسی کنید، از اطلاعات مهم نسخه‌ی پشتیبان تهیه کنید و دامنه‌ی تغییرات مجاز را روشن سازید.

۲ .اجرای دستورات نامناسب

یک دستور اشتباه می‌تواند باعث نصب بسته‌ی نامعتبر، تغییر تنظیمات یا حذف اطلاعات شود.

راهکار: دستورات حساس را پیش از اجرا بررسی کنید و از اعطای دسترسی مدیریتی غیرضروری خودداری کنید.

۳ .ارسال اطلاعات به سرویس خارجی

اگر ابزار متصل‌شده اجازه‌ی ارسال داده داشته باشد، ممکن است اطلاعات پروژه به خارج از محیط محلی منتقل شوند.

راهکار: مقصد ارسال داده، نوع اطلاعات و مجوز ابزار را بررسی کنید. اطلاعات محرمانه را بدون ضرورت و مجوز مناسب ارسال نکنید.

۴ .تکرار خودکار یک خطا

اگر گردش‌کار بدون بررسی نتیجه، یک مرحله را بارها تکرار کند، ممکن است مشکل تشدید شود یا منابع زیادی مصرف شوند.

راهکار: برای تکرار عملیات محدودیت تعریف کنید و در صورت شکست مکرر، فرایند را متوقف کرده و علت را بررسی کنید.

۵ .تغییرات گسترده بر اساس یک فرض اشتباه

گاهی ایجنت مسئله را به‌درستی درک نمی‌کند، اما به‌جای محدود کردن تغییرات، بخش‌های زیادی از پروژه را اصلاح می‌کند.

راهکار: از ایجنت بخواهید ابتدا برنامه‌ی تغییرات را ارائه دهد و پیش از اجرای عملیات گسترده، آن را بررسی کنید.

۶ .اعتماد بیش از حد به گزارش موفقیت

ممکن است ایجنت اعلام کند وظیفه انجام شده است، اما برنامه واقعاً اجرا نشده باشد یا آزمون کافی انجام نشده باشد.

راهکار: خروجی واقعی ترمینال، نتایج آزمون‌ها و فایل‌های تغییرکرده را بررسی کنید. میان «کد تولید شد»، «دستور اجرا شد» و «عملکرد صحیح تأیید شد» تفاوت قائل شوید.

کدام وظایف را نباید بدون نظارت سپرد؟

در حالت کلی، بهتر است وظایف زیر بدون کنترل مناسب انسانی انجام نشوند:

-حذف گسترده یا بازنویسی فایل‌های مهم

-تغییر ساختار پایگاه داده‌ی واقعی

-انتشار مستقیم برنامه در محیط عملیاتی

-ارسال پیام یا ایمیل به افراد واقعی

-انتقال اطلاعات حساس به سرویس‌های خارجی

-تغییر مجوزهای امنیتی یا تنظیمات مهم سیستم

-انجام عملیات مالی یا هر اقدامی با پیامدهای جدی و دشوار برای بازگشت

این موضوع به معنای ممنوع بودن استفاده از ایجنت در چنین کارهایی نیست؛ بلکه به این معناست که باید از مجوزهای محدود، محیط آزمایشی، بررسی نتایج و تأیید انسانی متناسب با خطر استفاده شود.

یک الگوی امن برای انجام وظایف

برای کاهش خطرات، می‌توانید هنگام سپردن وظایف به ایجنت از این الگو استفاده کنید:

۱ .بررسی: ابتدا از ایجنت بخواهید فایل‌ها و شرایط مرتبط را بررسی کند.

۲ .برنامه‌ریزی: از او بخواهید مراحل پیشنهادی و فایل‌هایی را که احتمالاً تغییر خواهند کرد مشخص کند.

۳ .تأیید: برنامه را بررسی کنید و در صورت نیاز، محدوده‌ی کار را اصلاح کنید.

۴ .اجرا: اجازه دهید فقط عملیات لازم و مجاز انجام شوند.

۵ .آزمایش: نتیجه را با اجرای برنامه یا آزمون‌های مرتبط بررسی کنید.

۶ .بازبینی: تغییرات نهایی، گزارش اجرا و پیامدهای احتمالی را بررسی کنید.

۷ .ثبت یا بازگردانی:اگر نتیجه مناسب بود، تغییرات را با Git ثبت کنید؛ در غیر این صورت، اصلاحات لازم را انجام دهید یا به وضعیت قبلی بازگردید.

این الگو به‌ویژه برای پروژه‌های آموزشی و کاری مفید است؛ زیرا هم از قابلیت‌های ایجنت استفاده می‌کند و هم مسئولیت تصمیم‌گیری نهایی را حفظ می‌کند.

امنیت هنگام کار با Google Antigravity به شناخت مجوزها، محدود کردن دسترسی‌ها، بررسی دستورات، حفاظت از اطلاعات محرمانه و نظارت بر عملیات خودکار وابسته است.

برای استفاده‌ی ایمن، اصل حداقل دسترسی را رعایت کنید، عملیات حساس را پیش از اجرا بررسی کنید، برای آزمایش از محیط‌های مناسب استفاده کنید و اطلاعات محرمانه را بی‌دلیل در اختیار ایجنت یا سرویس‌های خارجی قرار ندهید. همچنین، به خاطر داشته باشید کهSandbox ، فایل‌های دستورالعمل و تأییدهای کاربر هرکدام بخشی از فرایند حفاظت هستند و هیچ‌کدام به‌تنهایی امنیت کامل را تضمین نمی‌کنند.

مهم‌ترین اصل این فصل آن است که"توانایی ایجنت برای انجام یک عملیات، به معنای مناسب بودن آن عملیات یا بی‌خطر بودن اجرای آن نیست." کاربر باید بداند چه دسترسی‌هایی در اختیار ایجنت قرار گرفته‌اند، چه تغییراتی انجام می‌شوند و چگونه می‌توان پیامدهای ناخواسته را کنترل کرد.

فصل هشتم: خطاها، محدودیت‌ها و اصول استفاده‌ی صحیح

با وجود قابلیت‌های گسترده‌یGoogle Antigravity ، استفاده از ایجنت همیشه به نتیجه‌ی درست و بدون خطا منجر نمی‌شود. ایجنت ممکن است درخواست کاربر را اشتباه درک کند، راه‌حل نامناسبی انتخاب کند، کدی تولید کند که اجرا نمی‌شود یا تغییراتی ایجاد کند که با ساختار پروژه سازگار نیستند.

شناخت این محدودیت‌ها به کاربر کمک می‌کند به‌جای اعتماد کامل به خروجی ایجنت، عملکرد آن را ارزیابی کند و در صورت بروز مشکل، مسیر مناسبی برای اصلاح آن در پیش بگیرد.

هدف این فصل، معرفی خطاهای رایج هنگام کار با Antigravity و ارائه‌ی روش‌هایی کاربردی برای شناسایی، رفع و پیشگیری از آن‌هاست.

خطا در درک درخواست و برنامه‌ریزی

یکی از نخستین مشکلاتی که ممکن است هنگام کار با ایجنت رخ دهد، برداشت نادرست از درخواست کاربر است. ایجنت ممکن است از نظر زبانی درخواست را قابل فهم تشخیص دهد، اما هدف واقعی کاربر، محدودیت‌های پروژه یا نتیجه‌ی مورد انتظار را به‌درستی درک نکند.

برای مثال، اگر از ایجنت بخواهید «ساختار پروژه را بهتر کن»، مشخص نیست منظور شما مرتب‌سازی فایل‌هاست، اصلاح معماری برنامه است یا بهبود کیفیت کد. در چنین شرایطی، ایجنت ممکن است تغییری انجام دهد که از نظر خودش منطقی است، اما با خواسته‌ی شما مطابقت ندارد.

دلایل رایج برداشت اشتباه

-درخواست مبهم: هدف یا نتیجه‌ی مورد انتظار به‌صورت دقیق مشخص نشده است.

-کمبود اطلاعات: ایجنت به فایل‌ها، پیام‌های خطا یا توضیحات ضروری دسترسی ندارد.

-وجود چند هدف در یک درخواست: وظایف متعدد با اولویت‌های متفاوت در یک پیام ترکیب شده‌اند.

-فرض‌های نادرست: ایجنت درباره‌ی ساختار پروژه یا ابزارهای مورد استفاده، اطلاعاتی را فرض می‌کند که ممکن است صحیح نباشند.

-تغییر شرایط در طول کار: اطلاعات یا تصمیم‌های جدید با برنامه‌ی اولیه تفاوت پیدا می‌کنند.

چگونه این خطا را کاهش دهیم؟

پیش از ارسال درخواست، هدف، محدوده‌ی کار و نتیجه‌ی مورد انتظار را مشخص کنید. اگر وظیفه پیچیده است، از ایجنت بخواهید ابتدا برنامه‌ی پیشنهادی خود را ارائه دهد و پیش از اعمال تغییرات گسترده، آن را بررسی کنید.

برای مثال، به‌جای درخواست «برنامه را اصلاح کن»، می‌توانید بنویسید:

"فایل `data_loader.py` را بررسی کن و علت خطای خواندن فایل CSV را پیدا کن. ابتدا علت احتمالی و راه‌حل پیشنهادی را توضیح بده. فعلاً هیچ فایلی را تغییر نده. پس از تأیید من، فقط تغییرات ضروری را اعمال کن و نتیجه را آزمایش کن"

این درخواست محدوده‌ی کار، ترتیب مراحل و میزان اختیار ایجنت را روشن می‌کند.

خطا در برنامه‌ریزی وظایف

گاهی ایجنت هدف را درست متوجه می‌شود، اما مراحل نامناسبی برای رسیدن به آن انتخاب می‌کند. برای مثال، ممکن است برای رفع یک خطای ساده، چندین فایل را تغییر دهد یا پیش از شناخت ساختار پروژه، وابستگی جدیدی نصب کند.

برای جلوگیری از چنین وضعیتی:

-از ایجنت بخواهید پیش از تغییرات گسترده، مراحل کار را مشخص کند.

-وظایف پیچیده را به بخش‌های کوچک‌تر تقسیم کنید.

-از او بخواهید پیش از افزودن کتابخانه‌ی جدید، ضرورت آن را توضیح دهد.

-پس از هر مرحله‌ی مهم، نتیجه را بررسی کنید.

-اگر برنامه از هدف اصلی منحرف شد، درخواست را روشن‌تر کنید و محدوده‌ی کار را دوباره مشخص سازید.

برنامه‌ریزی ایجنت می‌تواند مفید باشد، اما نباید آن را بدون بررسی، بهترین یا تنها مسیر ممکن در نظر گرفت.

خطاهای کدنویسی، اجرا و وابستگی‌ها

یکی از محدودیت‌های مهم ایجنت‌های برنامه‌نویسی، احتمال تولید کد نادرست یا ناسازگار با محیط پروژه است. کدی که از نظر ظاهری صحیح به نظر می‌رسد، ممکن است هنگام اجرا خطا ایجاد کند یا در شرایط خاص عملکرد نادرستی داشته باشد.

به همین دلیل، تولید کد تنها بخشی از فرایند توسعه است و باید با اجرا، آزمایش و بررسی نتیجه همراه شود.

انواع خطاهای کدنویسی

۱ .خطاهای نحوی(Syntax Errors)

این خطاها زمانی رخ می‌دهند که کد از قواعد زبان برنامه‌نویسی پیروی نکند. برای مثال، ممکن است پرانتزی بسته نشده باشد، دونقطه‌ای در محل لازم قرار نگرفته باشد یا ساختار یک دستور به‌درستی نوشته نشده باشد.

در پایتون، چنین خطاهایی اغلب هنگام تفسیر کد شناسایی می‌شوند.

۲ .خطاهای زمان اجرا(Runtime Errors)

در این حالت، کد ممکن است از نظر نحوی صحیح باشد، اما هنگام اجرا با مشکل مواجه شود. برای مثال، برنامه ممکن است بخواهد فایلی را باز کند که وجود ندارد یا عملیاتی نامعتبر روی یک مقدار انجام دهد.

۳ .خطاهای منطقی(Logical Errors)

این خطاها از دشوارترین انواع خطا هستند؛ زیرا برنامه ممکن است بدون نمایش خطا اجرا شود، اما نتیجه‌ی نادرستی تولید کند.

برای مثال، تابعی که باید میانگین چند عدد را محاسبه کند ممکن است به دلیل اشتباه در فرمول، عدد نادرستی برگرداند.

۴ .خطاهای مربوط به وابستگی‌ها(Dependency Errors)

برخی برنامه‌ها برای اجرا به کتابخانه‌ها یا بسته‌های نرم‌افزاری خاصی نیاز دارند. اگر یک بسته نصب نشده باشد، نسخه‌ی آن ناسازگار باشد یا محیط اجرای پروژه به‌درستی تنظیم نشده باشد، برنامه ممکن است اجرا نشود.

۵ .خطاهای سازگاری

گاهی کد با نسخه‌ی پایتون، چارچوب نرم‌افزاری یا سیستم‌عامل مورد استفاده سازگار نیست. برای مثال، ممکن است ایجنت از تابعی استفاده کند که در نسخه‌ی نصب‌شده‌ی یک کتابخانه وجود ندارد.

روش مناسب برای رفع خطا

هنگام بروز خطا، بهتر است به‌جای درخواست کلی «همه‌چیز را درست کن»، فرایند رفع خطا را مرحله‌به‌مرحله پیش ببرید.

۱ .پیام کامل خطا را دریافت کنید.

۲ .مشخص کنید خطا در کدام فایل و کدام بخش رخ داده است.

۳ .شرایطی را که باعث بروز خطا می‌شوند بررسی کنید.

۴ .از ایجنت بخواهید علت احتمالی را توضیح دهد و راه‌حل محدود و مشخصی پیشنهاد کند.

۵ .پس از اعمال اصلاحات، همان شرایط را دوباره آزمایش کنید.

۶ .بررسی کنید که اصلاح جدید باعث بروز خطا در بخش دیگری از پروژه نشده باشد.

- مثال عملی

فرض کنید برنامه‌ی پایتون با خطای زیر مواجه شده است:

```
'ModuleNotFoundError: No module named 'pandas
```

این پیام نشان می‌دهد که محیط اجرایی فعلی نمی‌تواند ماژول `pandas` را پیدا کند. بااین‌حال، علت دقیق می‌تواند به نصب نبودن بسته، انتخاب محیط پایتون اشتباه یا تنظیمات نادرست محیط مجازی مربوط باشد.

در چنین شرایطی، نباید بلافاصله هر دستور پیشنهادی برای نصب بسته را اجرا کرد. ابتدا بررسی کنید برنامه از کدام مفسر پایتون استفاده می‌کند و آیا محیط مجازی پروژه فعال است یا خیر.

می‌توانید از ایجنت بخواهید:

"خطای `ModuleNotFoundError` مربوط به pandas را بررسی کن. ابتدا مفسر پایتون و محیط اجرای پروژه را شناسایی کن. مشخص کن آیا بسته نصب نشده یا برنامه از محیط اشتباهی استفاده می‌کند. پیش از نصب یا تغییر وابستگی‌ها، علت را توضیح بده"

این روش کمک می‌کند مشکل اصلی شناسایی شود و از تغییرات غیرضروری جلوگیری شود.

آیا اجرای موفق برنامه کافی است؟

خیر. اجرای موفق فقط نشان می‌دهد برنامه در شرایط آزمایش‌شده بدون خطای آشکار اجرا شده است. این موضوع صحت همه‌ی نتایج را تضمین نمی‌کند.برای مثال، یک تابع ممکن است برای ورودی‌های معمولی نتیجه‌ی صحیح تولید کند، اما در برابر ورودی خالی، مقادیر منفی یا داده‌های غیرمنتظره شکست بخورد.

بنابراین، پس از رفع خطا باید رفتار برنامه با ورودی‌های مختلف بررسی شود و در صورت امکان، آزمون‌های مناسب برای آن اجرا شوند.

تغییرات ناخواسته و مشکلات محیط پروژه

گاهی ایجنت علاوه بر انجام وظیفه‌ی اصلی، تغییراتی در فایل‌ها، تنظیمات یا وابستگی‌های پروژه ایجاد می‌کند که کاربر درخواست نکرده است. این تغییرات ممکن است به‌دلیل برداشت اشتباه از هدف، تلاش برای اصلاح مشکلات مرتبط یا انتخاب راه‌حل گسترده‌تر از نیاز واقعی رخ دهند.

برای مثال، ممکن است از ایجنت بخواهید یک تابع را اصلاح کند، اما او همزمان نام چند فایل را تغییر دهد، ساختار پوشه‌ها را جابه‌جا کند یا نسخه‌ی کتابخانه‌های پروژه را به‌روزرسانی کند.

چرا تغییرات ناخواسته مشکل‌ساز هستند؟

-ممکن است قابلیت‌هایی که قبلاً درست کار می‌کردند، دچار اختلال شوند.

-بررسی و فهم تغییرات دشوارتر می‌شود.

-احتمال ناسازگاری میان فایل‌ها افزایش می‌یابد.

-بازگرداندن پروژه به وضعیت قبلی زمان بیشتری می‌برد.

-ممکن است تغییرات جدید با کار سایر اعضای تیم تداخل پیدا کنند.

به همین دلیل، بهتر است تغییرات ایجنت تا حد امکان محدود، مرتبط و قابل بررسی باشند.

بررسی تغییرات باGit

Gitیکی از ابزارهای مهم برای مدیریت نسخه‌های پروژه است. با کمک آن می‌توان تغییرات فایل‌ها را مشاهده کرد و در صورت نیاز، به نسخه‌ی قبلی بازگشت.

اگر پروژه‌ی شما تحت مدیریت Git قرار دارد، می‌توانید پیش از شروع کار، وضعیت آن را بررسی کنید:

```
git status
```

این دستور وضعیت فایل‌های تغییرکرده، فایل‌های جدید و برخی تغییرات ثبت‌نشده را نمایش می‌دهد.

پس از انجام وظیفه توسط ایجنت، دوباره دستور را اجرا کنید و سپس تغییرات را بررسی کنید:

```
git diff
```

این دستور تغییرات ثبت‌نشده در فایل‌های تحت ردیابی Git را نمایش می‌دهد. برای مشاهده‌ی جزئیات بیشتر، می‌توانید فایل‌ها را جداگانه بررسی کنید یا از امکانات مقایسه‌ی نسخه‌ها در ویرایشگر استفاده کنید.

توجه کنید که `git diff` همه‌ی انواع تغییرات را نشان نمی‌دهد؛ برای مثال، فایل‌های جدیدی که هنوز تحت ردیابی Git نیستند ممکن است محتوایشان در خروجی معمول این دستور دیده نشود. وضعیت فایل‌ها را نیز بررسی کنید.

چگونه از تغییرات ناخواسته جلوگیری کنیم؟

-پیش از شروع، وضعیت فعلی پروژه را بررسی کنید.

-از ایجنت بخواهید فقط فایل‌های مرتبط با درخواست را تغییر دهد.

-پیش از تأیید تغییرات گسترده، فهرست فایل‌های تغییرکرده را بررسی کنید.

-از بازنویسی فایل‌های مهم بدون نسخه‌ی پشتیبان خودداری کنید.

-اگر پروژه متعلق به یک تیم است، تغییرات دیگر اعضا را پیش از بازگردانی یا بازنویسی فایل‌ها در نظر بگیرید.

-در پایان، آزمون‌های مرتبط را اجرا کنید تا اثر تغییرات مشخص شود.

بازگردانی تغییرات باGit

اگر تغییری نامناسب ایجاد شده باشد، Gitمی‌تواند به بازگرداندن وضعیت پروژه کمک کند. بااین‌حال، دستورات بازگردانی باید با دقت اجرا شوند؛ زیرا بعضی از آن‌ها ممکن است تغییرات ثبت‌نشده را از بین ببرند.

برای نمونه، دستور زیر تغییرات ثبت‌نشده‌ی یک فایل تحت ردیابی را به وضعیت نسخه‌ی فعلی ثبت‌شده بازمی‌گرداند:

```
git restore path/to/file
```

به‌جای `path/to/file` باید مسیر واقعی فایل قرار گیرد. این دستور می‌تواند تغییرات محلی آن فایل را حذف کند؛ بنابراین، پیش از اجرا، تغییرات را بررسی و در صورت نیاز ذخیره کنید.

اگر نمی‌دانید کدام تغییرات باید حفظ شوند، ابتدا از آن‌ها نسخه‌ی جداگانه تهیه کنید یا از قابلیت‌های مقایسه‌ی نسخه‌ها استفاده کنید. از اجرای دستورات بازگردانی گسترده بدون بررسی وضعیت پروژه خودداری کنید.

مدیریت زمینه و رفع خطاهای تکرارشونده

در برخی موارد، ایجنت یک خطا را اصلاح می‌کند، اما همان خطا دوباره رخ می‌دهد. ممکن است اصلاح اولیه کامل نبوده باشد، علت اصلی شناسایی نشده باشد یا تغییرات جدید مشکل دیگری ایجاد کرده باشند.

گاهی نیز در گفت‌وگوهای طولانی، اطلاعات مهم درباره‌ی ساختار پروژه یا اصلاحات قبلی به‌درستی در نظر گرفته نمی‌شوند. در نتیجه، ایجنت ممکن است راه‌حلی را تکرار کند که قبلاً ناموفق بوده است.

چرا خطاها تکرار می‌شوند؟

-علت اصلی خطا شناسایی نشده است.

-اصلاح انجام‌شده فقط نشانه‌ی مشکل را برطرف کرده است.

-نتیجه‌ی آزمون قبلی به‌درستی بررسی نشده است.

-ایجنت اطلاعات کافی درباره‌ی تغییرات قبلی ندارد.

-محیط اجرا یا نسخه‌ی وابستگی‌ها با شرایط مورد انتظار متفاوت است.

-چند خطای مرتبط وجود دارند و اصلاح یک مورد، مشکل دیگر را آشکار می‌کند.

روش جلوگیری از تکرار خطا

۱ .نتایج تلاش‌های قبلی را ثبت کنید.

مشخص کنید چه تغییری انجام شده، چه نتیجه‌ای داشته و چرا راه‌حل قبلی کافی نبوده است.

۲ .پیام خطای جدید را دوباره بررسی کنید

فرض نکنید خطای جدید همان خطای قبلی است. ممکن است پیام یا محل خطا تغییر کرده باشد.

۳ .از ایجنت بخواهید علت ریشه‌ای را بررسی کند.

به‌جای درخواست اصلاح مکرر، از او بخواهید شواهد موجود را بررسی کند و توضیح دهد چرا راه‌حل‌های قبلی موفق نبوده‌اند.

۴ .تغییرات را محدود نگه دارید.

اگر ایجنت در هر تلاش چند فایل را تغییر دهد، تشخیص عامل اصلی مشکل دشوارتر می‌شود. بهتر است هر بار یک فرضیه‌ی مشخص بررسی شود.

۵ .محیط اجرا را ثابت نگه دارید.

در صورت امکان، از همان محیط و شرایطی استفاده کنید که خطا در آن رخ داده است. تغییر هم‌زمان کد، نسخه‌ی پایتون و وابستگی‌ها می‌تواند تشخیص علت را دشوار کند.

نمونه‌ی درخواست برای رفع خطای تکرارشونده

فرض کنید ایجنت دو بار تلاش کرده خطایی را برطرف کند، اما مشکل همچنان وجود دارد. می‌توانید بنویسید:

"اصلاحات قبلی مشکل را برطرف نکرده‌اند. فعلاً تغییر جدیدی اعمال نکن. پیام خطای فعلی، تغییرات انجام‌شده و نتیجه‌ی آزمون‌های قبلی را بررسی کن. توضیح بده کدام فرضیه‌های قبلی رد شده‌اند، علت‌های احتمالی باقی‌مانده چیست و برای تشخیص علت اصلی چه آزمایشی لازم است. پس از مشخص شدن علت، یک راه‌حل محدود پیشنهاد بده"

این درخواست ایجنت را به بررسی شواهد و پرهیز از تکرار بی‌هدف اصلاحات هدایت می‌کند.

چه زمانی باید زمینه‌ی کار را دوباره تنظیم کرد؟

اگر گفت‌وگو طولانی شده، چند موضوع مختلف در آن مطرح شده یا ایجنت بارها اطلاعات مهم را نادیده گرفته است، خلاصه‌ای از وضعیت فعلی ارائه کنید.

این خلاصه بهتر است شامل موارد زیر باشد:

-هدف اصلی پروژه

-فایل‌ها و بخش‌های مرتبط

-خطای فعلی و شرایط ایجاد آن

-تغییرات انجام‌شده

-نتیجه‌ی آزمون‌های قبلی

-راه‌حل‌هایی که امتحان شده‌اند و نتیجه نداده‌اند

-محدودیت‌هایی که باید رعایت شوند

اگر لازم است، می‌توانید برای یک مسئله‌ی مستقل، وظیفه یا گفت‌وگوی تازه‌ای شروع کنید و اطلاعات ضروری را دوباره ارائه دهید. هدف از این کار، حفظ تمرکز و جلوگیری از اتکا به اطلاعات نامرتبط است.

بررسی خروجی و بازگردانی تغییرات

یکی از اصول اساسی کار با ایجنت این است که هیچ خروجی مهمی را صرفاً به دلیل تولید شدن توسط هوش مصنوعی، صحیح فرض نکنیم. بررسی خروجی باید متناسب با نوع وظیفه انجام شود؛ چه خروجی یک قطعه کد باشد، چه یک فایل تنظیمات یا مجموعه‌ای از تغییرات در پروژه.

چه مواردی باید بررسی شوند؟

۱ .تطابق با درخواست

آیا نتیجه دقیقاً همان هدفی را دنبال می‌کند که مشخص کرده‌اید؟ آیا قابلیت اضافی یا تغییری خارج از محدوده‌ی درخواست ایجاد شده است؟

۲ .صحت فنی

آیا کد از نظر نحوی و منطقی درست است؟ آیا از توابع، کتابخانه‌ها و روش‌های سازگار با محیط پروژه استفاده می‌کند؟

۳ .نتیجه‌ی اجرا

آیا برنامه در شرایط مورد انتظار اجرا می‌شود؟ آیا پیام خطا، هشدار یا رفتار غیرمنتظره‌ای مشاهده می‌شود؟

۴ .نتیجه‌ی آزمون‌ها

آیا آزمون‌های مرتبط با موفقیت اجرا شده‌اند؟ اگر آزمونی اجرا نشده، علت آن مشخص است؟ آیا آزمون‌ها برای ارزیابی رفتار مورد انتظار کافی هستند؟

۵ .دامنه‌ی تغییرات

آیا فقط فایل‌های مرتبط تغییر کرده‌اند؟ آیا فایل جدید، وابستگی یا تنظیمات غیرضروری اضافه شده است؟

۶ .امنیت

آیا اطلاعات محرمانه در فایل‌ها یا گزارش‌ها قرار گرفته‌اند؟ آیا دسترسی غیرضروری یا دستور خطرناکی ایجاد شده است؟

تفاوت تولید کد، اجرای موفق و صحت عملکرد

این سه مورد را باید از یکدیگر جدا کرد:

-تولید کد: ایجنت یک راه‌حل یا تغییر پیشنهادی ایجاد کرده است.

-اجرای موفق: کد یا دستور در شرایط مشخص بدون خطای آشکار اجرا شده است.

-صحت عملکرد: شواهد و آزمون‌های کافی نشان می‌دهند نتیجه با نیاز واقعی سازگار است.

برای مثال، ممکن است یک برنامه بدون خطا اجرا شود، اما داده‌ها را اشتباه محاسبه کند. بنابراین، اجرای موفق به‌تنهایی برای تأیید صحت کافی نیست.

بازگردانی تغییرات چه زمانی لازم است؟

اگر تغییرات باعث ایجاد خطای جدید شده‌اند، از هدف اصلی فاصله گرفته‌اند یا نتیجه‌ی قابل قبولی ندارند، ممکن است لازم باشد آن‌ها را اصلاح یا بازگردانی کنید.

پیش از بازگردانی:

۱ .تغییرات ایجادشده را بررسی کنید.

۲ .مشخص کنید کدام تغییرات نامناسب هستند و کدام بخش‌ها باید حفظ شوند.

۳ .وضعیت Git و تغییرات ثبت‌نشده را بررسی کنید.

۴ .در صورت نیاز، از فایل‌های مهم نسخه‌ی پشتیبان تهیه کنید.

۵ .از روش مناسب برای بازگردانی استفاده کنید و پس از آن وضعیت پروژه را دوباره آزمایش کنید.

اگر تغییرات قبلی مربوط به کار شما یا اعضای دیگر تیم باشند، بازگردانی گسترده ممکن است اطلاعات ارزشمندی را از بین ببرد. بنابراین، هیچ‌گاه بدون بررسی وضعیت فعلی پروژه، همه‌ی تغییرات را یک‌جا حذف نکنید.

نقش Git در کنترل کیفیت

Gitفقط ابزاری برای ثبت نسخه‌های پروژه نیست؛ بلکه در کنترل کیفیت کار ایجنت نیز کاربرد دارد. با ثبت تغییرات در مراحل مشخص، می‌توان روند اصلاحات را بهتر پیگیری کرد و در صورت بروز مشکل، تصمیم دقیق‌تری درباره‌ی بازگردانی گرفت.

یک گردش‌کار ساده می‌تواند چنین باشد:

۱ .وضعیت اولیه‌ی پروژه را بررسی کنید.

۲ .یک نقطه‌ی ثبت‌شده و قابل بازگشت داشته باشید.

۳ .وظیفه‌ی مشخصی را به ایجنت بسپارید.

۴ .تغییرات را با ابزارهای مقایسه بررسی کنید.

۵ .آزمون‌های مناسب را اجرا کنید.

۶ .اگر نتیجه صحیح بود، تغییرات تأییدشده را ثبت کنید.

۷ .اگر نتیجه مناسب نبود، علت را بررسی کنید و فقط تغییرات لازم را اصلاح یا بازگردانی کنید.

ثبت تغییرات باید پس از بررسی انجام شود، نه صرفاً به این دلیل که ایجنت اعلام کرده وظیفه به پایان رسیده است.

اصول طلایی کار باAntigravity

برای استفاده‌ی مؤثر ازGoogle Antigravity ، آشنایی با قابلیت‌ها کافی نیست؛ بلکه باید روش درستی برای تعریف وظایف، کنترل دسترسی‌ها و بررسی نتایج داشته باشید. اصول زیر می‌توانند به کاهش خطاها و افزایش کیفیت کار کمک کنند.

اصل اول: درخواست روشن بنویسید.

هدف، محدوده‌ی تغییرات و نتیجه‌ی مورد انتظار را مشخص کنید. درخواست مبهم، احتمال برداشت نادرست را افزایش می‌دهد.

اصل دوم: پیش از اجرا، مسئله را بررسی کنید.

اگر با خطا مواجه شدید، ابتدا علت را پیدا کنید. تغییر دادن چند بخش پروژه بدون شناخت مشکل ممکن است وضعیت را پیچیده‌تر کند.

اصل سوم: وظایف بزرگ را به مراحل کوچک تقسیم کنید

وظایف کوچک‌تر آسان‌تر بررسی و آزمایش می‌شوند. پس از تأیید هر مرحله، به مرحله‌ی بعد بروید.

اصل چهارم: به ایجنت اطلاعات کافی بدهید.

فایل‌های مرتبط، پیام‌های خطا، ساختار پروژه و محدودیت‌های مهم را در اختیار ایجنت قرار دهید؛ اما از ارائه‌ی اطلاعات نامرتبط یا محرمانه خودداری کنید.

اصل پنجم: تغییرات را کنترل کنید.

پس از انجام هر وظیفه، فایل‌های تغییرکرده را بررسی کنید. اجازه ندهید اصلاحات نامرتبط بدون بررسی وارد پروژه شوند.

اصل ششم: خروجی را آزمایش کنید.

کد تولیدشده را اجرا کنید و آزمون‌های مرتبط را انجام دهید. اگر امکان آزمایش وجود ندارد، این محدودیت را مشخص کنید و خروجی را تأییدشده تلقی نکنید.

اصل هفتم: دسترسی‌ها را محدود نگه دارید.

فقط مجوزهای لازم را در اختیار ایجنت قرار دهید. دسترسی به فایل‌های حساس، ابزارهای خارجی و عملیات مهم باید با دقت مدیریت شود.

اصل هشتم: پیش از اجرای دستور، پیامد آن را بفهمید.

اگر دستور ترمینال برای شما نامفهوم است، آن را بدون بررسی اجرا نکنید. از ایجنت بخواهید هدف و اثرات احتمالی دستور را توضیح دهد.

اصل نهم: از Git و نسخه‌ی پشتیبان استفاده کنید.

پیش از تغییرات مهم، وضعیت پروژه را بررسی کنید و برای اطلاعات ضروری امکان بازگشت فراهم سازید. بازگردانی نیز باید با دقت انجام شود تا تغییرات مفید از بین نروند.

اصل دهم: اشتباهات قبلی را دوباره تکرار نکنید.

نتیجه‌ی تلاش‌های قبلی را ثبت کنید و هنگام رفع خطاهای تکرارشونده، شواهد را بررسی کنید. اصلاح مکرر بدون شناخت علت اصلی، روش قابل اعتمادی نیست.

اصل یازدهم: محدودیت‌های ابزار را بپذیرید

ایجنت ممکن است اطلاعات کافی نداشته باشد، ابزار موردنیاز در دسترسش نباشد یا نتواند صحت نتیجه را به‌طور کامل بررسی کند. در چنین شرایطی، باید محدودیت را شناسایی کرد و از روش جایگزین مناسب استفاده کرد.

اصل دوازدهم: مسئولیت نهایی را حفظ کنید

ایجنت می‌تواند در تحلیل، تولید کد و انجام وظایف کمک کند؛ اما تصمیم درباره‌ی پذیرش تغییرات، انتشار برنامه و استفاده از داده‌های حساس همچنان نیازمند قضاوت و مسئولیت انسانی است.

خطا و محدودیت، بخشی از کار با ابزارهای مبتنی بر هوش مصنوعی است. ایجنت ممکن است درخواست را اشتباه تفسیر کند، کد ناقص تولید کند، وابستگی ناسازگار انتخاب کند یا تغییرات ناخواسته‌ای در پروژه ایجاد کند. بااین‌حال، شناخت این مشکلات و استفاده از روش‌های مناسب بررسی و اصلاح می‌تواند اثر آن‌ها را کاهش دهد.

برای کار مؤثر باAntigravity ، درخواست‌های روشن بنویسید، وظایف پیچیده را مرحله‌بندی کنید، نتایج را آزمایش کنید، تغییرات را با Git بررسی کنید و دسترسی‌های ایجنت را متناسب با نیاز پروژه محدود نگه دارید. در صورت بروز خطا نیز به‌جای تکرار بی‌هدف اصلاحات، علت اصلی را بررسی کنید و از نتایج تلاش‌های قبلی برای انتخاب راه‌حل بهتر استفاده کنید.

در نهایت، هدف از استفاده از ایجنت صرفاً سریع‌تر کردن کار نیست؛ بلکه باید کیفیت، قابلیت بررسی و ایمنی فرایند توسعه نیز حفظ شوند.

فصل نهم: جمع‌بندی و واژه‌نامه

مرور نکات کلیدی کتابچه

در فصل‌های پیشین، با Google Antigravity و شیوه‌ی استفاده از آن برای کمک به فرایند توسعه‌ی نرم‌افزار آشنا شدیم. هدف اصلی این کتابچه، معرفی قابلیت‌های این پلتفرم و توضیح روش استفاده‌ی آگاهانه و مؤثر از ایجنت‌های برنامه‌نویسی بود.

برای جمع‌بندی، مهم‌ترین نکات مطرح‌شده را مرور می‌کنیم.

۱ .شناخت ماهیت ایجنت

ایجنت برنامه‌نویسی صرفاً ابزاری برای پاسخ‌دادن به پرسش‌ها نیست؛ بلکه می‌تواند با توجه به درخواست کاربر، وظایف را برنامه‌ریزی کند، فایل‌های پروژه را بررسی کند، کد بنویسد یا تغییر دهد، ابزارهای در دسترس را به کار بگیرد و نتایج را بررسی کند. بااین‌حال، میزان استقلال و قابلیت‌های آن به ابزار، تنظیمات، مجوزها و محیط اجرا بستگی دارد.

۲ .نوشتن درخواست دقیق

کیفیت خروجی ایجنت تا حد زیادی به کیفیت درخواست کاربر وابسته است. درخواست مناسب باید هدف، محدوده‌ی کار، محدودیت‌ها و نتیجه‌ی مورد انتظار را مشخص کند. هرچه مسئله پیچیده‌تر باشد، تقسیم آن به وظایف کوچک‌تر و بررسی مرحله‌به‌مرحله‌ی نتایج اهمیت بیشتری پیدا می‌کند.

۳ .شناخت محیط کاری

برای کار مؤثر باAntigravity ، لازم است با بخش‌هایی مانند پنل ایجنت، ویرایشگر کد، ترمینال و فایل‌های پروژه آشنا باشیم. شناخت این محیط‌ها به ما کمک می‌کند بدانیم ایجنت چه کاری انجام می‌دهد و چگونه می‌توانیم نتیجه‌ی آن را بررسی کنیم.

۴ .استفاده‌ی هدفمند از ابزارها

ایجنت می‌تواند در انجام کارهایی مانند خواندن و ویرایش فایل‌ها، اجرای دستورات، بررسی خطاها و آزمودن برنامه کمک کند. ابزارهایی مانند Skills و اتصال‌های مبتنی بر MCP نیز در محیط‌هایی که از آن‌ها پشتیبانی می‌کنند، می‌توانند قابلیت‌های بیشتری فراهم کنند. بااین‌حال، پیش از استفاده از هر قابلیت باید نحوه‌ی عملکرد و سطح دسترسی آن را بشناسیم.

۵ .شخصی‌سازی رفتار ایجنت

تنظیمات، قوانین پروژه، فایل‌های راهنما و مهارت‌های قابل استفاده‌ی مجدد می‌توانند به ایجنت کمک کنند وظایف را مطابق نیاز پروژه انجام دهد. البته وجود یک فایل یا تنظیم خاص لزوماً به معنای استفاده‌ی خودکار ایجنت از آن نیست؛ پشتیبانی و نحوه‌ی فعال‌شدن هر قابلیت باید در نسخه‌ی مورد استفاده بررسی شود.

۶ .رعایت امنیت

دسترسی‌های ایجنت باید متناسب با نیاز واقعی پروژه باشند. اجرای دستورات ناشناخته، افشای اطلاعات محرمانه، اتصال سرویس‌های غیرقابل اعتماد و تأیید تغییرات بدون بررسی می‌توانند خطرآفرین باشند. استفاده از محیط آزمایشی، محدودکردن مجوزها و بررسی اقدامات حساس، بخش مهمی از کار با ایجنت است.

۷ .بررسی و آزمایش خروجی

تولید کد به‌تنهایی به معنای حل صحیح مسئله نیست. کد باید خوانده شود، اجرا شود و با تست‌های مناسب بررسی شود. همچنین، باید تغییرات فایل‌ها را بشناسیم و از ابزارهایی مانند Git برای پیگیری تغییرات و بازگشت به وضعیت قبلی استفاده کنیم.

در نهایت، مهم‌ترین اصل این است که "ایجنت می‌تواند در انجام کارها کمک کند، اما مسئولیت ارزیابی نتیجه و تصمیم‌گیری نهایی بر عهده‌ی کاربر باقی می‌ماند".

اصول استفاده‌ی مؤثر و ایمن ازAntigravity

برای استفاده‌ی بهتر ازGoogle Antigravity ، رعایت اصول زیر توصیه می‌شود.

۱ .پیش از شروع، هدف را مشخص کنید.

قبل از ارسال درخواست، روشن کنید که چه مشکلی باید حل شود و نتیجه‌ی مطلوب چیست. درخواست‌های مبهم ممکن است به تغییرات غیرضروری یا خروجی نامناسب منجر شوند.

۲ .پروژه را پیش از تغییر بشناسید.

اگر قصد دارید روی یک پروژه‌ی موجود کار کنید، ابتدا ساختار پوشه‌ها، فایل‌های اصلی، وابستگی‌ها و روش اجرای آن را بشناسید. از ایجنت بخواهید ابتدا پروژه را بررسی کند و بدون اجازه‌ی شما تغییری ایجاد نکند.

۳ .وظایف بزرگ را به مراحل کوچک تقسیم کنید.

به‌جای درخواست ساخت یک برنامه‌ی کامل در یک مرحله، کار را به بخش‌های کوچک‌تر تقسیم کنید. برای مثال، ابتدا ساختار برنامه را مشخص کنید، سپس قابلیت اصلی را پیاده‌سازی کنید و در پایان به سراغ تست و اصلاح بروید.

۴ .پیش از اجرای دستورات حساس، آن‌ها را بررسی کنید.

دستورات ترمینال ممکن است فایل‌ها را حذف کنند، تنظیمات را تغییر دهند، بسته‌های جدید نصب کنند یا اطلاعاتی را به سرویس‌های خارجی ارسال کنند. پیش از تأیید چنین دستوراتی، هدف و پیامد احتمالی آن‌ها را بررسی کنید.

۵ .تغییرات را به‌صورت منظم کنترل کنید

پس از هر مرحله‌ی مهم، فایل‌های تغییرکرده و تفاوت آن‌ها با نسخه‌ی قبلی را بررسی کنید. استفاده از Gitو ثبت تغییرات در نقاط مناسب، مدیریت خطاها و بازگردانی تغییرات را آسان‌تر می‌کند.

۶ .خروجی را آزمایش کنید.

تنها به توضیح ایجنت درباره‌ی موفقیت کار اکتفا نکنید. برنامه را اجرا کنید، تست‌های مناسب را انجام دهید و بررسی کنید که نتیجه با خواسته‌ی اولیه مطابقت دارد. اگر امکان اجرای واقعی وجود ندارد، این محدودیت را در ارزیابی نتیجه در نظر بگیرید.

۷ .اطلاعات محرمانه را محافظت کنید.

رمزهای عبور، کلیدهایAPI ، توکن‌های دسترسی و اطلاعات خصوصی نباید بدون ضرورت در اختیار ایجنت یا سرویس‌های خارجی قرار گیرند. فایل‌های حساس را از مخازن عمومی دور نگه دارید و تنظیمات دسترسی را با دقت بررسی کنید.

۸ .زمینه‌ی کافی در اختیار ایجنت قرار دهید.

ایجنت برای انجام درست کار به اطلاعات مرتبط نیاز دارد. توضیح ساختار پروژه، فناوری‌های استفاده‌شده، خطاهای مشاهده‌شده و محدودیت‌های مهم می‌تواند کیفیت پاسخ را افزایش دهد. در مقابل، اطلاعات نامرتبط یا دستورهای متناقض ممکن است موجب سردرگمی شوند.

۹ .تغییرات را مرحله‌به‌مرحله تأیید کنید.

بهتر است پس از هر مرحله‌ی مهم، نتیجه را بررسی کنید و سپس اجازه‌ی ادامه‌ی کار بدهید. این روش احتمال گسترش یک خطای اولیه به بخش‌های دیگر پروژه را کاهش می‌دهد.

۱۰ .مستندات رسمی را بررسی کنید.

ابزارهای هوش مصنوعی به‌سرعت تغییر می‌کنند. نام تنظیمات، مدل‌های در دسترس، روش نصب و قابلیت‌های اجرایی ممکن است در نسخه‌های مختلف متفاوت باشند. بنابراین، برای تصمیم‌گیری درباره‌ی قابلیت‌های فعلیAntigravity ، مستندات رسمی و اطلاعات نسخه‌ی نصب‌شده را در اولویت قرار دهید.

مسیر پیشنهادی برای استفاده‌ی روزمره

یک گردش‌کار ساده و قابل تکرار برای کار با Antigravity می‌تواند به این صورت باشد:

۱ .بررسی: ساختار پروژه و وضعیت فعلی آن را بشناسید.

۲. برنامه‌ریزی: از ایجنت بخواهید راه‌حل پیشنهادی و مراحل انجام کار را توضیح دهد.

۳ .اجرا: اجازه دهید وظیفه در محدوده‌ی مشخص‌شده انجام شود.

۴ .بازبینی: کدها، فایل‌های تغییرکرده و دستورات اجراشده را بررسی کنید.

۵ .آزمایش: برنامه یا تست‌های مربوط را اجرا کنید و نتیجه را بسنجید.

۶ .اصلاح: در صورت وجود خطا، پیام خطا و اطلاعات لازم را ارائه دهید و اصلاح را مرحله‌به‌مرحله انجام دهید.

۷ .ثبت: پس از اطمینان از نتیجه، تغییرات مناسب را با Git ثبت کنید.

این گردش‌کار قرار نیست تمام مراحل توسعه‌ی نرم‌افزار را ثابت و یکسان کند؛ بلکه چارچوبی برای حفظ کنترل، کاهش خطا و استفاده‌ی مسئولانه از ایجنت فراهم می‌کند.

مسیر پیشنهادی برای یادگیری بیشتر

برای یادگیری مؤثرAntigravity ، بهتر است از پروژه‌های کوچک و قابل کنترل شروع کنید و به‌تدریج پیچیدگی کارها را افزایش دهید. لازم نیست در ابتدای مسیر تمام قابلیت‌های ابزار را یاد بگیرید.

مرحله‌ی اول: آشنایی با محیط

در گام نخست، نصب و راه‌اندازی را انجام دهید و با محیط کار، پنل ایجنت، ویرایشگر، ترمینال و ساختار پروژه آشنا شوید.

تمرین پیشنهادی:

-یک پروژه‌ی آزمایشی ایجاد یا باز کنید.

-از ایجنت بخواهید ساختار آن را توضیح دهد.

-مشخص کنید فایل اصلی برنامه کدام است و برنامه چگونه اجرا می‌شود.

-بدون ایجاد تغییر، گزارش ایجنت را با فایل‌های واقعی پروژه مقایسه کنید.

مرحله‌ی دوم: انجام وظایف ساده

در این مرحله، درخواست‌های کوچک و مشخص مطرح کنید؛ مانند توضیح یک تابع، اصلاح یک خطای ساده یا افزودن یک قابلیت محدود.

تمرین پیشنهادی:

-یک برنامه‌ی ساده انتخاب کنید.

-از ایجنت بخواهید یک قابلیت کوچک به آن اضافه کند.

-تغییرات را بررسی کنید.

-برنامه را اجرا کنید و نتیجه را با خواسته‌ی اولیه مقایسه کنید.

مرحله‌ی سوم: یادگیری Git و مدیریت تغییرات

پس از آشنایی اولیه با ویرایش کد، یادگیری Git اهمیت زیادی دارد. این ابزار به شما امکان می‌دهد تغییرات را پیگیری کنید و در صورت نیاز به نسخه‌ی قبلی بازگردید.

تمرین پیشنهادی:

-یک مخزن Git برای پروژه‌ی آزمایشی ایجاد کنید.

-وضعیت پروژه را با `git status` بررسی کنید.

-تفاوت تغییرات را با `git diff` ببینید.

-پس از بررسی و آزمایش، تغییرات مناسب را ثبت کنید.

در این مرحله، یاد بگیرید که چگونه تغییرات ناخواسته را تشخیص دهید و پیش از بازگردانی آن‌ها، مطمئن شوید کار مهمی را از دست نمی‌دهید.

مرحله‌ی چهارم: انجام یک پروژه‌ی کوچک کامل

اکنون می‌توانید پروژه‌ای انتخاب کنید که چند بخش مرتبط داشته باشد؛ برای مثال، یک برنامه‌ی ساده‌ی مدیریت کارها، دفترچه‌ی یادداشت یا وب‌سایت شخصی.

در این مرحله تمرین کنید که چگونه:

-نیازمندی‌ها را مشخص کنید.

-وظایف را به مراحل کوچک تقسیم کنید.

-از ایجنت برای نوشتن و اصلاح کد کمک بگیرید.

-خطاها را با اطلاعات دقیق بررسی کنید.

-تست‌های پایه بنویسید یا اجرا کنید.

-تغییرات را با Git مدیریت کنید.

هدف این مرحله صرفاً تکمیل پروژه نیست؛ بلکه یادگیری فرایندی است که بتوانید آن را در پروژه‌های بعدی نیز تکرار کنید.

مرحله‌ی پنجم: یادگیری قابلیت‌های پیشرفته

پس از تسلط بر مراحل قبلی، می‌توانید به سراغ قابلیت‌هایی مانند قوانین پروژه، Skills، مدیریت زمینه، اتصال‌های MCP و خودکارسازی وظایف بروید.

پیشنهاد می‌شود هر قابلیت را جداگانه و ابتدا در یک پروژه‌ی آزمایشی بررسی کنید. هنگام اتصال سرویس‌های خارجی یا دادن دسترسی بیشتر به ایجنت، امنیت و مجوزهای موردنیاز را با دقت ارزیابی کنید.

مرحله‌ی ششم: تقویت دانش فنی مستقل

در کنار یادگیریAntigravity ، دانش برنامه‌نویسی خود را نیز توسعه دهید. یادگیری مبانی زبان برنامه‌نویسی، ساختار داده‌ها، دیباگ، تست، مدیریت وابستگی‌ها و Git به شما کمک می‌کند خروجی ایجنت را بهتر ارزیابی کنید.

اگر ایجنت کدی تولید کرد که دلیل کارکرد آن را نمی‌فهمید، از آن بخواهید کد را توضیح دهد، اما به توضیح بسنده نکنید؛ با مطالعه، آزمایش و تغییر کنترل‌شده‌ی کد، درستی توضیح را بررسی کنید.

"معیار پیشرفت واقعی این نیست که ایجنت چند خط کد تولید می‌کند؛ بلکه این است که شما بتوانید مسئله را تعریف کنید، راه‌حل را ارزیابی کنید، خطاها را تشخیص دهید و نتیجه‌ی قابل اعتمادی به دست آورید".

واژه‌نامه‌ی اصطلاحات تخصصی

در این بخش، اصطلاحات مهمی که در کتابچه به کار رفته‌اند، به‌صورت خلاصه توضیح داده می‌شوند. این واژه‌نامه برای مرور سریع مفاهیم و مراجعه‌ی دوباره هنگام کار با ابزار تهیه شده است.

۱ .ایجنت(Agent)

سامانه‌ای مبتنی بر هوش مصنوعی که می‌تواند برای رسیدن به هدفی مشخص، وظایف را برنامه‌ریزی کند و با استفاده از ابزارهای مجاز اقداماتی انجام دهد.

۲ .مدل هوش مصنوعی(AI Model)

مدلی که بر اساس داده‌های آموزشی، ورودی را پردازش می‌کند و خروجی‌هایی مانند متن، کد یا تحلیل تولید می‌کند. ایجنت می‌تواند از یک مدل برای استدلال و تولید پاسخ استفاده کند.

۳ .دستور یا درخواست(Prompt)

متنی که کاربر برای بیان هدف، ارائه‌ی اطلاعات و مشخص‌کردن محدودیت‌ها به مدل یا ایجنت می‌دهد.

۴ .زمینه(Context)

اطلاعاتی که هنگام انجام وظیفه در اختیار مدل قرار می‌گیرد؛ مانند درخواست کاربر، بخش‌هایی از کد، پیام‌های قبلی و اطلاعات مرتبط با پروژه.

۵ .محیط توسعه‌ی یکپارچه(IDE)

نرم‌افزاری که ابزارهایی مانند ویرایشگر کد، اجرای برنامه و امکانات کمکی توسعه را در یک محیط گرد هم می‌آورد.

۶ .فضای کاری(Workspace)

محیطی که کاربر در آن روی فایل‌ها و پروژه کار می‌کند. فضای کاری ممکن است شامل یک پوشه یا مجموعه‌ای از فایل‌های مرتبط باشد.

۷ .ویرایشگر کد(Code Editor)

ابزاری برای مشاهده، نوشتن و تغییر کدهای برنامه.

۸ .ترمینال(Terminal)

محیطی متنی برای واردکردن دستورها و اجرای ابزارهای سیستم یا پروژه.

۹ .مخزن(Repository)

محلی برای نگهداری فایل‌های پروژه و، در صورت استفاده از سیستم کنترل نسخه، تاریخچه‌ی تغییرات آن‌ها.

۱۰ .گیت (Git)

ابزاری برای کنترل نسخه که تغییرات فایل‌ها را پیگیری می‌کند و امکان مقایسه و بازگردانی نسخه‌های قبلی را فراهم می‌سازد.

۱۱ .ثبت تغییرات(Commit)

ثبت مجموعه‌ای از تغییرات در تاریخچه‌ی مخزن Git با یک پیام توضیحی.

۱۲ .تفاوت تغییرات(Diff)

نمایشی از تفاوت میان دو نسخه‌ی یک فایل یا مجموعه‌ای از فایل‌ها؛ برای مثال، نمایش خطوط اضافه‌شده، حذف‌شده یا تغییرکرده.

۱۳ .اشکال‌زدایی(Debugging)

فرایند پیدا کردن علت خطا یا رفتار نادرست برنامه و اصلاح آن.

۱۴ .آزمون نرم‌افزار(Testing)

بررسی برنامه برای ارزیابی عملکرد آن و تشخیص خطاها یا مغایرت‌ها با نیازمندی‌ها.

۱۵ .خروجی یا گزارش قابل بررسی(Artifact)

در ابزارهای ایجنتی، به خروجی‌هایی گفته می‌شود که برای مشاهده و بررسی کار انجام‌شده ارائه می‌شوند؛ مانند گزارش، طرح اجرا، نتیجه‌ی بررسی یا تغییرات پیشنهادی. معنای دقیق این اصطلاح به محیط و ابزار مورد استفاده بستگی دارد.

۱۶ .قوانین(Rules)

دستورالعمل‌هایی که برای هدایت رفتار ایجنت و مشخص‌کردن شیوه‌ی کار، محدودیت‌ها یا استانداردهای پروژه تعریف می‌شوند.

۱۷ .فایلAGENTS.md

فایلی متنی که می‌توان از آن برای مستندسازی راهنمای کار ایجنت در یک پروژه استفاده کرد؛ مانند روش اجرای برنامه، استانداردهای کدنویسی و محدودیت‌های مهم. نحوه‌ی شناسایی و استفاده‌ی خودکار از این فایل به ابزار و تنظیمات آن بستگی دارد.

۱۸ .مهارت(Skill)

مجموعه‌ای سازمان‌یافته از دستورالعمل‌ها و منابع برای کمک به ایجنت در انجام یک نوع وظیفه‌ی مشخص یا پیروی از یک گردش‌کار قابل استفاده‌ی مجدد.

۱۹ .پروتکل(Model Context Protocol) MCP

پروتکلی برای فراهم‌کردن روشی استاندارد جهت اتصال مدل‌ها و برنامه‌های هوش مصنوعی به ابزارها و منابع خارجی، در محیط‌هایی که از آن پشتیبانی می‌کنند.

۲۰ .رابط برنامه‌نویسی کاربردی(API) :رابطی که به نرم‌افزارها اجازه می‌دهد از طریق قواعد و درخواست‌های مشخص با یکدیگر ارتباط برقرار کنند.

۲۱ .افزونه(Extension)

قابلیتی تکمیلی که برای افزودن امکانات یا یکپارچه‌سازی ابزارهای دیگر به یک نرم‌افزار استفاده می‌شود.

۲۲ .محیط ایزوله(Sandbox)

محیطی محدودشده برای اجرای کد یا دستورها با هدف کاهش دسترسی به منابع دیگر و کنترل خطرهای احتمالی. میزان ایزوله‌سازی به پیاده‌سازی آن بستگی دارد و نباید آن را تضمینی مطلق برای امنیت دانست.

۲۳ .حداقل سطح دسترسی(Least Privilege)

اصل امنیتی‌ای که بر اساس آن، هر کاربر یا ابزار باید فقط به دسترسی‌هایی مجهز باشد که برای انجام وظیفه‌ی خود نیاز دارد.

۲۴ .تزریق دستور(Prompt Injection)

نوعی حمله یا تلاش برای فریب سامانه‌ی هوش مصنوعی از طریق واردکردن دستورهای گمراه‌کننده در متن، فایل یا محتوای خارجی، با هدف وادارکردن آن به نادیده‌گرفتن دستورالعمل‌های معتبر یا انجام اقدام ناخواسته.

۲۵ .وابستگی(Dependency)

کتابخانه، بسته یا ابزار دیگری که یک برنامه برای اجرا یا انجام بخشی از عملکرد خود به آن نیاز دارد.

۲۶ .خطای زمان اجرا(Runtime Error)

خطایی که هنگام اجرای برنامه رخ می‌دهد؛ برای مثال، تلاش برای دسترسی به فایلی که وجود ندارد یا استفاده از مقداری نامعتبر.

۲۷ .خطای منطقی(Logical Error)

اشکالی در منطق برنامه که ممکن است باعث شود برنامه اجرا شود، اما نتیجه‌ای نادرست تولید کند.

۲۸ .کنترل نسخه (Version Control)

روش و ابزارهایی برای ثبت، پیگیری، مقایسه و مدیریت تغییرات فایل‌ها در طول زمان Git .یکی از ابزارهای شناخته‌شده‌ی کنترل نسخه است.

۲۹. خودکارسازی(Automation)

استفاده از ابزارها و دستورالعمل‌ها برای اجرای یک فرایند یا مجموعه‌ای از وظایف با دخالت کمتر و در مواردی با نظارت انسانی.

۳۰. مدل استدلال(Reasoning Model)

مدلی که برای انجام وظایف نیازمند تحلیل چندمرحله‌ای و حل مسئله طراحی یا تنظیم شده است. قابلیت‌ها، محدودیت‌ها و گزینه‌های مربوط به استدلال در ابزارهای مختلف یکسان نیستند.

۳۱. کد منبع(Source Code)

متنی که برنامه‌نویس با یک زبان برنامه‌نویسی می‌نویسد و منطق و عملکرد نرم‌افزار را تعریف می‌کند.

۳۲. اشکال‌زدایی خودکار(Automated Debugging)

استفاده از ابزارها یا ایجنت برای کمک به شناسایی علت خطا، پیشنهاد اصلاح یا اجرای مراحل بررسی. نتیجه‌ی این فرایند همچنان باید ارزیابی و آزمایش شود.

۳۳. محیط اجرا(Runtime Environment)

مجموعه‌ی نرم‌افزارها، تنظیمات و منابعی که برنامه هنگام اجرا به آن‌ها نیاز دارد.

۳۴. مخزن عمومی(Public Repository)

مخزنی که محتوای آن برای عموم قابل مشاهده است. پیش از انتشار پروژه در چنین مخزنی، باید مطمئن شد که اطلاعات محرمانه، رمزها و کلیدهای دسترسی در فایل‌ها یا تاریخچه‌ی آن وجود ندارند.

۳۵. بازگردانی تغییرات(Reverting Changes)

فرایند بازگرداندن فایل‌ها یا پروژه به وضعیت قبلی یا خنثی‌کردن تغییرات ناخواسته. روش مناسب به ابزار مورد استفاده و وضعیت فعلی تغییرات بستگی دارد؛ بنابراین، پیش از اجرای دستورهای بازگردانی باید پیامد آن‌ها را بررسی کرد.

سخن پایانی

Google Antigravityو ابزارهای مشابه می‌توانند بخش‌هایی از فرایند توسعه‌ی نرم‌افزار را ساده‌تر کنند و به برنامه‌نویسان در بررسی پروژه، نوشتن کد، رفع خطا و انجام وظایف تکراری کمک برسانند. بااین‌حال، استفاده‌ی مؤثر از آن‌ها به شناخت ابزار، توانایی تعریف مسئله، آگاهی از اصول برنامه‌نویسی و دقت در ارزیابی نتایج وابسته است.

بهترین رویکرد آن است که از ایجنت برای افزایش توانایی‌های خود استفاده کنیم، نه اینکه مسئولیت یادگیری و تصمیم‌گیری را به‌طور کامل به آن واگذار کنیم. هرچه دانش فنی و مهارت ارزیابی ما بیشتر باشد، بهتر می‌توانیم از قابلیت‌های ایجنت بهره ببریم و خطر خطاها و تغییرات ناخواسته را کاهش دهیم.

این کتابچه نقطه‌ی شروعی برای آشنایی با Antigravity است. یادگیری واقعی زمانی شکل می‌گیرد که مفاهیم مطرح‌شده در یک پروژه‌ی عملی به کار گرفته شوند، نتایج بررسی شوند و تجربه‌های حاصل از خطاها به بخشی از مسیر یادگیری تبدیل شوند.

bitwithit